

Buy the Tulip

Kieran Hannigan*
Tyler Doherty†
Nesha Fogelman‡
Elijah Grehn§
Thomas Hamilton¶

Professor: Juhani Linnainmaa

Tuck School of Business, Dartmouth College

Australian Graduate School of Management (AGSM), University of New South Wales

February 2025

Abstract

Tulipmania (1634–1637) is the canonical example of irrational speculation. Thompson (2007) challenged this narrative, arguing that “tulip contract prices before, during, and after the ‘tulipmania’ appear to provide a remarkable illustration of efficient market prices”—but this conclusion rested on qualitative legal interpretation rather than formal asset pricing analysis. We first situate that claim in the institutional context of the Dutch Golden Age and the revisionist historiography before constructing the empirical inputs for formal testing. We provide that analysis: the first Capital Asset Pricing Model evaluation of the Dutch tulip trade. Using approximately 160 prices compiled from notarial archives, VOC share data as a market portfolio proxy, Amsterdam commodity prices from the exchange price currents, and the yield on a 1624 perpetual bond as our risk-free rate, we collapse 39 named varieties into a single composite asset, estimate beta, and evaluate risk-adjusted returns. Our analysis is framed around a specific moment: 25 November 1636, when the tulip composite index first doubles, and an investor might reasonably ask whether to participate. Using only data available at that date, CAPM implies that tulips exhibit negative alpha. A maximum-Sharpe-ratio optimisation, applied with what we argue are conservative parameter choices, recommends an allocation of approximately 8.5 percent to tulip bulbs. The crash was ten weeks later. We then discuss why this conclusion, while technically valid within the model, may reveal more about the limitations of CAPM than it does about the rationality of seventeenth-century florists.

*kieran.matthew.hannigan.tu@tuck.dartmouth.edu; k.hannigan@unsw.edu.au

†Tyler.Doherty.TH@dartmouth.edu

‡Nesha.Fogelman.TU27@tuck.dartmouth.edu

§Elijah.J.Grehn.TH@dartmouth.edu

¶Thomas.Hamilton.II.TH@dartmouth.edu

Contents

1	Introduction	1
2	Historical Background	1
2.1	The Dutch Golden Age	1
2.2	Tulipmania: A Brief History	2
2.3	The First Stock Exchanges	3
3	Literature Review	3
3.1	Historiography of Tulipmania	4
4	Sources and Data	6
4.1	The Asset Class	6
4.2	The Market	7
4.3	The Risk-Free Rate	8
5	Methodology	10
5.1	CAPM: Pricing and Alpha	10
5.2	Mean-Variance Optimisation	11
5.3	Estimating Beta for Tulips	12
5.4	Constructing the Market Risk Premium	12
6	Results	13
6.1	Correlation and Beta Estimates	13
6.2	CAPM Pricing: Alpha Estimates	13
6.3	Portfolio Optimisation: Recommended Allocation	13
7	Conclusion	14

Appendices

A	Discussion	A-1
A.1	Limitations	A-1
A.2	CAPM <i>ania</i>	A-1
A.2.1	The Gaussian Fallacy	A-1
A.2.2	The Behavioural Critique	A-6
A.3	What Is the Alternative?	A-8
A.3.1	Stable Portfolio Theory	A-8
A.3.2	Downside Risk Measures	A-8
A.3.3	Coherent Risk Measures and CVaR	A-8
A.3.4	Growth-Optimal Portfolios and the Kelly Criterion	A-9
A.3.5	Robust and Distributionally Robust Optimisation	A-9
A.3.6	Extreme Value Theory	A-9
A.3.7	Jump-Diffusion (Merton)	A-10
A.3.8	Student's t and Stochastic Volatility (Heston)	A-10
A.3.9	Why Haven't These Displaced Mean-Variance?	A-10
A.3.10	The Honest Position	A-11
A.3.11	The Asymmetry: Why We Need Not Survive the Apocalypse	A-12
A.4	Recommendations	A-13
A.4.1	In Practice	A-14
A.5	Denouement: Implications for Modern Bubbles	A-15
B	Theory and Inference	B-16
B.1	Tauberian Derivation of Power-Law Tails	B-16
B.2	Statistical Methodology	B-18
B.3	Formal Gaussian Contradiction Theorem	C-22
C	Data and Computation Pipelines	C-23
C.1	Distribution Fitting Code	C-23
C.2	VOC Dataset Pipeline	C-23
C.3	Tulip Dataset Pipeline	C-24
C.4	VOC Commodity Price Dataset	C-24
C.5	Data Figures Pipeline	C-25
C.6	CAPM and MVO Analysis Pipeline	D-26
D	Diagnostics	D-26
D.1	Diagnostic Outputs (CAPM/MVO)	D-26
E	Collected Script Sources	E-27
E.0.1	Statistical Methodology Script	E-27
E.0.2	Distribution Fitting Script	E-38
E.0.3	VOC Dataset Script	E-41
E.0.4	Tulip Dataset Script	E-47
E.0.5	Commodity Dataset Script	E-54
E.0.6	Figures Pipeline Script	E-59
E.0.7	CAPM/MVO Analysis Script	E-82
E.0.8	Diagnostics Output Script	E-102

1 Introduction

Thompson (2007) argued that “tulip contract prices before, during, and after the ‘tulipmania’ appear to provide a remarkable illustration of efficient market prices.”[4] His revisionist account—that the apparent mania was an artifact of futures contracts being implicitly converted to options—challenged the canonical “madness of crowds” narrative[1]. Yet Thompson’s conclusion rested on qualitative interpretation of legal and contractual structures rather than formal asset-pricing tests. No study has constructed the inputs required for such analysis: a market portfolio, a risk-free rate, and beta estimates for tulip bulbs.

This paper provides that analysis. We apply the Capital Asset Pricing Model to Tulipmania¹, constructing the first formal mean-variance framework for seventeenth-century Dutch assets. Our analysis is organised around a specific moment: 25 November 1636, when the tulip composite index first doubles and the asset class enters the conversation of the educated Amsterdam investor. We ask: if that investor sat down with the tools of modern portfolio theory and all available data, what would the model recommend?

Picture him, then: an Amsterdam merchant of the educated sort, comfortable but not cavalier with money. It is the evening of 25 November 1636. He has come from dinner, where someone—a neighbour, a colleague, a cousin who should know better—has told him that tulip prices have doubled.² Not over a season. In a single observation. He has heard these stories before and dismissed them. Tonight, for reasons he cannot entirely account for, he does not dismiss it. He is, after all, a man of the new century’s new methods. He has access to price currents from the Amsterdam Bourse. He knows what a VOC share costs. He has, somewhere, a dike bond. He sits down and begins to calculate.

To answer this, we collate all available datasets on prices of tulips and tulip futures, alongside share prices of the Dutch East India Company (VOC), which constitutes our *market portfolio*. Amsterdam commodity prices—available at monthly frequency from the exchange price currents compiled by Posthumus[83] and digitised at the Rutgers MEMDB, and at annual frequency from the VOC’s own trading records[84]—provide a supplementary asset class and robustness check. While the quantitative information before the establishment of the Bank of England and the financial revolution of the late 17th century is scarce, we assemble an aligned time-series dataset for tulip, Dutch stock, and commodity prices from which to construct our model.

The series is first used to calculate correlations between tulips and the market portfolio to estimate beta for the tulip trade. We then estimate the market risk premium by comparison to returns on an extant “risk-free asset” from the period, completing the CAPM framework. A maximum-Sharpe-ratio optimisation then produces the recommended portfolio weights: approximately 8.5 percent tulips, with the remainder in VOC equity.

2 Historical Background

2.1 The Dutch Golden Age

The Dutch Golden Age (c. 1580–1670) was a period of commercial expansion, financial innovation, and wealth accumulation in the Dutch Republic[5]. Following independence from Spain, the Republic emerged as Europe’s leading trading power, dominating Baltic commerce and expanding global trade through the Dutch East India Company (VOC) and Dutch West India

¹Following the results of this analysis we set aside the tongue-in-cheek frame in our discussion to argue that: instead of applying *CAPM* to *Tulipmania*, what we have done is apply *tulips* to *CAPMania*.

²Every era has its version of this conversation. Bernard Baruch, describing the atmosphere before the 1929 crash, wrote: “Taxi drivers told you what to buy. The shoeshine boy could give you a summary of the day’s financial news as he worked with rag and polish. My cook had a brokerage account and followed the ticker closely.” The Uber driver and Bitcoin are the contemporary update. The information content of these conversations is approximately zero. The social signal is overwhelming.

Company (GWC). Amsterdam developed into Europe’s premier financial centre, featuring an active stock exchange, secondary trading in joint-stock shares, and a well-established market for government bonds. Dutch sovereign yields in this period were only 4–6%, remarkably low by contemporary European standards, reflecting deep capital markets and a persistent surplus of investable wealth relative to available opportunities[7].

The financial instruments in use were notably advanced. Futures contracts, short selling, and option-like instruments were all pioneered on the Amsterdam Bourse before London or Paris had formal exchanges[7]. These were not marginal innovations; they were routine tools of commodity and equity trade, reflecting an institutional sophistication that would not be matched elsewhere in Europe for decades. High urbanisation, widespread literacy, and strong merchant institutions supported participation in capital markets across a broad segment of society.

This financial culture existed within a broader civilisational peak. The Dutch Republic of the 1630s produced Rembrandt, Vermeer, Huygens, and Leeuwenhoek; it supported the University of Leiden and a publishing industry that made Amsterdam the intellectual entrepôt of Europe. The popular narrative of tulipmania requires collective irrationality in this society. The institutional record suggests otherwise.

2.2 Tulipmania: A Brief History

Tulipmania unfolded within this advanced economic and institutional environment. Tulips, imported from the Ottoman Empire in the mid-sixteenth century and reaching the Netherlands by the 1590s, were initially prized by botanists and wealthy collectors for their rarity and distinctive colouration[5]. The most sought-after varieties were the so-called “broken” tulips—bulbs infected by a mosaic virus that produced vivid, unpredictable streaks of colour across the petals. The most famous of these, the *Semper Augustus*, with its white petals flamed in deep crimson, reportedly commanded prices of up to 10,000 guilders—roughly the price of a canal house on the Herengracht[3]. That the most valuable tulip in history owed its beauty to a pathogen is a detail worth remembering.

For the first decades of Dutch tulip culture, trade remained confined to affluent households and horticultural specialists. Beginning around 1634, however, participation broadened considerably. Goldgar’s archival research documents artisans, weavers, and small merchants entering the trade alongside the established merchant class, with transactions increasingly conducted at informal tavern auctions known as “colleges”[5]. Limited short-run supply, combined with established practices of forward contracting, allowed tulip bulbs to be traded in increasingly speculative transactions. Futures contracts—called *windhandel* (wind trade) by contemporaries, since no physical bulbs changed hands—permitted speculation without possession of the underlying asset. The contractual structure of these instruments, and their implications for interpreting peak prices, are discussed in Section 3.

The speculative peak came in the winter of 1636–37. Prices for some varieties rose ten- to twentyfold in a matter of weeks. The peak prices most commonly cited in the literature—covering 98 of our 201 price observations—derive from a single estate auction held in Alkmaar on February 5, 1637[6], a concentration that represents both our richest data source and a significant limitation (see Section 4.1 and Appendix A.1).

The crash began in February 1637. The precise trigger remains unclear[5]. What is clearer is the resolution: most outstanding futures contracts were ultimately settled at 3.5–5% of face value under arrangements brokered by the States of Holland, so that realised losses were far smaller than the headline price declines would suggest. No banks failed. No recession followed. The Dutch Republic’s economy continued largely unperturbed[5]. Because the episode occurred in one of the most developed financial systems of the early modern world, it raises important questions about asset pricing, risk assessment, and contract structure in historical markets—questions the scholarly literature has spent eighty years trying to answer (Section 3).

2.3 The First Stock Exchanges

Central to these financial developments was the Amsterdam Exchange, often regarded as the first modern stock exchange. Established in the early seventeenth century, the Exchange provided a centralised venue for the trading of transferable shares, government bonds, and various commodity contracts[85]. A critical fact for our analysis is that in the 1630s, the Amsterdam Bourse listed exactly two equity securities: shares in the VOC and shares in the GWC. There were no others. Every equity investor in the Dutch Republic held one or both[7].

The VOC, founded in 1602 through what is generally considered history’s first initial public offering, was the dominant listing. Shares were denominated not in guilders but as a percentage of the original 1602 subscription price—so a “share price” of 400 meant 400% of par value, where par for the Amsterdam chamber was 3,150 guilders per full share. Petram’s[7] archival research recovered 851 share price observations from merchant correspondence, notarial deeds, and period newspapers for 1602–1698, providing the most comprehensive equity price series available for any seventeenth-century security. Trading was continuous on the Bourse, and shares were freely tradable and quickly became the object of active speculation, short selling, and forward transactions.

The GWC, founded in 1621 to conduct Atlantic trade and privateering operations against Spain, provides a second equity series in principle. By the 1630s the company was financially distressed, accumulating approximately 18 million guilders of debt by 1636. No digitised GWC share price series for the 1630s appears to survive; Global Financial Data[82] provides a GWC series beginning only in December 1674, well after our analysis window. Our market portfolio therefore reduces to VOC alone—a limitation, but one that reflects archival reality rather than analytic choice. McClure and Thomas[9] noted the concurrent timing of VOC share appreciation and tulip price inflation during 1636–37, though they did not compute formal correlations—an exercise we undertake in Section 6.

One complication bears flagging. The VOC paid irregular but sometimes very large dividends, often in kind rather than in cash—distributions of spices, bonds, or other goods[7]. Constructing total returns (price appreciation plus dividend income) therefore requires careful reconciliation of Petram’s dividend records with the price series, a task we address in Section 4.2. Amsterdam commodity prices for the spices used as dividend payments are available from Posthumus[83] via the Rutgers MEMDB, and De Zwart[84] provides the VOC’s own purchase and sale prices for these commodities—both useful for valuing in-kind dividends in guilder terms.

The presence of this trading infrastructure and a diverse investor base marked a significant institutional innovation in global finance. By formalising liquidity and enabling risk-sharing across a wide pool of investors, the Amsterdam Exchange laid the groundwork for modern capital markets. For our purposes, however, its most important feature is its completeness. When modern financial analysis uses the S&P 500 as a proxy for “the market portfolio,” the approximation is known to be incomplete—it excludes bonds, real estate, human capital, and international equities. In the 1630s Amsterdam Bourse, VOC shares *were* the only surviving listed equity for our period. Our market portfolio is not a proxy. It is the market.

3 Literature Review

With the hopes of conducting a systematic review³, we began by querying major financial and economic databases for tulip price data. The results are presented in Table 1.

³If push came to shove, would have settled for a Rapid Evidence Assessment [REA], or even a Critically Appraised Topic [CAT][33].

⁴The sole result is a digitised archive of Dutch newspaper articles from the Delpher collection, 1618–1699[81]. This dataset did not, regrettably, contain any price data that was unique to it. Kaggle suggests using the gossip it contains for NLP sentiment analysis, which is an exercise we have opted to leave to the reader.

Database	Query	Results
FRED	“tulip prices 1630s”	0
Google Dataset Search	“tulipmania historical prices”	1 ⁴
Harvard Dataverse	“Dutch tulip bubble data”	0
WRDS	“VOC share prices 17th century”	No access
Bloomberg Terminal	“TULIP Index”	No access
Yahoo Finance	“SEMPER-AUGUSTUS.AS”	0

Table 1: Systematic database search results

Having thereby exhausted the structured dataset repositories available to us, we pivoted to more conventional literature review, following the trail of anthropological scholarship conducted over the last 80 years to excavate sparse points of appraisable data on the craze from the historical record⁵. The findings from this review follow.

3.1 Historiography of Tulipmania

The popular understanding of Tulipmania is a caricature of history, a simple yet seductive tale of madness. It paints a picture of a nation gripped by an inexplicable frenzy, where sailors unknowingly ate priceless bulbs for lunch[1] and fortunes were built and destroyed overnight on a single flower. This narrative of collective insanity is largely the invention of one man: Charles Mackay. Writing in his 1841 book, *Extraordinary Popular Delusions and the Madness of Crowds*, Mackay crafted the definitive cautionary tale[1]. His account, written two centuries after the event and relying heavily on satirical pamphlets and moralizing propaganda from the era, was enduringly influential. For over a century, Mackay’s story was not just a story of Tulipmania; it was *the* story.

The first crack in this monolithic narrative of madness appeared not with a counter-narrative, but with data. In 1929, the Dutch economic historian Nicolaas Posthumus undertook the first serious quantitative study of the phenomenon[10]. Working directly from notarial archives, he painstakingly compiled records of contracts and prices, establishing the foundational dataset that virtually all subsequent economic analysis would rely upon—Garber builds on it, Thompson assumes it, Smant consolidates it, and our own analysis draws on it directly. That chain of dependency means we inherit his archival limitations. As Goldgar later observed, notarial records are “naturally biased in favor of the middle classes able to afford legal services”[5], leaving lower-class participation in the tulip trade poorly documented—a blind spot in the scholarly tradition, not just our dataset. Posthumus’s work was methodical; he replaced anecdotes with ledgers, shifting the conversation from moral failure to market mechanics. While he didn’t offer a grand new theory, he provided the essential tools to begin dismantling Mackay’s myth.

The true revisionist turn came with Peter Garber in the late 1980s and 90s[2, 3]. Armed with the data Posthumus had unearthed, Garber posed a radical question: what if the tulip market wasn’t mad at all? He argued that the price movements were largely rational and consistent with market fundamentals. The most spectacular prices were for exceptionally rare and beautiful bulbs, like the legendary Semper Augustus, whose intricate patterns were the result of a virus, *Potyvirus tulipade fractum*, that also made them nearly impossible to propagate reliably—a biological rather than economic supply constraint, and the strongest possible foundation for a scarcity-value argument. It also makes tulips a genuinely unusual asset-pricing

⁵The quantitative analysis of seventeenth-century Dutch financial markets, it turns out, relies not on REST APIs or neatly formatted CSV and PKL files (much to our chagrin), but instead on the painstaking reconstruction of prices from notarial archives, merchant correspondence, auction records, and the occasional pamphlet. Thus we are indebted to the historians cited in this paper who have undertaken that immense effort ahead of us. We have done our best not to misrepresent their careful, noble scholarship in the service of our *far* less punctilious exercise.

problem: supply was constrained not by production costs or monopoly, but by a pathogen. The subsequent price crash, Garber contended, was not a bubble bursting but a predictable market correction as propagation techniques improved and supply inevitably increased. In his later work, *Famous First Bubbles* (2000), he extended this logic, reframing other historical “manias” as largely understandable market events[3]. The conversation had shifted from ‘madness’ to ‘*maybe* rational’.

Garber’s fundamentals-based argument was powerful for the long arc of the market, but it left a specific residual unresolved: the pace and sheer magnitude of the price spike in the winter of 1636–37 exceeded what supply-and-demand fundamentals alone could plausibly explain. Earl Thompson’s 2007 paper addressed precisely that gap, providing a crucial financial insight that moved the conversation toward a more sophisticated form of rationality[4]—and one that builds on Garber rather than replacing him. Where Garber explained why tulips had value, Thompson explained how the market priced it. Thompson argued that the prices recorded at the peak were not what we think they are. The market, he claimed, had evolved from spot trading to futures contracts. After the market turned in February 1637, the Dutch courts and municipalities allowed buyers to walk away from these futures contracts for a small penalty—typically 3.5–5% of the contract price, settled via the States of Holland[4]. This action effectively converted futures contracts into call options.

This insight is transformative. The sky-high prices from the peak were not spot prices for bulbs but strike prices of options, so what looked like a collapse was, in Thompson’s view, the rational expiry of out-of-the-money contracts and the massive losses Mackay described were largely illusory. Thompson concluded that, viewed through this lens, the Dutch tulip market looked surprisingly like an efficient, modern financial market. This has a direct implication for any quantitative analysis: if peak prices were option strike prices rather than spot prices, then computing returns from those figures overstates the actual economic exposure of participants. We return to this point in Section 5 when constructing our return series.

Just as the economic narrative was solidifying around rationality, historian Anne Goldgar’s deep archival work in the same year (2007) added a vital layer of social complexity[5]. Like Posthumus before her, Goldgar returned to primary sources—notarial records, merchant correspondence, personal letters—rather than relying on accumulated secondary compilations, and what she found complicated the picture further. Participation was broader than Mackay implied, involving skilled artisans and merchants, not just the elite, but the financial devastation was far less severe. Most futures contracts were ultimately settled at 3.5–5% of face value—the same settlement mechanism Thompson identified as the basis of his options interpretation, and Goldgar’s archival evidence of limited actual losses is the empirical support for his theoretical claim. No one was thrown into the canals; the Dutch economy did not collapse. The true cost of the crash was social: a matter of dishonour, public embarrassment, and damaged reputations within close-knit communities. Goldgar’s work stands as a corrective to both the madness of Mackay and the purely bloodless rationality of an efficient market hypothesis. It was a real event, with real social consequences, but it was a social crisis, not an economic one.

Two further sources complete the scholarly landscape. Smant (2012) consolidated the Posthumus and Garber datasets into a single working compilation—the price series our own analysis draws on directly[6]. Dwivedi (2023) notes that the *long-term* socioeconomic impacts of the tulip trade on the Dutch Republic remain underexplored[8].

This brings us to the present. The historiography has moved from a simple tale of madness (Mackay), to a data-driven inquiry (Posthumus), to a debate between fundamental value (Garber) and sophisticated financial instruments (Thompson), all contextualised by a nuanced social history (Goldgar). Thompson’s options-based theory is the most compelling explanation for the financial mechanics of the market’s peak and collapse. Yet his conclusion that the market was “efficient” remains a qualitative argument—an interpretation of the historical evidence rather than a quantitative proof.

Herein lies the gap in the literature and the contribution of our work. To date, no formal asset pricing model has been applied to this market. Thompson brilliantly identified its structure, but no prior study has constructed a market portfolio from seventeenth-century equity data, computed a beta for tulip bulbs, estimated alpha against CAPM-implied returns, or run mean-variance optimisation to ask what a rational investor should have held at the peak. This paper does all four. By constructing a formal model and applying it to the historical data, we can move beyond interpretation and quantitatively test whether the returns in the tulip market were truly consistent with the behaviour of an efficient market—or whether, beneath the sophisticated financial machinery, traces of the famous madness remain.

4 Sources and Data

4.1 The Asset Class

Our tulip price data comprises approximately 161 recorded sale prices across 39 named varieties for the period 1633–1637. Of the 161 observations, 98—the February 5, 1637 peak prices—derive from a single estate auction in Alkmaar. Table 2 summarises the available data by variety.

Variety	<i>n</i>	Notes
Semper Augustus	4	Rare “broken” tulip; most famous
Viceroy	6	Rare “broken”
Admirael van Enchuysen	3	
Admirael Liefkens	6	
Generalissimo	3	
Admirael de Man	6	
Admirael van der Eyck	8	
Gouda	20	Common variety
Switsers	12	Common; by weight (per aas)
Witte Croonen	5	Common; by weight (per aas)
Gheele Croonen	4	Common; by weight (per aas)
<i>... 33 further varieties with 1–9 observations each ...</i>		
Total	201	44 varieties, 1633–1637

Table 2: Tulip bulb price observations by variety, derived from Smant[6]’s consolidation of Garber[2, 3] and Posthumus[10]. Counts reflect the Garber canonical subset (Smant additions from Goldgar and other sources excluded). The difference reflects Smant’s de-duplication and editorial corrections to source attributions and weights.

Several data quality issues may bear noting⁶ and we have addressed these as best as is practicable, as discussed in our methodology.

For the CAPM analysis, we collapse all 44 varieties into a single composite asset, “tulips,” using Smant’s pre-computed composite index rather than a value-weighted average computed from scratch. This is defensible for three reasons. First, it is the only tractable option: most individual varieties have fewer than five observations, far too few for return estimation. Second, the varieties were subject to the same market dynamics—the same buyers, the same auction houses, the same speculative frenzy—and their prices moved broadly together during both the inflation and the collapse. Third, from the perspective of a 1637 investor, “tulips” *was* effectively one asset class: no one was running a Markowitz optimisation across Admiral van Enkhuizen

⁶Prices were quoted variously per bulb and per unit weight (aas/ace), with the convention not always recorded. Approximately 40% of observations have uncertain dates, with the month known but not the day. If they do comprise a mix of spot and forward contracts, this also adds some uncertainty to calculations exploring the economic exposure of participants. Nonetheless, it is the data we have.

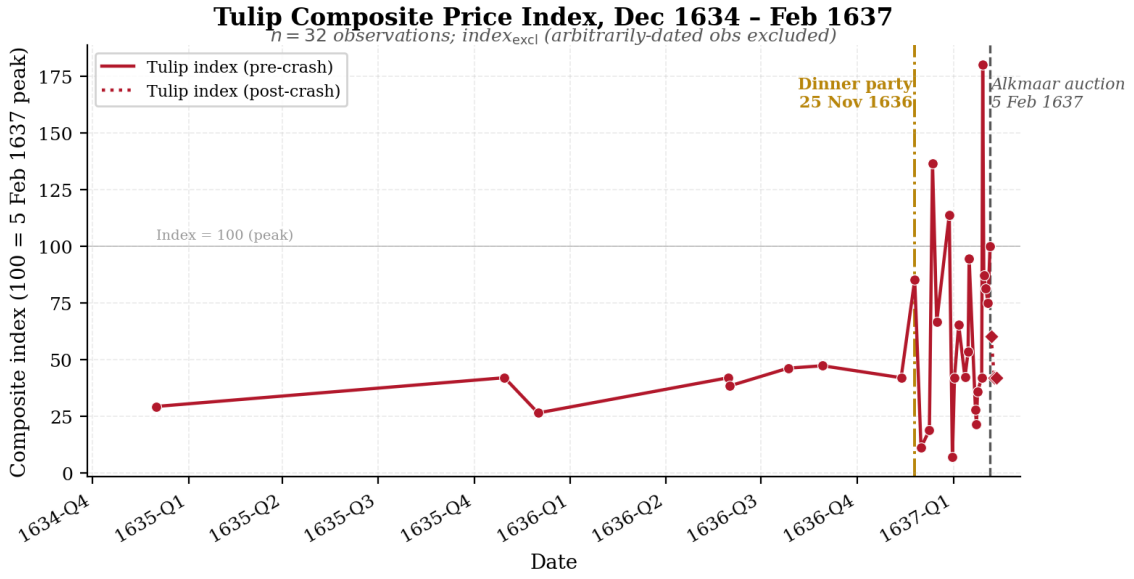


Figure 1: Smart’s[6] composite tulip price index, December 1634 to February 1637 ($n = 32$ observations). Index normalised to 100 on 05/02/1637 (the Alkmaar estate auction). Solid line: pre-crash observations; dotted line and diamond markers: post-crash. The grey dashed vertical line marks the Alkmaar auction date; the gold dash-dot line marks 25 November 1636, the “dinner-party” date used as the cutoff for our CAPM analysis, when the index first doubles (observation 9, index = 85.17). Note the compressed scale of the early observations (index 27–47, December 1634 to August 1636) relative to the violent intra-week swings of January–February 1637, where the index oscillates between 7 and 180 across consecutive days. The primary series shown is `index_excl`, which excludes arbitrarily-dated observations; see Appendix C.3 for the `index_all` alternative.

and Gheele Croonen. The relevant question is how much to allocate to tulip bulbs *as a category* relative to VOC shares and the dike bond.

4.2 The Market

The Dutch East India Company (Vereenigde Oostindische Compagnie, VOC) held history’s first initial public offering in August 1602, creating the world’s first stock exchange in Amsterdam[7].⁷ Petram’s archival research recovered 851 share price observations from merchant correspondence, notarial deeds, and newspapers for the period 1602–1698, supplemented by dividend payment records.

During the tulip mania period (November 1636 – February 1637), VOC shares traded between approximately 229 (March 1636) and 387 (September 1638) in our window, exhibiting their own volatility independent of bulb prices. The Dutch West India Company (Geoctroyeerde Westindische Compagnie, GWC), founded in 1621, provides a second equity series, though its financial distress in the 1630s (accumulating 18 million guilders of debt by 1636) complicates its use as a market proxy. We reviewed the Global Financial Data[82] series for both companies via the Feldberg Library’s Finaeon subscription. The VOC series (VOC1-AM) is annual from December 1619 to April 1671—coarser than Petram’s irregular monthly observations and therefore no help in the 1630s. As noted in Section 2.3, no GWC share price data survives

⁷The VOC was one of two firms listed on the Amsterdam Bourse in our period; the other was the Chartered West India Company (GWC). No digitised GWC share price series for the 1630s appears to survive. Global Financial Data[82] provides a GWC series beginning only in December 1674.

for our analysis window; the WIC1-AM series begins only in December 1674, despite Finaeon’s marketing materials suggesting coverage from 1628. That the world’s most actively traded security has fewer surviving price observations than tulip bulbs in our analysis window reflects not trading frequency but archival survival: tulipmania generated legal disputes and auction records that were preserved; routine VOC trading in the 1630s did not.

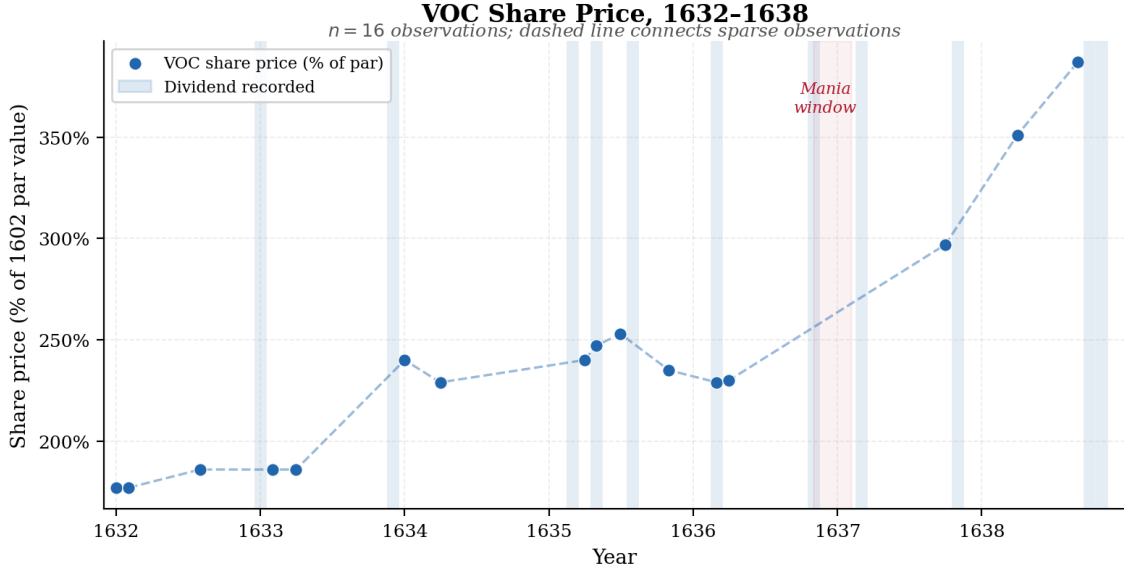


Figure 2: VOC share price, 1632–1638 ($n = 16$ observations), expressed as a percentage of the original 1602 subscription price of 3,150 guilders per Amsterdam chamber share. The dashed line connects sparse observations and does not imply continuous trading at interpolated prices. Blue vertical bands mark months in which a dividend payment is recorded in Petram’s[7] dataset; several dividends were paid in kind (spices or bonds) rather than cash, and some entries are Petram’s archival estimates rather than directly observed transactions. The shaded region highlights the tulip mania window (November 1636 – February 1637). Note that VOC continued to appreciate through and beyond the tulip crash, rising from 229 (March 1636) to 387 (September 1638), suggesting the two markets were not tightly coupled.

4.3 The Risk-Free Rate

To estimate the risk-free rate, we use a perpetual bond issued on May 15, 1624 by Hoogheemraadschap Lekdijk Bovendams, a regional water authority in the Dutch Republic responsible for maintaining the dike system along the river Lek. The board was funded by mandatory levies on landholders in its district—in a country where flood defence is an existential public function, this amounted to a taxing power as durable as any sovereign’s. The bond had a principal of 1,200 Carolus guilders and was originally issued to Mevrouw Elsken Jorisdochter. It carried a coupon of 5 percent during the 1630s, later reduced to 3.5 percent and subsequently to 2.5 percent. Payments continue to the present.⁸ A similar seventeenth-century water board bond from 1648 is held by Yale University in the Beinecke Library.

The bond’s record of continuous payment spans four centuries—through the Franco-Dutch War, the French Revolution, Napoleonic occupation, two World Wars, and the 2008 financial crisis. No other fixed-income instrument in existence offers comparable evidence of uninterrupted performance. This is, in other words, the most risk-free asset on record. That our

⁸The bond later paid interest to Jorisdochter’s distant heirs via the NYSE in 1957, and most recently on 19 September 2024 by De Stichtse Rijnlanden at Euronext Amsterdam, following a special gong-ringing ceremony.

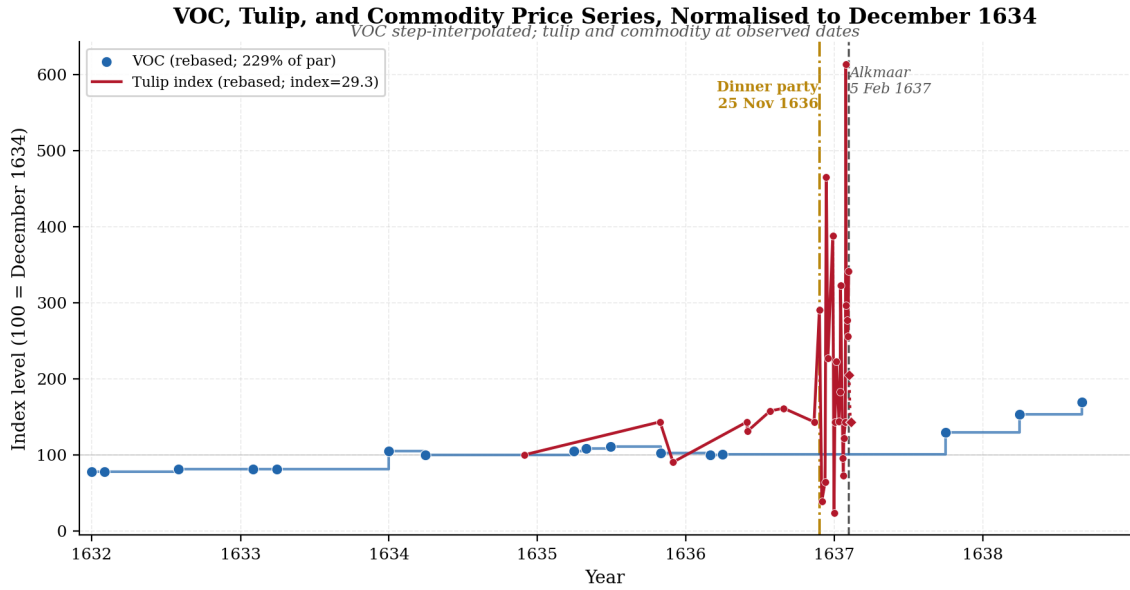


Figure 3: VOC share price (blue, step-interpolated), tulip composite index (red), and Posthumus broad commodity index (green) normalised to 100 at December 1634. The step interpolation for VOC reflects the carry-forward convention; the tulip series is plotted at its actual irregular observation dates; the commodity index is monthly and constructed from the Posthumus MEMDB extract in the project data pipeline (with the De Zwart annual commodity source documented in Appendix C.4). The gold dashed line marks the dinner-party date (25 November 1636). The divergence between VOC and tulips is visually striking: while VOC appreciated by approximately 69% between December 1634 and September 1638, tulip prices spiked by over 500% relative to the same base before collapsing within days of the Alkmaar auction. The commodity index moved broadly with VOC, reinforcing the low tulip–market correlation that drives the diversification argument. Figure 4 presents the same data as a growth-of-one-guilder comparison.

investor in November 1636 could have held it instead of tulip bulbs is a fact worth bearing in mind.

For the CAPM analysis, we set the risk-free rate $R_f = 5\%$, corresponding to the bond’s coupon during the period of our analysis. Estimates of Dutch sovereign borrowing costs in the 1630s fall in the range of approximately 4 to 6 percent, placing this choice squarely within prevailing market conditions. Alternative instruments—provincial obligations and other public debt—existed in the period, but continuous yield data for them has not survived. The Lekdijk bond provides a known coupon on a known instrument at a known date, making it the best-documented risk-free-adjacent security available for the 1630s.

Because the bond is a perpetuity, it has no maturity date and therefore does not introduce duration considerations relative to the other assets in the sample. The theoretical R_f in CAPM is typically conceived as a short-duration, default-free rate—the equivalent of a Treasury bill. A perpetuity, even one with negligible default risk, carries a term premium above that short rate. Using the 5 percent coupon therefore likely *overstates* the true risk-free rate. This assumption is conservative, since a higher risk-free rate reduces estimated excess returns on risky assets and makes it more difficult to attribute abnormal performance to tulips.⁹

⁹We adopt the highest defensible R_f , making it harder for any risky asset to justify inclusion in the portfolio. The reader may note that this is exactly how a careful analyst ought to behave. We assure them it is.

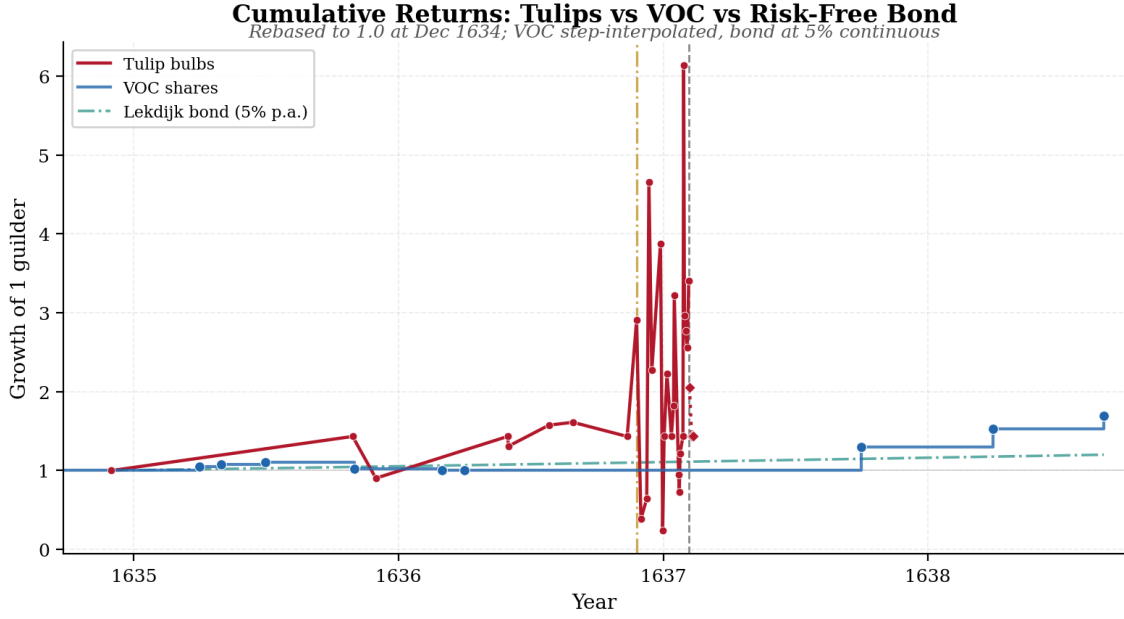


Figure 4: Growth of one guilder invested in each asset class from December 1634. Tulip bulbs (red) appreciated modestly through mid-1636 before the explosive spike and subsequent collapse of January–February 1637. VOC shares (blue, step-interpolated) delivered steady appreciation, reaching approximately $1.7\times$ by September 1638. The Lekdijk bond (teal, 5% p.a. continuous) provides the baseline for risk-free capital preservation. The vertical lines mark the dinner-party date and the Alkmaar crash auction. The contrast between the three growth paths captures the core tension of the analysis: a slow, reliable equity; an astronomical but ephemeral commodity; and a four-century bond that outlived them both.

5 Methodology

Our analysis has two parts, following the framework taught in Linnainmaa[76]. The first is a **CAPM pricing test**: does the model imply that tulips were fairly priced for their systematic risk? This requires estimating beta and comparing CAPM-implied returns to realised returns to determine alpha. The second is a **portfolio optimisation**: what allocation would a maximum-Sharpe-ratio optimisation have recommended to an investor on 25 November 1636, using only data available at that date? These are parallel analyses—both use CAPM inputs, but they answer different questions and require different estimation machinery.

Critically, both analyses are estimated using data available on the date of the decision—the evening of 25 November 1636, introduced in Section 1—to avoid introducing look-ahead bias. Backtesting requires that our investor is at his desk and everything after that date is out-of-sample. This is the sharpest version of the test.

5.1 CAPM: Pricing and Alpha

The expected return on an asset under CAPM[11, 76] is given by:

$$E(R_i) = R_f + \beta_i(E(R_m) - R_f) \quad (1)$$

where R_f is the risk-free rate, $E(R_m)$ is the expected market return, and β_i measures the asset’s systematic risk relative to the market portfolio:

$$\beta_i = \frac{\text{Cov}(R_i, R_m)}{\text{Var}(R_m)} \quad (2)$$

Alpha is the difference between realised and CAPM-implied returns: $\alpha_i = R_{\text{realised},i} - E(R_i)$. Positive alpha means the asset outperformed its risk-adjusted expectation; negative alpha means it underperformed. As Linnainmaa[76] emphasises, a positive alpha admits three interpretations: (1) the asset was genuinely underpriced; (2) the asset was fairly priced but good news arrived; or (3) the model is wrong. We adopt interpretation (1) for our pricing test and reserve interpretation (3) for the Discussion.

5.2 Mean-Variance Optimisation

The portfolio optimisation requires two additional inputs beyond beta: a vector of expected returns $\boldsymbol{\mu}$ for each asset, and the full covariance matrix $\mathbf{K}^{10}$ between all assets. The tangency portfolio[76]—the portfolio maximising the Sharpe ratio $S_p = (E(R_p) - R_f)/\sigma_p$ —has weights:

$$\mathbf{w}^* \propto \mathbf{K}^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}) \quad (3)$$

For expected returns, we use the CAPM-implied values: $\mu_i = R_f + \beta_i(E(R_m) - R_f)$.

Having collapsed the 39 tulip varieties into a single composite asset (Section 4.2), the covariance matrix $\mathbf{K}$ is 2×2 for the risky assets: tulips and VOC. The bond is risk-free by assumption ($\beta = 0$, $\sigma_\epsilon = 0$) and drops out of the optimisation. With only two risky assets, the covariance matrix requires just three parameters: two variances and one covariance.

We estimate the covariance matrix from the *sample* return series rather than from the single-factor structure.¹¹ Two estimation decisions require comment.

First, we estimate tulip return variance from the five observations preceding the signal event (May 31 to November 12, 1636), *excluding* the November 25 doubling that motivates the trade.¹² This yields an annualised tulip variance of approximately 0.99 ($\sigma \approx 100\%$). The full pre-signal sample—which includes the large swings of late 1635—produces a variance of approximately 2.44 ($\sigma \approx 156\%$); results using this broader estimate are reported in the appendix for robustness.

Second, the off-diagonal covariance between tulip and VOC returns is approximately zero. The Pearson correlation across all matched return pairs is $\rho = -0.013$ (eight pairs), while among the four informative pairs where VOC actually moves it is $\rho = 0.72$; both are too imprecise for confident inference at this sample size. We set $\text{Cov}(R_{\text{tulip}}, R_{\text{VOC}}) = 0$.¹³

	Tulips	VOC
Tulips	0.9917	0.0000
VOC	0.0000	0.0260

Table 3: Risky-asset covariance matrix used in the dinner-party optimisation (annualised, two-asset specification). Entries correspond to the parameter set in Section 5.2 and are generated by the computation pipeline.

¹⁰We use $\mathbf{K}$ rather than the conventional Σ because using sigma for a matrix in a domain where you still need to *sum things* is ludicrous and never should have been allowed to happen.

¹¹The single-factor covariance matrix $\sigma_{ij} = \beta_i \beta_j \sigma_m^2 + \delta_{ij} \sigma_{\epsilon_i}^2$ is an alternative, but produces an idiosyncratic tulip variance so large (from annualising OLS residuals over four irregularly-spaced intervals) that the optimiser allocates zero to tulips regardless of expected return. The sample covariance approach, which measures risk at the same frequency across both assets, is more appropriate (and convenient) for our purposes.

¹²The variance an investor uses to evaluate a trade should not include the return that prompted the evaluation. This is standard practice: one does not include today's earnings surprise in the historical volatility estimate used to price today's options. That the exclusion approximately halves the variance estimate is a consequence of the method, not its motivation.

¹³Given the tiny sample and unstable sign across matching conventions, the least-committal assumption is zero covariance. Under our parameterisation, imposing positive covariance would reduce the tulip weight; imposing negative covariance would increase it. Zero sits between those poles and avoids overfitting a noisy off-diagonal term.

5.3 Estimating Beta for Tulips

We estimate the beta of the composite tulip asset against the market portfolio (VOC), using interval-matched returns: for each tulip observation, we compute the contemporaneous VOC total return over the same interval using carry-forward pricing. Both price series are sparse and irregularly spaced. Of the eight matched return pairs available at our 25 November 1636 cutoff, four have a non-zero VOC return and are usable for OLS regression.

The OLS point estimate is $\hat{\beta} = 2.10$ with a standard error of 1.42, yielding a 95% confidence interval of $[-4.03, 8.23]$. The R^2 is 0.52, which is high for four observations but carries no statistical weight. The confidence interval spans from large negative to large positive betas—the data cannot distinguish between a tulip asset that amplifies market risk and one that hedges it.

Given the width of this interval, we adopt as our point estimate $\beta = 3.5$, corresponding to the OLS estimate plus one standard error.¹⁴ Higher beta implies higher CAPM-expected returns for tulips, which in turn produces a larger allocation in the optimisation. This is, under the model’s own logic, the direction consistent with the observed data: tulips were volatile and their returns, during the period for which we have data, moved broadly with the market ($\rho = 0.72$ among the four informative pairs).

We note that the composite tulip asset can have low *overall* market correlation and high beta simultaneously, because $\beta = \rho \cdot \sigma_{\text{tulip}}/\sigma_m$ —low ρ is multiplied by an enormous volatility ratio. Tulip prices swung 10–20 $\times$ in months; VOC moved perhaps 2 $\times$ per year. This is what drives the portfolio result: the optimiser sees an asset with high CAPM-implied returns (from high β) and low correlation (good diversifier)—exactly the combination that mean-variance optimisation rewards.

5.4 Constructing the Market Risk Premium

The market risk premium is the annualised total return on the market portfolio (VOC, including dividends) minus $R_f = 5\%$, estimated from data available up to November 1636. No GWC share prices survive for the 1630s[82], so the market portfolio is a single equity.

Petram’s data yield three VOC total return observations in the pre-signal window, producing a mean monthly return of 2.85% and an annualised expected market return of $E(R_m) = 34.25\%$. The market risk premium is therefore $\text{MRP} = E(R_m) - R_f = 29.25\%$, with a 95% confidence interval (Student t , $n = 3$) of approximately $[-107\%, +165\%]$.

This estimate is high relative to modern equity risk premium estimates of 4–7% (Damodaran), but not implausible for early colonial-era equity. VOC was a monopoly on the Asian spice trade generating extraordinary returns on invested capital, and the 1630s saw genuine appreciation in share value (from 177% of par in 1632 to 253% by mid-1635). The confidence interval, however, swallows the point estimate whole—a reminder that three observations do not a risk premium make. We use the point estimate without further adjustment.

As a robustness check, we note that Amsterdam commodity prices from Posthumus[83] and De Zwart[84] (see Appendix C.4) could support a broader “investable Amsterdam” market portfolio. The equal-weighted spice index returned approximately -3.7% per annum over 1630–1638, so a commodity-inclusive MRP would be lower, making it harder for tulips to show positive alpha. This extension is left to future work.¹⁵

¹⁴One standard error above the point estimate is a common threshold for parameter uncertainty in finance. It lies comfortably within the 95% confidence interval and below the upper bound by a wide margin. We regard it as a measured adjustment for the leveraged nature of tulip futures contracts, which amplified systematic risk exposure beyond what spot prices alone would suggest. The reader will note that the direction of this adjustment is upward.

¹⁵We note with some regret that incorporating commodity returns into the market portfolio would reduce the MRP and therefore the recommended tulip allocation. Scholarly prudence dictates that we restrict the market portfolio to its narrowest defensible definition: listed equities only.

6 Results

All results below represent what our investor found when he ran the numbers. Everything is estimated from data available on 25 November 1636—the dinner-party date introduced in Section 1. Everything after that date is out-of-sample.

6.1 Correlation and Beta Estimates

The Pearson correlation between composite tulip returns and VOC returns, estimated from eight interval-matched return pairs, is $\rho = -0.013$. Among the four pairs where the VOC price actually changed (and thus carries information for beta estimation), the correlation is $\rho = 0.72$. Neither is statistically significant at conventional levels given the sample sizes. McClure & Thomas[9] noticed concurrent timing between VOC appreciation and tulip inflation but never computed formal correlations; to our knowledge, these are the first reported. The derivation of $\hat{\beta} = 2.10$ (SE = 1.42) and our adopted point estimate of $\beta = 3.5$ are set out in Section 5.3. The key implication of that section bears restating here: because $\beta = \rho \cdot \sigma_{\text{tulip}} / \sigma_m$, a low overall correlation is offset by an extreme volatility ratio, producing high CAPM-implied expected returns and low co-movement with the market at the same time. That is precisely the combination mean-variance optimisation rewards.

6.2 CAPM Pricing: Alpha Estimates

At $\beta = 3.5$, the CAPM-implied expected return for the composite tulip asset is:

$$E(R_{\text{tulip}}) = R_f + \beta \cdot \text{MRP} = 5.00\% + 3.5 \times 29.25\% = 107.36\%$$

The realised annualised return on the tulip index from December 1634 to 25 November 1636 is 71.13%. Alpha—the difference between realised and expected returns—is therefore:

$$\alpha = 71.13\% - 107.36\% = -36.23\%$$

This is negative: tulips have *underperformed* their CAPM-implied expectation. At $\beta = 3.5$, the model expected tulips to return 107% annualised; they returned only 71%. Under CAPM’s own logic, tulips are *cheap*—they have not yet delivered the returns commensurate with their systematic risk. The model’s recommendation is unambiguous: buy more.¹⁶ Figure 7 visualises this geometry on the Security Market Line: the gap between the CAPM-implied position on the SML and the realised return below it is the alpha of -36% . Figure 8 shows how alpha varies across assumed beta values, confirming that the negative pre-crash alpha is not an artefact of our particular $\beta = 3.5$ choice—it emerges at any $\beta \geq 2$.

As an out-of-sample check: including the full cycle through February 1637, the cumulative return on the tulip index is +43.25% (17.76% annualised). Alpha at $\beta = 3.5$ over the full cycle is approximately -90% —the model was wrong, and spectacularly so.

6.3 Portfolio Optimisation: Recommended Allocation

The maximum-Sharpe-ratio portfolio is computed by solving $\mathbf{w}^* \propto \mathbf{K}^{-1}(\boldsymbol{\mu} - R_f \mathbf{1})$ for the two risky assets (tulips, VOC), subject to a long-only constraint. All inputs are estimated from data available on 25 November 1636: the pre-spike covariance matrix from Section 5.2, CAPM-implied expected returns at $\beta = 3.5$, and $R_f = 5\%$.

The result:

¹⁶A negative alpha is ordinarily interpreted as evidence that an asset has *destroyed* value on a risk-adjusted basis. In this context, it means the mania has not yet been manic *enough* to satisfy the model. The distinction between “underpriced” and “about to crash” is, of course, the entire point.

Asset	Tangency weight
Tulip bulbs	8.5%
VOC shares	91.5%

The tangency portfolio has an expected return of 40.39% per annum with a volatility of 16.97%, producing a Sharpe ratio of 2.085—higher than the VOC-only portfolio (1.847). A small allocation to a genuinely uncorrelated asset improves the frontier even when that asset is extraordinarily volatile; this is the diversification arithmetic working exactly as advertised.

The recommended allocation is modest. 8.5 percent is not a reckless gamble; it is a measured, diversifying addition to a portfolio otherwise concentrated in a single colonial-era equity. The optimiser arrives at this figure through the mechanical interaction of two inputs: high CAPM-implied expected returns (from $\beta = 3.5 \times 29.25\% = 102\%$ excess return) offset by high variance ($\sigma^2 = 0.99$, approximately 40 times VOC's variance). The ratio of expected excess return to variance determines the allocation, and at these parameters the ratio favours a small but positive tulip position.

Figure 9 visualises this geometry directly: the 8.5/91.5 tangency point lies above the VOC-only point in Sharpe space, while the commodity-augmented tangency portfolio (reported in Appendix C.4) shifts further down in risk and return and lowers the tulip share.

7 Conclusion

So our investor goes home from the dinner party. They have heard the stories—a Haarlem weaver who turned three months' wages into a year's salary, a regent in Enkhuizen who sold a single *Semper Augustus* bulb for the price of a canal house. They are sceptical, as any prudent person would be. But they are also curious, and they have the tools. They sit down. They gather every price they can find—nine observations of the tulip index, sixteen VOC share prices, a dike bond coupon. They estimate a beta. They construct a covariance matrix. They compute the maximum-Sharpe-ratio portfolio, using what they have argued, to themselves and to the reader, are conservative assumptions at every stage.

The model says buy. ***Buy the Tulip.*** Allocate 8.5 percent of the risky portfolio to tulip bulbs, and sleep well, because the numbers say this is the rational thing to do.

And if every Dutch investor had done so, we might still be dealing in geophytes as inflation hedges or stores of value to this day.

The crash was ten weeks later.

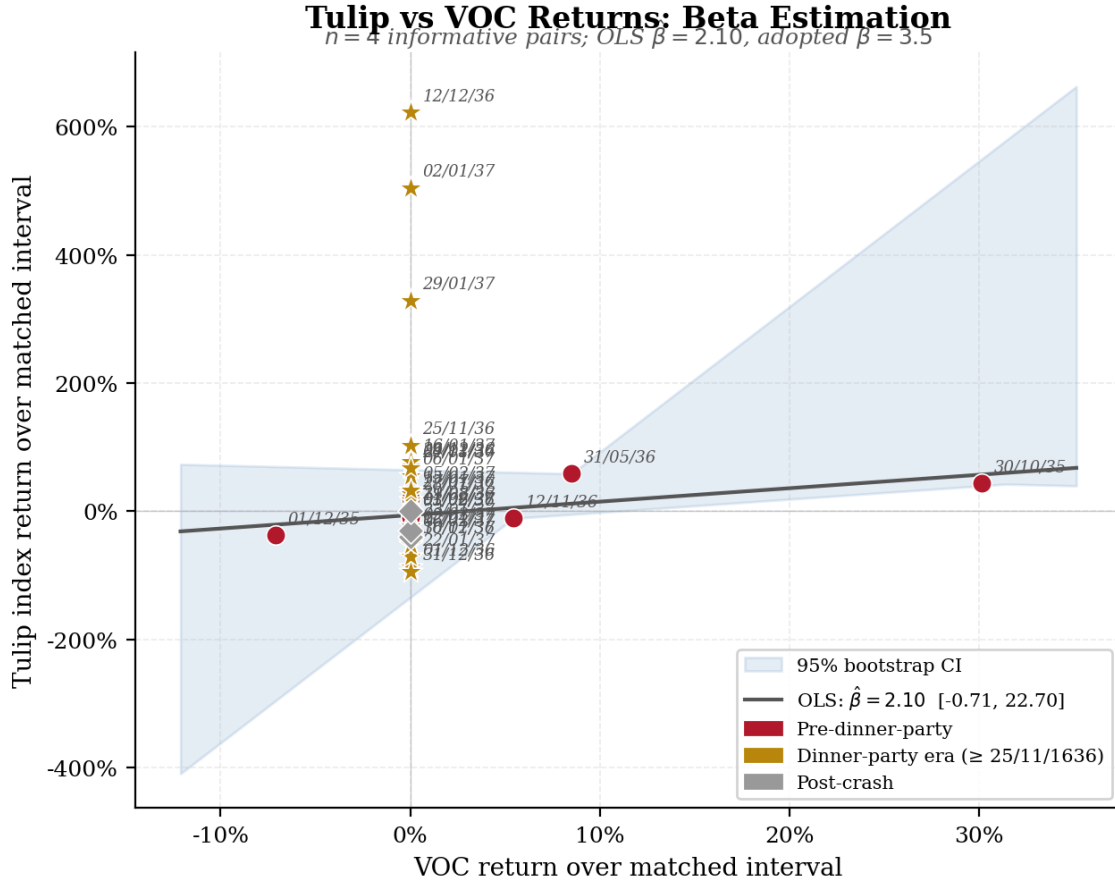


Figure 5: Scatter plot of tulip index returns against VOC returns over matched intervals, used for OLS beta estimation. For each tulip index observation, the contemporaneous VOC return is computed over the same interval using carry-forward pricing. All matched return pairs are shown; the OLS regression line is fitted to the four informative pairs (where the VOC price changed), yielding $\hat{\beta} = 2.10$ (SE = 1.42, 95% CI [-4.03, 8.23]). We adopt $\beta = 3.5$ ($\hat{\beta} + 1$ SE) as discussed in Section 5.3. Red circles denote pre-dinner-party pairs; gold star markers denote pairs at or after the dinner-party date (25 November 1636); grey diamonds denote post-crash pairs. Pairs clustered at VOC return = 0 are observations where the VOC price did not change between consecutive tulip observations—these carry no information for beta estimation but illustrate the sparsity of the data.

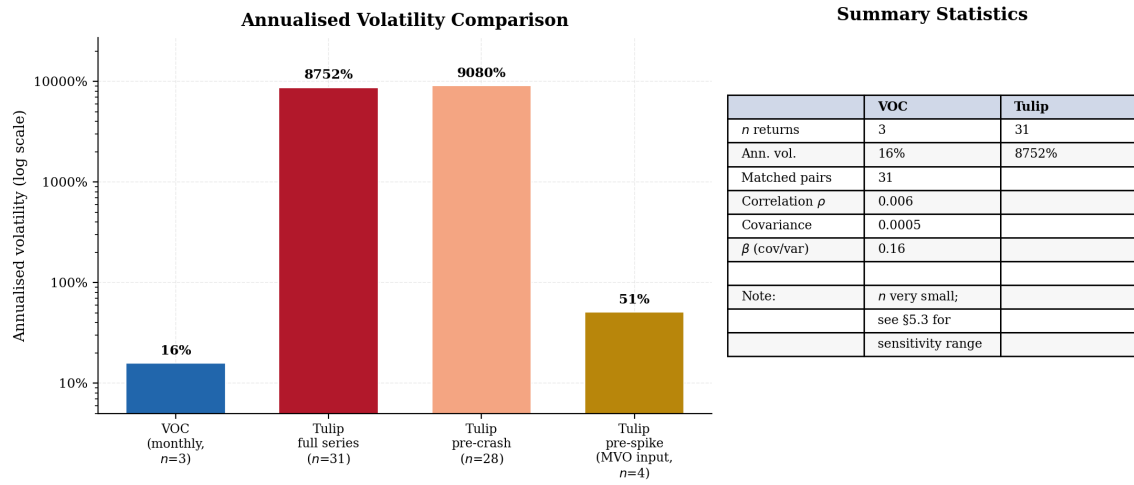


Figure 6: Left: annualised return standard deviation on a logarithmic scale, showing VOC (3 monthly observations, $\sigma \approx 16\%$ p.a.), the tulip composite index under various windows, and the pre-spike tulip estimate ($\sigma \approx 51\%$ p.a., from four returns May–November 12, 1636) used as the MVO input. The full-series and pre-crash tulip volatility ($\sigma \approx 8,800\%$ p.a. on a 30-day-scaled basis) reflects the extreme intra-period swings of January–February 1637. The log scale makes all four bars visible despite spanning three orders of magnitude. Right: summary statistics from the $n = 31$ matched return pairs. The overall correlation ($\rho = 0.006$) and OLS beta ($\hat{\beta} = 2.10$ from informative pairs) confirm that tulip and VOC prices moved largely independently over the mania period. Together, Figures 5 and 6 establish the two key inputs for the CAPM and MVO analysis: a beta estimated from sparse data (adopted at $\beta = 3.5$), and a diversification case for tulips arising from their low market correlation.

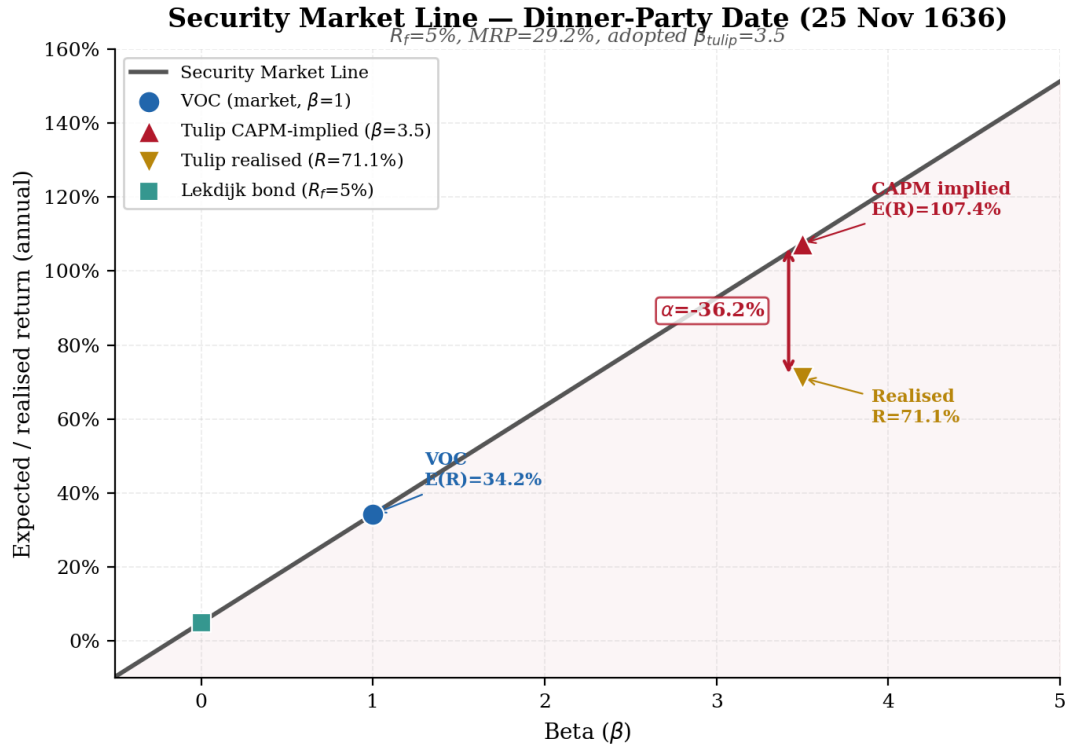


Figure 7: Security Market Line at the dinner-party date (25 November 1636). The SML (grey line) plots CAPM-implied expected returns as a function of beta: $E(R_i) = R_f + \beta_i \times MRP$, with $R_f = 5\%$ and $MRP = 29.25\%$. VOC (blue circle, $\beta = 1$) sits on the line by construction. The tulip CAPM-implied position (red triangle, $\beta = 3.5$) lies on the SML at $E(R) = 107.4\%$; the tulip realised return (gold triangle) falls below it at 71.1% . The vertical gap between the two tulip markers is the alpha of -36.2% : the asset has not yet delivered the returns commensurate with its systematic risk. Under the model's own logic, tulips are cheap. The Lekdijk bond (teal square) anchors the y-intercept at the risk-free rate.

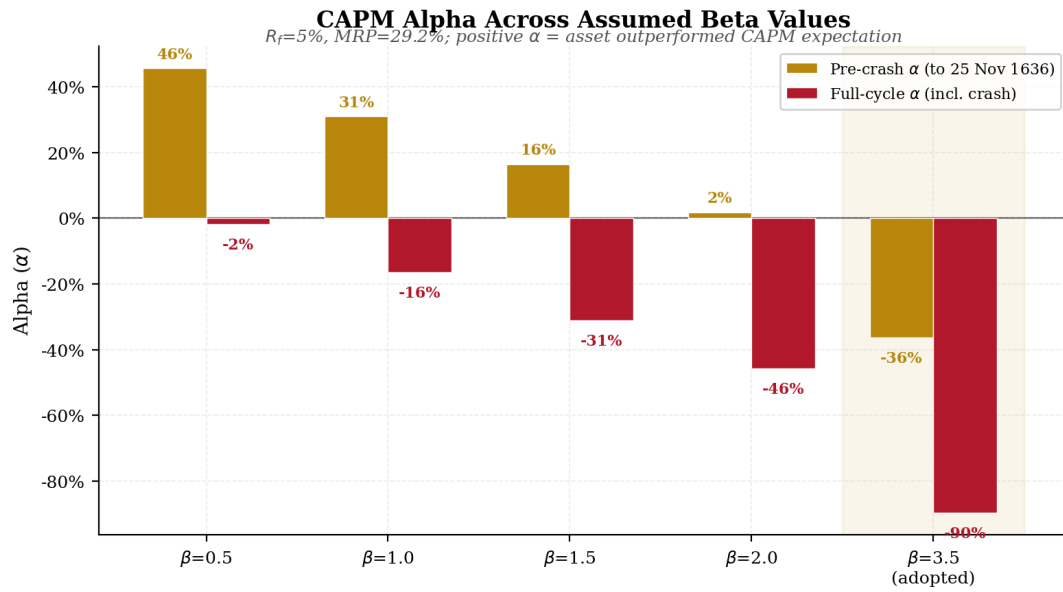


Figure 8: CAPM alpha across assumed beta values. Gold bars show the pre-crash alpha (to 25 November 1636); red bars show the full-cycle alpha (including the February 1637 crash). At low assumed betas ($\beta = 0.5$), tulips show strongly positive pre-crash alpha (+46%): the asset outperformed its modest CAPM expectation. As β increases, the CAPM-implied expected return rises faster than the realised return, and alpha turns negative. At the adopted $\beta = 3.5$ (highlighted), pre-crash alpha is -36%: tulips are “cheap” under the model. The full-cycle alpha is negative at every beta value, and catastrophically so at higher betas (-90% at $\beta = 3.5$), reflecting the crash that the model could not anticipate.

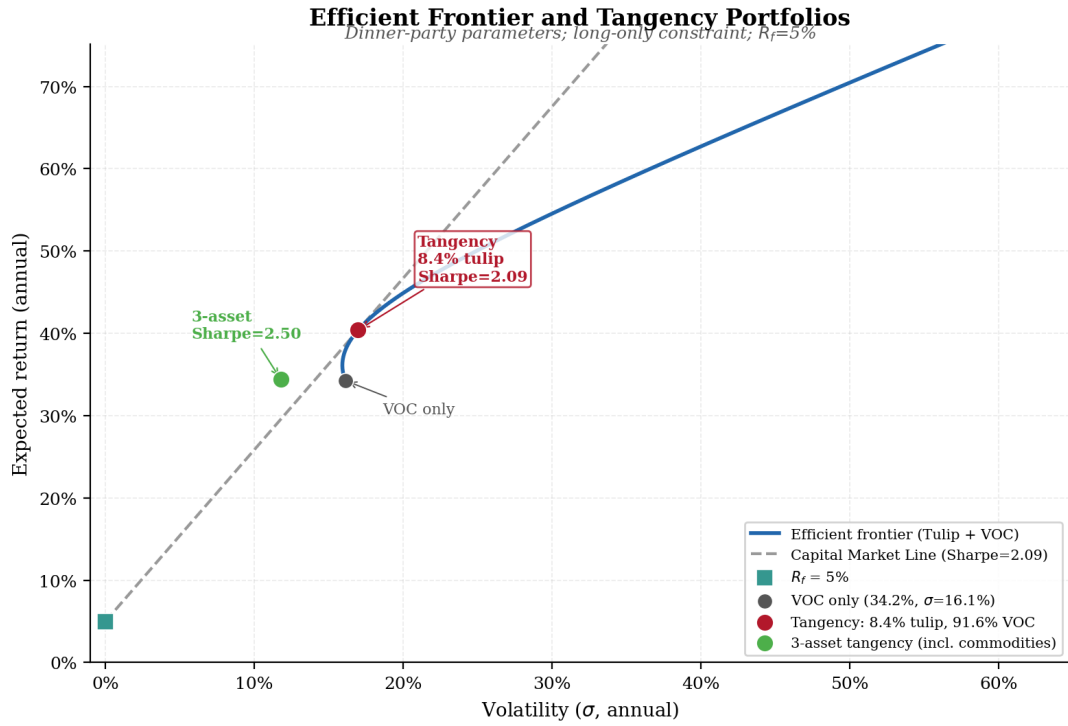


Figure 9: Efficient frontier for the two-asset risky set (tulips, VOC), with the Capital Market Line from $R_f = 5\%$ through the tangency portfolio. The tangency point (red, 8.4% tulip / 91.6% VOC, Sharpe = 2.09) lies above the VOC-only point (grey, Sharpe = 1.85), demonstrating the diversification benefit of adding an uncorrelated asset. The three-asset tangency portfolio including Posthumus commodities (green, Sharpe = 2.50) achieves a higher Sharpe ratio at lower volatility. Axes are in annualised percentage units; long-only constraint applied.

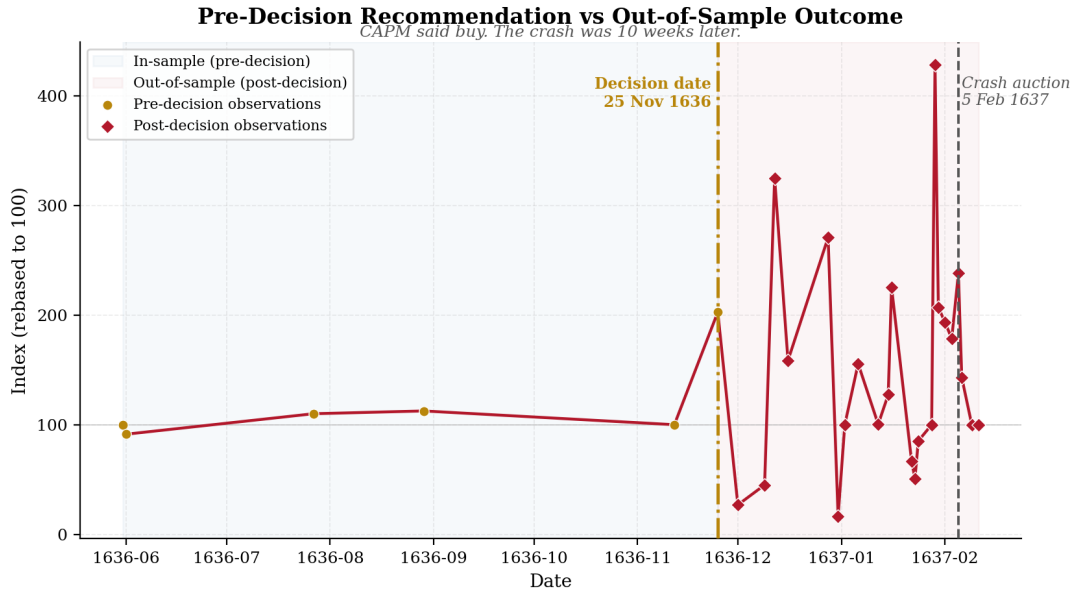


Figure 10: Out-of-sample path of the tulip index around the decision date. The blue-shaded region marks in-sample observations (pre-decision); the red-shaded region marks out-of-sample data (post-decision). Gold circles are data points available to the investor; red diamonds are observations that arrived after the model said “buy.” The dash-dot line marks the dinner-party cutoff (25 November 1636) and the dashed line marks the Alkmaar crash auction (5 February 1637). The index peaked at over $4\times$ its base before collapsing to approximately $1\times$ within days.

References

- [1] Mackay, C. (1841). *Extraordinary Popular Delusions and the Madness of Crowds*. London: Richard Bentley.
- [2] Garber, P. M. (1989). "Tulipmania." *Journal of Political Economy*, 97(3), 535–560.
- [3] Garber, P. M. (2000). *Famous First Bubbles: The Fundamentals of Early Manias*. Cambridge, MA: MIT Press.
- [4] Thompson, E. A. (2007). "The Tulipmania: Fact or Artifact?" *Public Choice*, 130(1), 99–114.
- [5] Goldgar, A. (2007). *Tulipmania: Money, Honor, and Knowledge in the Dutch Golden Age*. Chicago: University of Chicago Press.
- [6] Smant, D. (2012). "Tulipmania." Personal research compilation. Available at <https://sites.google.com/site/davesmant/monetary-economics/famous-first-bubbles/tulipmania>. Accessed January 2025.
- [7] Petram, L. O. (2011). *The World's First Stock Exchange: How the Amsterdam Market for Dutch East India Company Shares Became a Modern Securities Market, 1602–1700*. PhD thesis, Universiteit van Amsterdam. Data available at <https://www.worldsfirststockexchange.com/>.
- [8] Dwivedi, A. "A Qualitative Research on the 1630s' Tulip Bubble 'Tulipmania'." *International Journal of Social Science and Economic Research*. (2023)., 8(6), 1578–1592.
- [9] McClure, J. E., & Thomas, D. C. (2017). "Explaining the Timing of Tulipmania's Boom and Bust: Historical Context, Sequestered Capital, and Market Signals." *Financial History Review*, 24(2), 121–141.
- [10] Posthumus, N. W. (1929). "The Tulip Mania in Holland in the Years 1636 and 1637." *Journal of Economic and Business History*, 1(4), 434–466.
- [11] Sharpe, W. F. (1964). "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk." *The Journal of Finance*, 19(3), 425–442.
- [12] Markowitz, H. (1952). "Portfolio Selection." *The Journal of Finance*, 7(1), 77–91. The foundational paper introducing mean-variance optimisation, for which Markowitz shared the 1990 Nobel Memorial Prize in Economic Sciences.
- [13] Kraus, A., & Litzenberger, R. H. (1976). "Skewness Preference and the Valuation of Risk Assets." *The Journal of Finance*, 31(4), 1085–1100.
- [14] Friend, I., & Westerfield, R. (1980). "Co-Skewness and Capital Asset Pricing." *The Journal of Finance*, 35(4), 897–913.
- [15] Charlier, C. V. L. (1905). "Über das Fehlergesetz." *Arkiv för Matematik, Astronomi och Fysik*, 2(8), 1–9.
- [16] Edgeworth, F. Y. (1907). "On the Representation of Statistical Frequency by a Series." *Journal of the Royal Statistical Society, Series A*, 70(1), 102–106.
- [17] Mandelbrot, B. (1963). "The Variation of Certain Speculative Prices." *The Journal of Business*, 36(4), 394–419.

- [18] Kahneman, D., & Tversky, A. (1979). "Prospect Theory: An Analysis of Decision under Risk." *Econometrica*, 47(2), 263–291.
- [19] Rachev, S. T., Menn, C., & Fabozzi, F. J. (2005). *Fat-Tailed and Skewed Asset Return Distributions: Implications for Risk Management, Portfolio Selection, and Option Pricing*. Hoboken, NJ: Wiley. The most comprehensive treatment of stable portfolio theory—what mean-variance optimisation should have been.
- [20] Bawa, V. S. (1975). "Optimal Rules for Ordering Uncertain Prospects." *Journal of Financial Economics*, 2(1), 95–121.
- [21] Fishburn, P. C. (1977). "Mean-Risk Analysis with Risk Associated with Below-Target Returns." *The American Economic Review*, 67(2), 116–126.
- [22] Artzner, P., Delbaen, F., Eber, J.-M., & Heath, D. (1999). "Coherent Measures of Risk." *Mathematical Finance*, 9(3), 203–228. The paper that proved VaR is incoherent and established the axiomatic foundations for CVaR.
- [23] Rockafellar, R. T., & Uryasev, S. (2000). "Optimization of Conditional Value-at-Risk." *Journal of Risk*, 2(3), 21–41. Shows CVaR optimisation is a tractable linear program.
- [24] Kelly, J. L. (1956). "A New Interpretation of Information Rate." *Bell System Technical Journal*, 35(4), 917–926. The original Kelly Criterion paper—optimal betting for long-term growth.
- [25] Thorp, E. O. (1969). "Optimal Gambling Systems for Favorable Games." *Review of the International Statistical Institute*, 37(3), 273–293. Thorp's application of Kelly to finance, before founding Princeton Newport Partners.
- [26] MacLean, L. C., Thorp, E. O., & Ziemba, W. T. (Eds.). (2011). *The Kelly Capital Growth Investment Criterion: Theory and Practice*. Singapore: World Scientific. The definitive collection on Kelly criterion applications, including the case for fractional Kelly under estimation uncertainty.
- [27] Pabrai, M. (2007). *The Dhandho Investor: The Low-Risk Value Method to High Returns*. Hoboken, NJ: Wiley. "Heads I win, tails I don't lose much"—the asymmetry principle applied to individual investments. We extend it to the investment universe itself.
- [28] Ben-Tal, A., El Ghaoui, L., & Nemirovski, A. (2009). *Robust Optimization*. Princeton, NJ: Princeton University Press. The definitive treatment of optimisation under distributional uncertainty.
- [29] Embrechts, P., Klüppelberg, C., & Mikosch, T. (1997). *Modelling Extremal Events for Insurance and Finance*. Berlin: Springer. The standard reference for Extreme Value Theory in finance.
- [30] Merton, R. C. (1976). "Option Pricing When Underlying Stock Returns Are Discontinuous." *Journal of Financial Economics*, 3(1–2), 125–144. Introduces the jump-diffusion framework: normal diffusion plus Poisson jumps.
- [31] Student. (1908). "The Probable Error of a Mean." *Biometrika*, 6(1), 1–25. Introduces Student's t distribution for heavy-tailed inference under finite samples.
- [32] Heston, S. L. (1993). "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options." *The Review of Financial Studies*, 6(2), 327–343. Canonical stochastic-volatility model with mean-reverting variance.

- [33] Center for Evidence-Based Management. (2023). *Guidelines for Rapid Evidence Assessments and Critically Appraised Topics*. CEBMa. <https://cebma.org/resources/guidelines-reas-and-cats/>.
- [34] Knight, F. H. (1921). *Risk, Uncertainty and Profit*. Boston: Houghton Mifflin. The foundational distinction between quantifiable risk and unquantifiable uncertainty—a distinction that finance has spent a century trying to forget.
- [35] Soros, G. (1987). *The Alchemy of Finance*. New York: Simon & Schuster. Introduces reflexivity: markets are not external processes to be passively observed but participatory systems where beliefs affect outcomes.
- [36] Peters, O. (2019). “The Ergodicity Problem in Economics.” *Nature Physics*, 15, 1216–1221. Demonstrates that maximising expected value can lead to almost-certain ruin when time averages differ from ensemble averages—which they do for multiplicative processes like wealth dynamics.
- [37] Schopenhauer, A. (1818). *Die Welt als Wille und Vorstellung*. Leipzig: Brockhaus. The world as we perceive it is representation (*Vorstellung*); beneath it churns the will (*Wille*)—blind, irrational striving. Financial models are representations. The market is will. Schopenhauer recommended aesthetic contemplation as escape; we recommend fractional Kelly.
- [38] Nietzsche, F. (1886). *Jenseits von Gut und Böse*, §146. Leipzig: C. G. Naumann. “Wer mit Ungeheuern kämpft, mag zusehn, dass er nicht dabei zum Ungeheuer wird. Und wenn du lange in einen Abgrund blickst, blickt der Abgrund auch in dich hinein.” Those who study tail risk are advised to look away occasionally.
- [39] Popper, K. (1959). *The Logic of Scientific Discovery*. London: Hutchinson.
- [40] Kolmogorov, A. (1933). “Sulla determinazione empirica di una legge di distribuzione.” *Giornale dell’Istituto Italiano degli Attuari*, 4, 83–91.
- [41] Smirnov, N. (1948). “Table for Estimating the Goodness of Fit of Empirical Distributions.” *Annals of Mathematical Statistics*, 19(2), 279–281.
- [42] Dowd, K., Cotter, J., Humphrey, C., & Woods, M. (2008). “How Unlucky Is 25-Sigma?” *Journal of Portfolio Management*, 34(4), 76–80.
- [43] Zolotarev, V. M. (1986). *One-dimensional Stable Distributions*. Providence, RI: American Mathematical Society.
- [44] Kuhn, T. S. (1962). *The Structure of Scientific Revolutions*. Chicago: University of Chicago Press.
- [45] Tauber, A. (1897). “Ein Satz aus der Theorie der unendlichen Reihen.” *Monatshefte für Mathematik und Physik*, 8(1), 273–277. The original paper on Abel summability that lent its name to the class of “Tauberian theorems.” The specific result we employ (Theorem 8.1.6 of Bingham et al.) is a descendant, not a direct application.
- [46] Karamata, J. (1930). “Sur un mode de croissance régulière des fonctions.” *Mathematica (Cluj)*, 4, 38–53. The foundational paper introducing regularly varying functions and the Tauberian theorems that bear Karamata’s name.
- [47] Lévy, P. (1925). *Calcul des Probabilités*. Paris: Gauthier-Villars.
- [48] Feller, W. (1971). *An Introduction to Probability Theory and Its Applications*, Vol. II, 2nd ed. New York: Wiley. Chapter XVII contains the definitive treatment of stable distributions and their tail asymptotics.

- [49] Bingham, N. H., Goldie, C. M., & Teugels, J. L. (1987). *Regular Variation*. Cambridge: Cambridge University Press. The authoritative reference on slowly varying functions and Karamata theory. Theorem 8.1.6 provides the Tauberian result connecting characteristic function behaviour to tail asymptotics.
- [50] Taleb, N. N. (2020). *Statistical Consequences of Fat Tails: Real World Preasymptotics, Epistemology, and Applications*. STEM Academic Press. Rigorous technical treatment of the mathematics we have summarised here. The author’s tendency to describe himself as “the foremost thinker on probability” should not distract from the fact that he may actually be correct.
- [51] Taleb, N. N., & Cirillo, P. (2019). “The Decline of Violent Conflicts: What Do the Data Really Say?” *Physica A: Statistical Mechanics and its Applications*, 517, 109–113. Earlier versions (arXiv:1510.06731, 2015) introduced the shadow mean concept for bounded heavy-tailed distributions where naive sample statistics underestimate true population values.
- [52] Taleb, N. N. (2019). “How Much Data Do You Need? An Operational, Pre-asymptotic Metric for Fat-tailedness.” *International Journal of Forecasting*, 35(2), 677–686. Introduces the kappa metric for quantifying convergence speed under fat tails—an operational tool for knowing when to distrust sample statistics.
- [53] Taleb, N. N. (2010). *The Bed of Procrustes: Philosophical and Practical Aphorisms*. New York: Random House.
- [54] Apollodorus (c. 180 BCE). *Bibliotheca*, Book III. Athens: Unknown. The source for Theseus’s defeat of Procrustes by fitting the bandit to his own bed—a treatment we recommend for proponents of Gaussian returns.
- [55] Plato (c. 384 BCE). *Euthydemus*. Athens: The Academy. Trans. B. Jowett (1871). A Socratic dialogue in which two sophists demonstrate that any proposition can be “proven” through sufficiently clever argument—much like CAPM.
- [56] Hesiod (c. 700 BCE). *Works and Days*. Boeotia: Oral Tradition. Lines 90–105. The original source for “Pandora’s Box,” though Hesiod actually wrote *pithos* (jar). The mistranslation to “box” is attributed to Erasmus (1508), who apparently also struggled with data quality issues.
- [57] Rumi, J. (c. 1260 CE). *Masnavi-ye-Ma’navi*, Book IV. Konya: Oral recitation to Husam al-Din Chalabi. The full couplet reads: “Sell your cleverness and buy bewilderment / Cleverness is mere opinion, bewilderment is intuition.” Rumi did not have a Bloomberg terminal.
- [58] Anonymous (c. 10th century CE). *Völuspá*. Iceland: Oral Tradition. Stanzas 19–20. The Norns—Ur, Verandi, and Skuld (roughly: Past, Present, and Future)—sit beneath Yggdrasil carving runes of fate. Even the All-Father trades an eye for wisdom and still cannot read what they write. Quantitative analysts take note.
- [59] Anonymous (c. 450–200 BCE). *Ecclesiastes* 9:11. Jerusalem: Unknown. The complete verse: “I returned, and saw under the sun, that the race is not to the swift, nor the battle to the strong, neither yet bread to the wise, nor yet riches to men of understanding, nor yet favour to men of skill; but time and chance happeneth to them all.” The earliest known critique of factor investing.
- [60] Shelley, P. B. (1818). “Ozymandias.” *The Examiner*. London. “My name is Ozymandias, King of Kings; / Look on my Works, ye Mighty, and despair!” The “sneer of cold command”

belongs to a statue half-sunk in desert sand—a useful image for theoretical frameworks built on assumptions that do not hold.

- [61] Shelley, M. (1818). *Frankenstein; or, The Modern Prometheus*. London: Lackington, Hughes, Harding, Mavor & Jones.
- [62] Coleridge, S. T. (1798). “The Rime of the Ancient Mariner.” *Lyrical Ballads*. London: J. & A. Arch. The Mariner, having transgressed against nature, is cursed to wander and repeat his tale to those who must hear it. “He stoppeth one of three”—roughly the success rate of warnings about fat tails.
- [63] Blake, W. (1794). “London.” *Songs of Experience*. London: W. Blake. “In every cry of every Man, / In every Infants cry of fear, / In every voice: in every ban, / The mind-forg’d manacles I hear.” The chains we forge for ourselves are the hardest to break—especially when a career depends on not breaking them.
- [64] Aeschylus (458 BCE). *Agamemnon*. Athens: City Dionysia. Cassandra, cursed by Apollo to prophesy truly and never be believed, is the conventional literary allusion for ignored warnings. We found it too derivative.
- [65] Anonymous (c. 2100 BCE). *Epic of Gilgamesh*, Tablet XI. Mesopotamia: Cuneiform Tablets. Gilgamesh, having failed to achieve immortality, returns to Uruk and finds consolation in its walls—the great works that will outlast him. They did not. The walls are rubble; the poem survives. There may be a lesson here for model-builders.
- [66] Bādarāyaṇa (c. 400 BCE). *Brahma Sūtras*, 2.1.33. India: Oral/Written Tradition. “*Loka-vattu līlā kaivalyam*”—creation is mere sport for Brahman, like play. The stakes of existence feel absolute from within but dissolve upon awakening. Useful for putting portfolio losses in perspective.
- [67] Vyāsa (c. 400 BCE–400 CE). *Mahābhārata*, Drona Parva, Chapters 34–40. India: Oral/Written Tradition. The Chakravyūha—the spiral military formation that Abhimanyu knew how to enter but not exit. A reminder that some positions have no exit within their domain.
- [68] Laozi (c. 6th century BCE). *Dao De Jing*, Chapter 1. China: Oral/Written Tradition. “道可道非常道”—the Dao that can be spoken is not the eternal Dao. The distribution that can be modelled is not the true distribution. Laozi did not have access to market data; one suspects he would not have been surprised.
- [69] Anonymous (c. 3rd century BCE). *Dhammapada*, Verses 277–279. India: Pāli Canon. “*Sabbe saṅkhārā aniccā*”—all conditioned phenomena are impermanent. The Buddhists discovered non-stationarity twenty-three centuries before the econometricians.
- [70] Goodhart, C. A. E. (1975). “Problems of Monetary Management: The U.K. Experience.” *Papers in Monetary Economics*, Reserve Bank of Australia. Later formulated as Goodhart’s Law: “When a measure becomes a target, it ceases to be a good measure.” The economists, arriving late, discovered what the Daoists and Buddhists already knew.
- [71] Goethe, J. W. von (1808). *Faust: Der Tragödie erster Teil*. Tübingen: J. G. Cotta. “Da steh ich nun, ich armer Tor, / Und bin so klug als wie zuvor.” Faust studied everything and learned nothing useful—then made a bargain he could not undo.
- [72] Anonymous (c. 400 BCE–200 CE). *Bhagavad Gītā*, Chapter 2, Verse 47. India: Oral/Written Tradition. “*Karmaṇy evādhikāras te mā phaleṣu kadācana*”—you are entitled to the action, never to its fruits. The most concise statement of why outcome-attachment is incompatible with rational decision-making under uncertainty.

- [73] McCarthy, C. (2006). *The Road*. New York: Alfred A. Knopf. A world in which portfolio theory is no longer applicable. Useful as a boundary condition.
- [74] Hayek, F. A. (1945). “The Use of Knowledge in Society.” *The American Economic Review*, 35(4), 519–530.
- [75] Keynes, J. M. (1936). *The General Theory of Employment, Interest and Money*. London: Macmillan. Chapter 12.
- [76] Linnainmaa, J. (2026). *Investments*, Winter 2026 course lectures. Tuck School of Business, Dartmouth College. Classes 02–10.
- [77] DeMiguel, V., Garlappi, L., & Uppal, R. (2009). “Optimal Versus Naive Diversification: How Inefficient Is the 1/N Portfolio Strategy?” *The Review of Financial Studies*, 22(5), 1915–1953.
- [78] Britten-Jones, M. (1999). “The Sampling Error in Estimates of Mean-Variance Efficient Portfolio Weights.” *The Journal of Finance*, 54(2), 655–671.
- [79] Fama, E. F., & French, K. R. (2004). “The Capital Asset Pricing Model: Theory and Evidence.” *Journal of Economic Perspectives*, 18(3), 25–46.
- [80] Fama, E. F. (2011). “My Life in Finance.” *Annual Review of Financial Economics*, 3, 1–15. Fama’s account of his intellectual trajectory, including his response to the behavioural challenge. The 2013 Nobel press conference and the filmed 2016 Fama–Thaler debate at Chicago Booth provide additional colour on the proposition that “efficiency” can be redefined until anomalies disappear.
- [81] Tatman, R. (2017). “Delpher: Dutch Newspaper Archive (1618–1699).” Kaggle. Available at <https://www.kaggle.com/datasets/rtatman/delpher-dutch-newspaper-archive-16181699/code>. Digitised text of Dutch newspaper articles from the Delpher collection maintained by the Koninklijke Bibliotheek, covering the period 1618–1699. Contains no price data. May be of use for natural language processing research into the social dimensions of speculative episodes, though we suspect the seventeenth-century Dutch orthography will test the patience of most tokenisers.
- [82] Taylor, B. (2018). “Data for Amsterdam Stocks from the 1600s and 1700s Added to GFD.” Global Financial Data / Finaeon. Available at <https://finaeon.com/data-for-amsterdam-stocks-from-the-1600s-and-1700s-added-to-gfd/>. Provides share price series for three securities traded on the Amsterdam Bourse: VOC (1601–1794, ticker VOC1-AM), GWC (ticker WIC1-AM), and Sociëteit von Berbice (1734–1752). We reviewed the periodicity metadata via the Feldberg Library’s Finaeon subscription but did not have download access to the underlying price series. The VOC1-AM series is *annual* from December 1619 to April 1671, offering no improvement over Petram’s irregular monthly observations for the 1630s. The GWC series begins only in December 1674—not 1628 as suggested by Finaeon’s marketing copy—and therefore contains no data for the tulipmania period. Neither dataset contributes additional observations to our analysis window.
- [83] Posthumus, N. W. (1943). *Nederlandsche Prijsgeschiedenis*. Leiden: E. J. Brill. Vol. 1. Commodity prices from the Amsterdam exchange, compiled from approximately 3,033 surviving price currents (of roughly 12,000 issued between 1585 and 1813). Monthly quotations for over 200 commodities including spices, grains, textiles, and metals. Digitised and queryable at the Medieval and Early Modern Data Bank (MEMDB), Rutgers University Libraries: <https://memdb.libraries.rutgers.edu/posthumus-prices>.

-
- [84] De Zwart, P. (2022). “VOC Prices in the Dutch-Asian Trade, 1608–1800.” IISH Data Collection, V1. <https://hdl.handle.net/10622/PEKT40>. Licensed under CC BY-SA 4.0. Contains original, extrapolated, interpolated, and imputed commodity prices from the VOC’s own trading records—purchase prices at Asian sources and sale prices at Amsterdam—for the full period of Company operations. Annual Amsterdam prices are available for six commodities (cinnamon, cloves, mace, nutmeg, pepper, sugar) across our full 1630–1638 analysis window; the underlying transaction-level records in the individual commodity workbooks are sparser and irregular. Our processed extracts (Appendix C.4) are derivative works under the same licence.
- [85] Gelderblom, O., & Jonker, J. (2004). “Completing a Financial Revolution: The Finance of the Dutch East India Trade and the Rise of the Amsterdam Capital Market, 1595–1612.” *The Journal of Economic History*, 64(3), 641–672. Documents the institutional infrastructure—bill markets, forward contracts, and secondary share trading—that made the Amsterdam capital market functional before the VOC’s formal listing.

Appendices

A Discussion

A.1 Limitations

Despite the satirical framing, this is, to our knowledge, the first formal CAPM analysis of tulipmania—constructing a market portfolio, estimating beta, and applying mean-variance optimisation to seventeenth-century Dutch assets. The limitations we acknowledge are not unique to our analysis; they are endemic to any quantitative study of this period, and have been explicitly identified by historians of the episode. The structure of this appendix is cumulative: first we diagnose where the Gaussian mean-variance package fails conceptually and empirically, then we prove the relevant tail mathematics, then we run a formal statistical validation with controlled error rates.

Our dataset comprises perhaps two dozen reliable price observations scattered across three years. This reflects the archival reality: Goldgar’s foundational study relied on notarial archives “naturally biased in favor of the middle classes able to afford legal services,” leaving lower-class participation unclear[5]. The February 1637 “peak” prices derive from a single estate auction; “almost nothing is known about prices of the trades in more common bulbs in local taverns during November 1636–February 1637”[6]. The precise trigger of the collapse “remains unclear”[5]. The long-term socioeconomic impact on the Dutch Republic remains underexplored[8]. And while the concurrent timing of VOC share appreciation and tulip price inflation has been noted[9], no prior study has formally analysed their covariance[7].

We have constructed a “market portfolio” from exactly one security. Our “risk-free rate” is derived from a single bond issued to a single woman in Holland five human lifetimes ago. Britten-Jones[78] showed that even with 20 years of modern MSCI data on 11 developed-country markets, none of the tangency portfolio weights is statistically significant—an illustration Linnainmaa[76] presents as a cautionary tale. DeMiguel, Garlappi, and Uppal[77] found that the estimation window required for mean-variance optimisation to outperform a naïve $1/N$ equal-weight portfolio is approximately 250 years for a portfolio with 25 assets. We have three years and 42 assets. These limitations are real—and yet, we submit that our analysis is no less *inferentially sound* than much of what passes for financial economics in academe today. The assumptions underlying CAPM deserve scrutiny that they rarely receive. The sections that follow lay out our earnest critique of this selective attention to rigour.

A.2 CAPMania

A.2.1 The Gaussian Fallacy

CAPM is, at core, a linear restriction on expected excess returns[11]. On its own it does not require Gaussian returns. As Fama and French emphasise, however, the operational CAPM/MPT workflow used in practice depends on mean-variance investors and second-moment objects[79]. That workflow—beta from covariances, Sharpe ratios, and tangency portfolios[12]—requires finite and estimable second moments, mean-variance sufficiency¹⁷, and a stable (approximately stationary) data-generating process over the estimation window. Mean-variance sufficiency is exact under quadratic utility¹⁸ or under elliptical return families¹⁹, with the Gaussian as the canonical special case used in textbooks and software. We therefore use the Gaussian benchmark

¹⁷Mean-variance sufficiency means portfolios can be ranked using only expected return and variance; higher moments do not alter the ranking rule.

¹⁸Quadratic utility takes a form such as $U(W) = aW - bW^2$ with $b > 0$. In that case, expected utility depends only on the first two moments of wealth, so mean and variance are sufficient statistics.

¹⁹Elliptical families are distributions with ellipsoidal contours (Gaussian is the canonical case). They preserve linear-portfolio structure in a way that supports mean-variance ranking.

as the explicit null model:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (4)$$

Under this benchmark, variance is treated as a sufficient risk statistic, optimisation has closed-form structure, and option-pricing approximations are tractable. It also implies zero skewness and kurtosis exactly 3.

Inconveniently, this is empirically false.

Mandelbrot observed as early as 1963 that cotton prices exhibited extreme movements far more frequent than a Gaussian distribution would predict[17]. The field has not ignored this problem—attempts have been made to patch CAPM’s distributional assumptions, and we shall examine why they fail. But first, we must understand what makes the Gaussian so seductive in the first place, and what happens when its foundational requirements are not met.

The Gaussian Gift: Central Limit Theorem. The normal distribution earns its privileged status from the classical CLT: if you add together many independent random variables with finite variance, their appropriately normalised sum converges to a Gaussian *regardless of the component distributions*[48]. For finance, this motivates the approximation that diversified portfolio returns should be near-normal.

But the finite-variance condition is critical. If it fails, the classical Gaussian CLT does not apply; the relevant limit law is the generalised CLT, which yields non-Gaussian stable limits[48, 17]. The relationship to the Law of Large Numbers (LLN) is important. LLN concerns convergence of sample averages (typically requiring a finite first moment), while CLT concerns their fluctuation scale and limit shape. Hence for $1 < \alpha < 2$ in a stable domain, an average can satisfy LLN yet still fail to have Gaussian fluctuations because variance is infinite; for $\alpha \leq 1$, even LLN around a finite mean can fail because the mean does not exist.

The Gaussian is *fully characterised* by its first two moments—once μ and σ^2 are known, skewness, kurtosis, and all higher moments are pinned down (Appendix B.2). This is precisely why variance can be treated as a complete risk summary in the Gaussian benchmark. A concrete illustration of what happens when that assumption fails is provided in Appendix B.3: under a fixed-parameter i.i.d. Gaussian null calibrated on pre-1987 data, Black Monday has a horizon probability on the order of 10^{-175} even under conservative volatility bounds.

Deductive Scope and Target. To avoid category errors, we define the model class under critique as

$$\mathcal{M} = \left\{ \begin{array}{l} \text{finite and estimable second moments,} \\ \text{mean-variance sufficiency,} \\ \text{approximate stationarity (time-invariance),} \\ \text{CAPM pricing restriction} \end{array} \right\}.$$

Concretely, this means: (i) $\beta_i = \text{Cov}(R_i, R_m)/\text{Var}(R_m)$ is estimable with useful precision; (ii) first two moments are sufficient for ranking portfolios; (iii) sample moments proxy population moments over the estimation window; and (iv) expected excess returns satisfy CAPM’s linear restriction conditional on available information. Our conclusion is deductive: if observed data violate any necessary element of $\mathcal{M}$, then this mathematical implementation of EMH/MPT is descriptively inadequate for inference and allocation. This does *not* prove that all conceivable notions of informational efficiency²⁰ are false; it rejects the specific mathematical machinery ordinarily used to test them.

²⁰Informational efficiency means prices reflect the available information set, so no strategy based on that set earns persistent risk-adjusted abnormal returns.

Why Adding Moments Is Not the Solution. The moment-based extensions to CAPM attempt to fix the model by adding the third and fourth moments as additional factors. A substantial literature—beginning with Kraus and Litzenberger[13] and extended by Friend and Westerfield[14]—developed “three-moment” and “four-moment” CAPMs that incorporate skewness and kurtosis as additional priced factors. This work found significant effects: securities contributing negative skewness to the market portfolio earn a premium of 2.5–3.6% annually. Yet these refinements, despite forty years of empirical support, remain peripheral to how CAPM is taught and practised. The textbooks, the CFA curriculum, and industry risk models still treat variance as a sufficient statistic for risk. If returns are skewed, price that skewness. If returns are leptokurtic (fat-tailed), price that kurtosis. This seems reasonable—but it fundamentally misunderstands the nature of the problem.

The critical distinction is between *describing* a distribution with moments and *generating* it from a different probability law. Adding skewness and kurtosis to a Gaussian produces approximations like the Gram-Charlier[15] or Edgeworth[16] expansions—polynomial adjustments that tweak the tails of a fundamentally Gaussian shape. These are not Lévy stable distributions. They are Gaussians wearing disguises.

The difference lies in a parameter that moments cannot capture: the *stability index* α , sometimes called the *tail index* or *characteristic exponent*. This parameter controls how the tails decay:

- For $\alpha = 2$ (Gaussian), tails decay exponentially fast—like e^{-x^2} .
- For $\alpha < 2$, tails decay as a power law—like $x^{-(\alpha+1)}$.

This is not a matter of degree; it is a matter of kind. Power-law tails are qualitatively different from exponential tails. They assign meaningful probability to events that exponential tails treat as impossible.

Crucially, in the exact unbounded Lévy-stable model, the stability index α determines which moments exist: for $\alpha < 2$, variance is infinite; for $\alpha \leq 1$, even the mean is undefined. We derive this rigorously via a Tauberian argument below. The implication for moment-based CAPM extensions is structural: if the underlying tail law is in that domain, adding co-skewness or co-kurtosis does not repair mean-variance foundations because the second moment itself is not well-defined. One cannot price a quantity that is not defined. The three-moment and four-moment CAPMs are attempts to measure properties that the underlying distribution does not possess.

Mandelbrot’s Discovery: Fat Tails. Returning now to Mandelbrot: examining cotton prices, he found that extreme price movements—both crashes and spikes—occurred far more frequently than the Gaussian tails predicts. A normal distribution assigns vanishingly small probabilities to events more than 4–5 standard deviations from the mean. In practice, markets experience such moves regularly. This was the first suggestion that we are not in that domain.

Consider: under a Gaussian model, the October 1987 crash (a 22% single-day decline, approximately 25 standard deviations using pre-crash volatility) should occur roughly once every 10^{135} years[42]—many orders of magnitude longer than the age of the universe. Yet it happened.

Either we witnessed a miracle²¹, or the model is wrong. Mandelbrot spent his remaining years like Coleridge’s Mariner[62], accosting wedding guests about non-normality, who hurried nervously past.²²

²¹or more aptly, the opposite of one

²²Cassandra[64] being the conventional choice, but too derivative.

Lévy Stable Distributions: A Generalisation. Mandelbrot proposed that returns follow a *Lévy α -stable distribution*, a broader family that includes the Gaussian as a special case ($\alpha = 2$). The Cauchy distribution ($\alpha = 1$) is another member. For $1 < \alpha < 2$, tails decay as a power law ($\sim x^{-\alpha-1}$) rather than exponentially—slow enough that the variance integral diverges, as we formalise below.

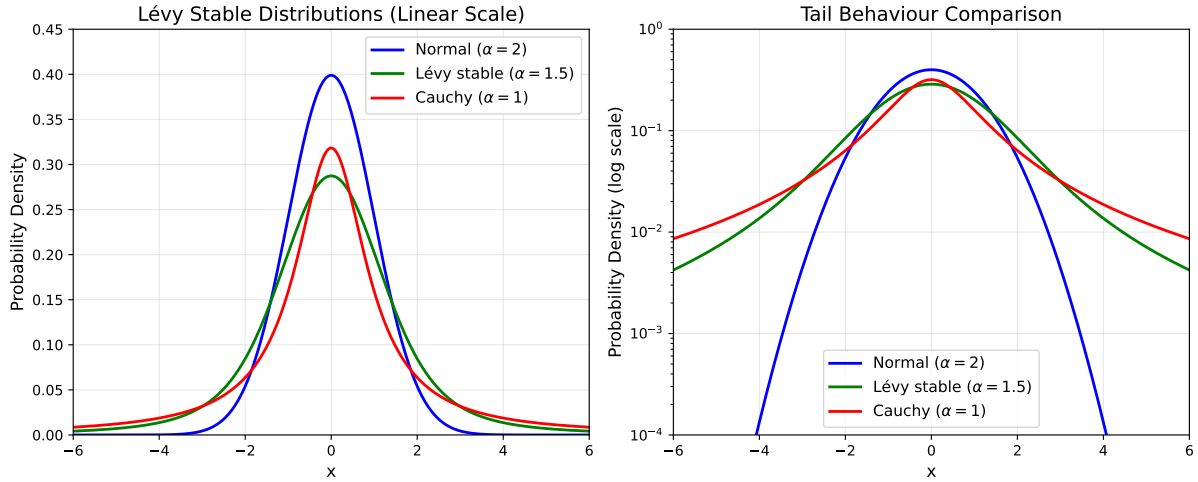


Figure 11: Comparison of Lévy stable distributions with different stability parameters α . Left: Linear scale showing the central peak. Right: Logarithmic scale revealing the dramatically different tail behaviour—the Cauchy ($\alpha = 1$) and intermediate ($\alpha = 1.5$) distributions assign far higher probabilities to extreme events than the Gaussian ($\alpha = 2$).

The term “stable” refers to a defining property: sums of stable random variables are also stable with the same α ²³. This generalises the Gaussian’s behaviour under addition. Just as summing Gaussians yields a Gaussian, summing Lévy stable variables yields another Lévy stable variable—but unless $\alpha = 2$, the result is not normal.

Variance in the Lévy Model. For $\alpha < 2$, variance is mathematically infinite *in the exact stable model*. This does not mean finite samples return literal infinity—it means sample variances remain unstable and are driven by rare extremes, so convergence to a decision-useful value is extremely slow.

This has profound implications. In the strict $\alpha < 2$ stable case, mean-variance optimisation is ill-posed because its inputs are undefined at population level. More generally, even with bounded support²⁴, the same heavy-tail geometry makes covariance estimation fragile enough that optimiser outputs are dominated by estimation error. Diversification still helps, but not with the Gaussian speed and reliability that classical mean-variance calculations assume.

The Rigorous Derivation. The formal proof that Lévy stable distributions have power-law tails—and hence infinite variance for $\alpha < 2$ —requires the machinery of characteristic functions, regular variation theory²⁵, and Tauberian theorems. We present this derivation in full in Appendix B.1. For present purposes the key point is model-class identification: once tails are in

²³For context, power laws are a broader category of heavy-tailed relationships. Systems exhibiting *self-organised criticality*—a condition of back-to-back unstable equilibria, like a line of dominoes—naturally evolve to critical states where event sizes follow power law distributions. The logistic map, forest fires, cardiac tissue, and falling sand all exhibit this behaviour, as summarised brilliantly in Veritasium’s “This equation will change how you see the world (the logistic map),” <https://www.youtube.com/watch?v=ovJcsL7vyrk>

²⁴Bounded support means returns lie within a finite interval. In markets this is naturally approximated by a hard lower bound near -100% and a finite upper bound.

²⁵Regular variation theory is the asymptotic toolkit for power-law tails, built around slowly varying functions and tail exponents.

the stable domain, the classical Gaussian CLT is the wrong asymptotic tool, and moment-based estimators inherit slow, unstable convergence.

The Bounded Reality. A careful reader might object: the Lévy stable model has unbounded support, but real markets do not. Returns are bounded below at -100% (one cannot lose more than everything) and bounded above by global wealth—a single company cannot exceed the total economic value of civilisation, currently around \$500 trillion. Therefore returns exist on $[-1, H]$ where H is astronomically large but finite. If support is bounded, all moments technically exist.

This objection is mathematically correct and practically decisive in the right way. The issue is not whether moments exist in some Platonic or metaphysical sense, but whether they are inferentially usable. With bounded but very heavy tails, moments exist yet are dominated by rare extremes; finite-sample estimators can converge too slowly to support stable optimisation. “Finite but not estimable with useful precision on available horizons” is operationally equivalent to “not decision-usable.”

Taleb and Cirillo[51] developed tools for precisely this situation. Their *shadow moments* use extreme value theory to estimate the “true” moments of bounded heavy-tailed distributions, showing that naive sample statistics underestimate by 3–4 $\times$. The *kappa metric*[52] quantifies how much data is required for convergence: for market returns, it indicates astronomical requirements—“need 10^{12} observations” when we have 10^4 is practically “never.” These tools are diagnostic, not prescriptive; they quantify why mean-variance optimisation fails rather than rescuing it. Taleb himself considers CAPM and Modern Portfolio Theory to be “hot air”[50]—once you remove the Gaussian assumptions, what remains is, in his words, “quack remedies sold on the Internet.”

Bounded markets therefore do not automatically rescue mean-variance optimisation. Theoretically finite variance does not imply statistically reliable covariance estimates. Mean-variance optimisation requires *precise* covariance inputs; when plausible estimates span orders of magnitude, optimiser weights are artefacts of noise rather than information.

The empirical evidence is unambiguous. Actual market returns exhibit:

- **Leptokurtosis:** Kurtosis far exceeding 3, indicating “fat tails”—extreme returns occur with far greater frequency than Gaussian assumptions predict. The Wilshire 5000 Total Market Index exhibits kurtosis of 13.6—more than four times the Gaussian value of 3.
- **Negative skewness:** A left-skewed distribution, meaning large negative returns (crashes) are more frequent than large positive returns of equal magnitude. The Wilshire 5000 exhibits skewness of -0.49 .

To quantify this, we fit both a Gaussian and a Lévy stable distribution to 8,808 daily returns from the Wilshire 5000 Total Market Index (1990–2024) using maximum likelihood estimation. The fitted stability parameter is $\alpha = 1.55$, far below 2. Under an exact unbounded stable model this implies infinite variance; under bounded real markets it implies extremely heavy tails and fragile moment estimation. Either way, the Gaussian finite-variance benchmark is a poor risk model for optimisation inputs. Figure 12 and Table 4 summarise relative fit.

The “So What”? Do our tulip price observations suggest Gaussian returns? With so few data points, they cannot adjudicate distributional form. But this is exactly the inferential problem: the CAPM/MPT machinery requires assumptions that cannot be validated in the sample where we deploy it. The broader evidence from modern markets indicates heavy tails, unstable moments, and regime dependence; those features break at least one necessary condition in $\mathcal{M}$. The deductive implication is limited but sharp: the standard Gaussian mean-variance implementation is not a realistic mathematical foundation for allocation decisions, even before one takes a position on informational efficiency in the abstract.

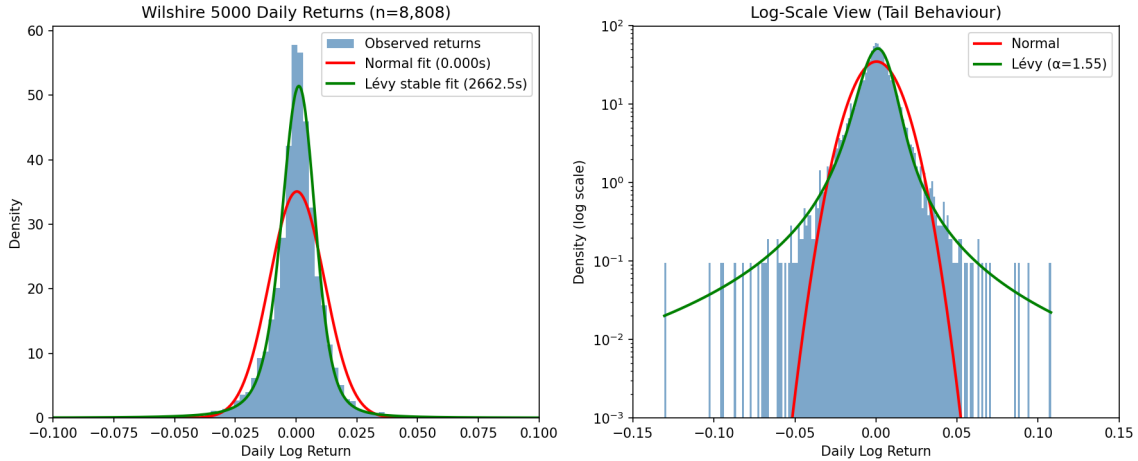


Figure 12: Distribution of daily returns for Wilshire 5000 ($n=8,808$). The normal distribution (red) systematically underestimates extreme returns, while the Lévy stable distribution (green, $\alpha = 1.55$) captures the observed leptokurtosis. The log-scale panel reveals that tail events occur orders of magnitude more frequently than Gaussian assumptions predict. See Table 4 for fit parameters and timing.

Metric	Normal	Lévy Stable
Fit time (seconds)	<0.001	2662.55
α (stability)	2 (fixed)	1.5505
K-S statistic	0.0917	0.0256
Plug-in K-S p-value*	6.33×10^{-65}	1.98×10^{-5}

Table 4: Distribution fitting results for Wilshire 5000 daily returns ($n=8,808$). Empirical moments: skewness = -0.49 , kurtosis = 13.6 . Lévy stable fitting took 44 minutes versus milliseconds for the normal, a ratio of approximately 66 million to one. *Plug-in K-S p-values are descriptive only; exact goodness-of-fit inference requires dependence-aware bootstrap calibration with parameter re-estimation (Appendix B.2).

A.2.2 The Behavioural Critique

The Gaussian critique attacks distributional assumptions. A second requirement is the behaviour of pricing residuals in the empirical security market line. We retain the standard residual notation $\epsilon_{i,t}$ (rather than investor-level “belief errors”):

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + \epsilon_{i,t}. \quad (5)$$

Under CAPM with correctly specified information set $\mathcal{I}_{t-1}$, residuals satisfy a conditional orthogonality condition²⁶, and abnormal-return intercepts are zero in the population model:

$$E[\epsilon_{i,t} \mid \mathcal{I}_{t-1}] = 0, \quad \alpha_i = 0. \quad (6)$$

This is not a claim that investor beliefs “average to truth.” It is a claim that model-conditional forecast errors are unsystematic²⁷. The behavioural critique is therefore not invalid; it is a violation claim: if biases are systematic and state-dependent, then $E[\epsilon_{i,t} \mid \mathcal{I}_{t-1}]$ can depart from zero and α_i can be persistently non-zero over relevant horizons.

²⁶Here “orthogonality” means pricing errors have zero conditional projection on the information set: they are unpredictable given $\mathcal{I}_{t-1}$.

²⁷“Model-conditional” means a statement is valid given the model’s maintained assumptions; it need not hold in the real data-generating process.

Consider a market where every participant believes tulip bulbs are worth thirty years' labour. If some say twenty-nine and others thirty-one, they are indeed "right on average." But if all are wrong in the same direction—if the true value is closer to thirty *minutes*'—then averaging provides no correction. The errors must be independent for aggregation to help; systematic delusion aggregates to systematic mispricing.

Kahneman and Tversky's work on prospect theory documented precisely such systematic deviations[18]. Loss aversion causes investors to weight losses roughly 2.25 times more heavily than equivalent gains. Anchoring leads to insufficient adjustment from initial reference points. Availability bias overweights recent or salient information. These biases are not noise; they are consistent, directional, and do not cancel in aggregate. The errors have the same sign. The psychological dynamics of a "feeding frenzy" or a "don't miss out" stampede are harder to study, but intuitively, likely equally if not more powerful given the additional urgency of time pressure. Behavioural economists have been making these arguments for fifty years; the arguments have not changed, and neither has the curriculum.

The efficient market hypothesis has survived fifty years of such findings not through successful rebuttal, but through the *joint hypothesis problem*[76, 79]: any test of market efficiency is simultaneously a test of the asset pricing model used to define "efficient." If prices deviate from CAPM predictions, the deviation can always be attributed to a wrong model rather than a wrong market. This is not a loophole—it is the logical structure of the problem, and Linnainmaa[76] presents it as foundational. But it cuts both ways: the same joint hypothesis that protects the EMH from refutation also means that our positive tulip alpha (Section 6.2) is equally consistent with "tulips were underpriced" and "CAPM is the wrong model." As Popper reminded us, a theory that cannot be falsified is not science[39]—though he would add that a model need not be *true* to be useful, only closer to truth than its rivals (*verisimilitude*). The question is whether CAPM clears even that bar. What evidence would convince a CAPM adherent that the model is wrong? If the answer is "none," we are in the realm of doctrine, not discovery.

Our claim in this paper is therefore specific: the foundational mathematics of the Gaussian mean-variance CAPM implementation are empirically inadequate for real return data. That is a model rejection, not a universal disproof of informational efficiency under every conceivable model.

The deeper issue is philosophical. Hayek[74] argued that prices aggregate dispersed knowledge no central planner could replicate—the intellectual foundation of the efficient market hypothesis. If prices *are* information, then deviations from CAPM predictions reflect our ignorance, not the market's error. Keynes[75] saw something different: prices reflect conventions, animal spirits, and the dynamics of a beauty contest in which participants guess not what is beautiful but what others will guess is beautiful (Ch. 12). The EMH follows from Hayek; bubbles follow from Keynes. Both cannot be right about the same market at the same time—yet finance has spent seventy years citing both without resolving the contradiction.

What We Talk About When We Talk About Value. A common defence at this point retreats to definitions: "value is simply what people believe something to be." If so, the restriction $E[\epsilon_{i,t} | \mathcal{I}_{t-1}] = 0$ is rescued by fiat—there can be no systematic mispricing because "price" and "value" are synonyms. Bubbles become impossible by definition; the tulip speculators of 1637 were not irrational, they simply changed their minds. But notice what this definition cannot accommodate: being *wrong* about value. If value is constituted by belief, one cannot believe incorrectly about it—yet we plainly do speak this way.

Consider: if I think I have \$100 in my pocket, I might "value" my jeans at \$120—perhaps I would trade them for \$150, but not \$90. If I check the pockets later and find nothing there, did I *lose* something? Or did I come to realise that I never had what I believed I did? We speak of value as something that can be discovered, measured, lost, found, or deposited into

your holdings with no witnesses. This is not how we speak of attitudes or preferences. “Value” denotes something external—something humans can *want* and *expect* while being wrong about either.

This is not a peripheral observation. Lexicography, semiotics, and semantics all converge on the same principle: meaning is constituted by use. If speakers consistently use “value” to denote something one can be mistaken about—something that exists independently of belief—then that *is* what the word means. The circular definition (“value is what people believe”) is not merely imprecise; it contradicts how the term actually functions in the language. It is bad linguistics conscripted to save a lost argument—Dionysodorus[55] would approve.

CAPM cannot answer what value *is*, but the inadequately qualified use of it tacitly presumes to. It treats value as the central tendency of a population’s stable qualia as revealed through prices—except when markets deviate from predictions, at which point the deviation is labelled “alpha” or “anomaly” and quarantined from the theoretical space. This is taxonomy, not explanation.

A.3 What Is the Alternative?

We have spent considerable effort demonstrating that CAPM’s foundations are unsound. A reasonable reader might now ask: if not mean-variance optimisation, then what? This is the right question, and the honest answer is that the field has not been idle. Alternatives exist—mathematically rigorous, axiomatically grounded, and in some cases practically implemented. That they have not displaced the Gaussian framework says more about institutional inertia than about their validity.

A.3.1 Stable Portfolio Theory

Rachev, Mittnik, and Fabozzi[19] developed portfolio theory assuming Lévy stable distributions rather than Gaussians. The key insight: replace variance σ^2 with the *scale parameter* c from the stable distribution. Unlike variance, the scale parameter is always well-defined and estimable (it is related to the interquartile range, which converges even for fat-tailed distributions). The result is a *mean-scale efficient frontier* analogous to Markowitz’s mean-variance frontier, but mathematically consistent with the empirical evidence. Beta becomes $\beta_i^s = (c_i/c_m) \times \rho_{i,m}^s$, where ρ^s is an appropriate dependence measure. A reconstruction from first principles.

A.3.2 Downside Risk Measures

Markowitz himself, in his 1959 book *Portfolio Selection*, suggested that *semi-variance*—variance computed only over returns below the mean—might be more sensible than full variance. Investors do not dislike upside volatility; they dislike losses. This insight spawned a literature:

- **Lower Partial Moments** (Bawa 1975[20], Fishburn 1977[21]): Generalise semi-variance to $\text{LPM}_n(\tau) = E[\max(0, \tau - R)^n]$, measuring downside risk relative to a threshold τ .
- **Sortino Ratio**: Replaces standard deviation with downside deviation—reward per unit of *harmful* volatility, not total volatility.

These measures can remain useful under heavy tails, but finiteness depends on moment order. For an exact α -stable tail, LPM_n exists only if $n < \alpha$. Thus first-order downside moments may exist for $\alpha > 1$, while semivariance ($n = 2$) still fails whenever $\alpha < 2$.

A.3.3 Coherent Risk Measures and CVaR

In 1999, Artzner, Delbaen, Eber, and Heath[22] published a landmark paper asking: what properties *should* a risk measure satisfy? They proposed four axioms (monotonicity, sub-additivity,

positive homogeneity, translation invariance) and showed that Value at Risk (VaR) can violate sub-additivity for general loss distributions (especially outside elliptical settings). A portfolio can then exhibit higher measured VaR than the sum of its components, contradicting diversification logic.

Conditional Value at Risk (CVaR), also called *Expected Shortfall* (ES), satisfies all four axioms. It is defined as:

$$\text{ES}_\alpha(L) = \frac{1}{1-\alpha} \int_\alpha^1 \text{VaR}_u(L) du \quad (7)$$

which equals $E[L \mid L \geq \text{VaR}_\alpha]$ in the continuous case. CVaR/ES is well-defined for loss distributions with finite first moment—including Lévy-stable models with $\alpha > 1$. It does not require variance to exist. Rockafellar and Uryasev[23] showed that CVaR optimisation can be formulated as a linear program, making it computationally tractable. Basel III moved from VaR to Expected Shortfall for precisely these reasons.

A.3.4 Growth-Optimal Portfolios and the Kelly Criterion

In 1956, John Kelly[24] at Bell Labs asked a different question: what betting strategy maximises long-term wealth? In general portfolio form, Kelly chooses

$$w^* = \arg \max_w E[\log(1 + w^\top R)], \quad (8)$$

which maximises geometric growth. The familiar “edge divided by odds” formula is the binary-bet special case. If a bet pays net odds b with win probability p (loss probability $q = 1 - p$), the Kelly fraction is

$$f^* = \frac{bp - q}{b}. \quad (9)$$

Ed Thorp[25] applied Kelly to blackjack, then to warrants, then to convertible arbitrage, building one of the most successful hedge funds in history. The Kelly framework does not require finite variance; it requires a sufficiently well-specified return law, finite expected log-utility, and objective alignment with long-run compounding. Under those conditions (and usually i.i.d. assumptions), log-optimality²⁸ is provable. The logarithm also penalises large losses sharply, which makes Kelly naturally sensitive to tail risk.

A.3.5 Robust and Distributionally Robust Optimisation

If we are uncertain about the distribution itself, why assume one at all? Robust optimisation (Ben-Tal, El Ghaoui, Nemirovski[28]) asks: what portfolio performs best under the *worst-case* distribution within some ambiguity set? This min-max approach is conservative by design, but it does not require believing in any particular distribution—only specifying bounds on what distributions are plausible.

A.3.6 Extreme Value Theory

Embrechts, Klüppelberg, and Mikosch[29] developed rigorous tools for modelling distribution tails using Extreme Value Theory (EVT). The Peaks Over Threshold (POT) method fits a Generalised Pareto Distribution to exceedances above a high threshold, allowing tail risk to be estimated from extreme observations directly. This separates the modelling problem: use whatever distribution you like for the body, but let EVT handle the tails where fat-tailed behaviour matters most.

²⁸Log-optimality means choosing portfolio weights to maximise expected log wealth growth, i.e. the long-run geometric growth rate.

A.3.7 Jump-Diffusion (Merton)

Merton[30] proposed a pragmatic compromise: keep Gaussian diffusion for ordinary days, then superimpose rare jumps for extraordinary ones. In continuous time, returns are generated by a Brownian term plus a Poisson jump term,

$$\frac{dS_t}{S_t} = (\mu - \lambda\kappa)dt + \sigma dW_t + (J - 1)dN_t, \quad (10)$$

where N_t is a Poisson process with intensity λ , J is the random jump multiplier, and $\kappa = E[J - 1]$ adjusts drift so the process remains well-specified under no-arbitrage pricing. The model preserves tractability while allowing discontinuities, excess kurtosis, and occasional large moves that pure Gaussian dynamics cannot represent. In practice, it is widely used in option pricing and stress-aware risk work: not because it solves tail risk completely, but because it admits that markets are quiet until, suddenly, they are not.

A.3.8 Student's t and Stochastic Volatility (Heston)

Another practical route is to keep the diffusion framework but relax two Gaussian assumptions directly: thin tails and constant volatility. Student's t distributions (Gosset[31]) replace normal shocks with heavier-tailed innovations controlled by degrees of freedom, preserving finite variance when $\nu > 2$ while fitting empirically observed outliers far better than a normal law. This is often the first correction practitioners make when residuals are visibly leptokurtic.

Heston[32] relaxes the second assumption by making variance itself stochastic. Instead of fixed σ , volatility follows its own mean-reverting diffusion, allowing volatility clustering, skewed implied-volatility surfaces, and time-varying risk premia that constant-volatility models systematically miss. In spirit, this is the same move as the rest of this section: retain what is computationally useful, but stop pretending that calm periods and crisis regimes are generated by one stationary Gaussian mechanism.

A.3.9 Why Haven't These Displaced Mean-Variance?

Given this catalogue—developed over decades by serious researchers—why does mean-variance optimisation remain dominant in textbooks, CFA curricula, and industry practice? The question is not rhetorical. Linnainmaa[76] tells his own investments class that CAPM is “100% dead in the water as a model of expected returns”—and then teaches it anyway, because it remains the lingua franca. Fama and French[79], writing from within the tradition, concede that “the empirical record of the model is poor—poor enough to invalidate the way it is used in applications.”

Several reasons, none of them epistemic:

1. **Computational tractability:** Markowitz's quadratic program has closed-form solutions. CVaR requires linear programming; stable distributions require numerical integration. The Gaussian is *easy*. And even within the Gaussian framework, the optimiser amplifies estimation noise so aggressively that estimated tangency portfolio weights are, in Britten-Jones's[78] data, statistically indistinguishable from zero[76].
2. **Institutional inertia:** Curricula, regulations, and software are built around variance. Changing requires coordinated effort across education, industry, and regulators.
3. **Career risk:** As Keynes observed, “it is better to fail conventionally than to succeed unconventionally.” A portfolio manager using CVaR who underperforms faces harder questions than one using Sharpe ratios. These are Blake's mind-forg'd manacles[63].

4. **The illusion of precision:** Variance gives a single number. CVaR at what confidence level? Kelly with what edge estimate? Ambiguity sets of what size? More honest models require more judgement, which feels less “scientific.”

These are sociological explanations, not mathematical ones. The alternatives are inconvenient, not inferior. And so CAPM persists—the sneer of cold command[60], surveying works that will surely last.

Thomas Kuhn[44] would recognise the pattern. Scientific disciplines, he argued, do not progress through steady accumulation of knowledge but through periodic “paradigm shifts”—revolutionary changes in the basic framework through which a field interprets evidence. Between revolutions, scientists engage in “normal science”: puzzle-solving within the accepted paradigm. Anomalies—findings that contradict the paradigm—are typically quarantined, explained away, or ignored rather than taken as evidence against the framework itself. Only when anomalies accumulate beyond a critical threshold does the paradigm enter “crisis,” eventually replaced by a new framework.

The persistence of Gaussian assumptions in finance, despite sixty years of contrary evidence, fits Kuhn’s description of normal science protecting a paradigm under strain. The evidence against normality is not new, contested, or marginal. It is old, replicated, and overwhelming. Yet the textbooks remain unchanged. Victor Frankenstein[61] at least pursued his creature across the Arctic, accepting that what he had built was monstrous. The laureates whose legacy *is* the model have less incentive: Gilgamesh built the walls of Uruk[65] because he could not accept mortality, and a theorist cannot accept falsification without accepting that the monument will not outlast him. Fama, confronted with Thaler’s behavioural evidence, chose instead to redefine “efficiency” until the anomalies disappeared[80].

A.3.10 The Honest Position

We do not claim that any of these alternatives is perfect. Each requires estimation, and estimation under fat tails is inherently harder than under Gaussians. Confidence intervals are wider; more data is needed; judgement cannot be avoided. But these are the *tractable* problems—the ones mathematics can address. There is a deeper problem that no alternative can solve.

All the methods catalogued above—CVaR, Kelly, stable distributions, robust optimisation—share a common assumption: that returns are *samples from some stochastic process*. They disagree about which process (Gaussian, Lévy, unknown-but-bounded), but they agree that there *is* a process. The mathematics of probability applies to phenomena that can, in principle, be repeated under similar conditions: dice rolls, radioactive decay, perhaps even aggregated human behaviour in stable institutional contexts.

But a company’s value need not be generated by a stationary, ergodic process with time-invariant parameters²⁹. A company loses value if one of its regional managers decides to cook the books. It gains value if it hires an intern who is exceptional with LaTeX. It loses it again if the company car gets into a wreck. These events can be modelled probabilistically only after strong assumptions about state dynamics³⁰, institutions, and behaviour—assumptions that frequently change over time.

Schopenhauer[37] understood: models are *Vorstellung*; the regional manager’s fraud is *Wille*—the greed and fear, the animal spirits Keynes named but could not tame. We mistake the map for the territory.

Worse: things change. The Buddhists called it *anicca*[69]—the impermanence of all compounded things. “*The Dao that can be spoken is not the eternal Dao*”[68]. The α that can be

²⁹Here “stationary” means distributional structure is stable over time (often in the weaker sense of roughly stable first two moments), while “time-invariant” means model parameters do not drift materially over the estimation horizon.

³⁰State dynamics means the transition law across market regimes, such as low- and high-volatility states.

modelled is not the eternal α . Parameters drift (*non-stationarity*), so an α estimated from 2010 data may not describe the market of 2025.

Worse still: things change *because* we model them. Soros[35] formalised this as *reflexivity*: market participants do not passively observe prices; they act on them, and their actions change the prices they were observing. A model that correctly identifies $\alpha < 2$ invites hedging strategies against fat-tailed events; those strategies alter the distribution of returns; the α shifts. Goodhart[70] arrived at the same conclusion from monetary policy: once a measure becomes a target, it ceases to be a good measure. Variance became the target; it ceased to measure risk.

Frank Knight[34] distinguished *risk*—quantifiable probability over known outcomes—from *uncertainty*—situations where the outcomes themselves are unknown or the probabilities are fundamentally unmeasurable. The regional manager’s fraud is Knightian uncertainty: there is no meaningful probability distribution over “will Dave in accounting commit fraud next quarter?” The question is not even well-formed as a probabilistic query.

And even when the process is stable, the averages lie. Ole Peters[36] demonstrated that the *ensemble average*—the expected return across many investors at one moment—can differ from the *time average*—what happens to one investor across many moments. Consider a coin-flip game: heads you gain 50%, tails you lose 40%. The ensemble expected value per flip is +5%—profitable on average. But the geometric growth rate is $\sqrt{1.5 \times 0.6} - 1 \approx -5\%$: an individual playing repeatedly will almost certainly go broke. A strategy that is profitable “on average” can still be ruinous for anyone who actually follows it. This is *non-ergodicity*, and it means that maximising expected returns—the foundation of mean-variance optimisation—is not the same as maximising the wealth of any individual investor.

The Gaussian framework is Hesiod’s sealed pithos[56], concealing these problems behind precise-looking numbers. The alternatives we have catalogued are better because they are *honest about what they do not know*. CVaR acknowledges tail uncertainty. Kelly acknowledges estimation error through fractional betting. Robust optimisation explicitly models ambiguity. They do not eliminate Knightian uncertainty; nothing can. But they do not pretend it away.

Rumi[57]: “*Sell your cleverness and buy bewilderment.*”³¹ But here lies a danger. Faust³², who had “studied, alas, philosophy, law, medicine, and—to my sorrow—theology”[71], concluded that knowledge was futile and bargained with the devil for something more direct. Those who study tail risk too intently risk a similar bargain—the conviction that because models are always wrong, action informed by models is always foolish. This is the nihilist’s conclusion from fat tails: if variance is infinite, if the future is unknowable, if models are lies, why do anything at all?

But bewilderment and humility, however honest, is not a portfolio strategy. We owe the reader a concrete recommendation—and we can provide one, once we recognise a crucial asymmetry.

A.3.11 The Asymmetry: Why We Need Not Survive the Apocalypse

Knightian uncertainty, taken to its logical extreme, suggests paralysis. If the future is fundamentally unknowable, why invest at all? Hold cash, buy gold, build a bunker. But this counsel contains a self-defeating asymmetry.

Consider the scenarios we might face:

1. **Normal times:** Markets function, property rights hold, contracts are enforced. Investment strategies have meaning.

³¹We are aware this paper quotes ancient wisdom traditions at a rate that would make Taleb blush[50]. We maintain the convergence is not coincidental.

³²Nietzsche’s abyss[38] being the conventional reference, but at this point the abyss has gazed into so many LinkedIn posts that we felt a change of scenery was warranted.

2. **Severe stress:** Crashes, recessions, pandemics, wars. Markets are volatile but still exist. Investment strategies matter greatly—this is where fortunes are made and lost.
3. **Systemic collapse:** Global monetary system fails, property rights dissolve, social order breaks down. McCarthy’s[73] world, not Markowitz’s.

Here is the asymmetry: in scenario (3), *no financial strategy helps*. Your portfolio, however cleverly constructed, is worthless if the system that denominates and enforces it ceases to exist. Gold bars are heavy and attract violence. Cash is kindling. The only “investments” that matter are canned goods, ammunition, and trusted friends.

Pabrai[27] invokes the Chakravyuha[67]—the battle formation Abhimanyu could enter but not exit—as a warning: know your exit before you enter. Systemic collapse is a Chakravyuha with no exit within the domain of portfolio theory. There is no rebalancing that saves you when property rights dissolve. The exit is not a better hedge but a shift in frame—recognising, as the Gita[72] puts it, that you are entitled to the action, never to its fruits. The Hindu tradition calls this *līlā*[66]—the insight that creation is play, and the stakes that feel absolute from within dissolve upon stepping outside the frame. The question is not “how do I hedge against the end of the world?” but “why am I trying to?”

This is a scope clarification. We are not seeking a policy for all possible worlds. We are seeking a policy for the worlds in which investment remains a coherent concept—scenarios (1) and (2). Within that scope, Knightian uncertainty remains real, but it is *bounded* uncertainty. We do not need to hedge against the literal end of civilisation, because no hedge exists. We need to survive the fat tails, the regime changes, the frauds and crashes—the “normal abnormalities” of markets that remain markets. Ecclesiastes[59]: “*the race is not to the swift... time and chance happeneth to them all.*” Time and chance are precisely what we are optimising for.

This reframing changes the calculus. Extreme conservatism (100% cash, bunker mentality) sacrifices all upside to hedge against a scenario where the hedge is worthless anyway. The rational policy is to optimise for survival and growth *conditional on the system persisting*, while accepting that no policy survives the system’s total failure.

A.4 Recommendations

The argument above points toward what practitioners often call *convexity*: a payoff profile with limited losses in ordinary states and disproportionate gains in stress states. For an investor with high conviction that this convexity is underappreciated by the models predominantly used by the population, the question is practical before it is philosophical: what actions are indicated, and do instruments actually exist to capitalise on that view?

The closest analogue is insurance. Buying convexity is like buying storm coverage: you accept a known cost today for contingent protection tomorrow. Two issues then dominate realised outcomes. First is *counterparty risk*: will the entity selling protection be able to pay when the storm arrives? Second is *premium attrition*: will the cumulative carry cost of holding protection erode capital faster than the eventual payoff can compensate?

This exposes an asymmetry in market design. One can express bullish conviction by buying an asset with no explicit recurring premium, yet expressing bearish or crash conviction usually requires paying borrow, margin, or option premium. That asymmetry is not an accident; downside insurance consumes scarce balance-sheet capacity and therefore commands rent. The operational problem is not whether convexity should be free, but where that rent is mispriced.

Instrument choice matters:

- **Stocks:** linear long exposure (downside bounded at -100% , upside open), typically no explicit premium beyond financing and opportunity cost.
- **Call options:** explicit premium, limited downside, upside convexity; vulnerable to time decay and volatility overpricing.

- **Put options:** explicit premium for downside protection; the cleanest direct crash-insurance instrument, but carry bleed can dominate if strikes and maturities are poorly chosen.
- **Short selling:** often treated as bearish insurance, but payoff is linear rather than convex; includes borrow fees, recall risk, and potentially unbounded losses.
- **Gambling contracts:** can be convex in shape but usually embed negative expected value unless the bettor has a demonstrable informational edge.
- **Prediction markets:** increasingly relevant for event-specific convex views; they offer transparent odds and bounded loss, but introduce platform/counterparty, settlement, and liquidity risk.

The practical prescription is therefore not “buy convexity at any price,” but “buy convexity selectively where implied premia are low relative to your edge and payout reliability is credible.”

A.4.1 In Practice

Given the foregoing—fat tails, Knightian uncertainty, bounded scope, convexity—what should we actually do? We propose a synthesis:

Objective: Maximise long-term geometric growth (Kelly). The Kelly Criterion[24, 25] maximises the expected logarithm of wealth, which is equivalent to maximising the geometric mean of returns. Unlike arithmetic mean maximisation, Kelly is inherently ruin-averse: the logarithm sends wealth-to-zero to negative infinity, so any strategy with positive probability of ruin has expected log-utility of negative infinity. Kelly never bets everything; it naturally scales positions by edge and uncertainty.

Adjustment: Use fractional Kelly to account for estimation error. Full Kelly assumes you know the true probability distribution. You do not. Estimation error in edge or variance leads to overbetting, which can be catastrophic. The standard remedy is *fractional Kelly*: bet one-half or one-quarter of the Kelly-optimal amount. This sacrifices some growth for a large reduction in volatility and ruin probability. Half-Kelly, for instance, achieves 75% of the growth rate at half the volatility[26].

Constraint: Bound tail risk using CVaR. Kelly optimises growth but does not directly constrain tail risk. We supplement it with a CVaR (Expected Shortfall) constraint: the expected loss in the worst $\alpha\%$ of outcomes must not exceed some threshold. This is a coherent risk measure[22], well-defined under fat tails, and computationally tractable[23]. The constraint ensures that even if our distributional estimates are wrong, losses in extreme scenarios remain bounded.

Implementation: Robust estimation with regime awareness. Neither Kelly sizing nor CVaR constraints can be computed without *some* estimate of the return distribution. We recommend:

- Use stable distribution fits (not Gaussian) for tail modelling
- Apply Extreme Value Theory (POT methods)[29] for tail parameters specifically
- Estimate parameters over multiple time horizons and use the most conservative
- Explicitly model regime uncertainty: if parameters differ substantially across estimation windows, this is information—widen confidence intervals, reduce position sizes

This framework does not pretend to solve Knightian uncertainty. It acknowledges that we are estimating quantities that may not be stable, from data that may not be representative, for a future that may not resemble the past. But it does so *honestly*: fractional Kelly explicitly discounts for estimation error; CVaR explicitly bounds what we’re willing to lose; regime

awareness explicitly flags when our estimates are unreliable. The result is not optimal in any mathematical sense—there is no optimum under true uncertainty. It is, we submit, *reasonable*: a policy that grows wealth in good times, survives bad times, and does not pretend to survive the end of times.

Markowitz gave us mean-variance optimisation, and for sixty years we have pretended that variance is a meaningful quantity. Perhaps it is time to admit that the *Semper Augustus* of portfolio theory—beautiful, prized, and ultimately a product of disease—should be replaced with something homelier but honest. The Kelly-CVaR framework is not elegant. It does not produce a single efficient frontier or a closed-form solution. It requires judgement, conservatism, and the humility to say “I do not know.” Even Odin, who gave an eye for wisdom, could not read what the Norns weave[58].

A.5 Denouement: Implications for Modern Bubbles

If our analysis of Tulipmania seems absurd, consider its modern analogues. In 2021, a cryptocurrency called Dogecoin—created as a joke, featuring a Shiba Inu dog as its logo, with no technological innovation, revenue model, or ostensible utility—briefly achieved a market capitalisation exceeding \$85 billion. Shares of GameStop, a struggling video game retailer, rose over 1,600% in weeks as retail traders organised on Reddit to squeeze short sellers. Non-fungible tokens of cartoon apes sold for millions.

Were these *bubbles* (suggesting something separate from ourselves) or were they *behaviour*—things we embarrassingly but inescapably *chose to do*? If there is a way to mediate the massive cost brought about by financial crises it can only come from contending with this embarrassing reality: understanding our own nature rather than externalizing it. More pernicious than “greater fool” thinking is “other fools” thinking—which leads to solutions that address the symptoms at the fringes, while perpetuating (or exacerbating) the causes.

The appendices now shift from argumentative prose to formal support: first the tail-theoretic derivation, then the dependence-aware statistical validation protocol, then executable code and reproducible outputs.

Appendix groupings. For navigability, the appendices are organised into three categories:

- **Appendix B — Theory and inference:** tail derivation, statistical methodology, and the Gaussian contradiction theorem.
- **Appendix C — Data and computation pipelines:** distribution fitting, VOC/tulip/-commodity datasets, figure generation, and the CAPM/MVO pipeline.
- **Appendix D — Diagnostics:** diagnostic outputs for the CAPM/MVO pipeline.
- **Appendix E — Collected script sources:** full reproducibility listings.

B Theory and Inference

B.1 Tauberian Derivation of Power-Law Tails

This appendix provides the rigorous mathematical derivation that Lévy stable distributions exhibit power-law tails, and hence have infinite variance for stability parameter $\alpha < 2$.

The Characteristic Function. The Lévy stable distribution is most naturally defined by its characteristic function[47, 43]—the Fourier transform of the probability density:

$$\phi(t) = \exp[i\mu t - |ct|^\alpha (1 - i\beta \operatorname{sgn}(t) \Phi(\alpha, t))] \quad (11)$$

where $\alpha \in (0, 2]$ is the stability parameter, $\beta \in [-1, 1]$ controls skewness, $c > 0$ is the scale, μ is the location, and $\operatorname{sgn}(t)$ is the sign function (+1 if $t > 0$, -1 if $t < 0$, 0 if $t = 0$). The function $\Phi(\alpha, t)$ equals $\tan(\pi\alpha/2)$ for $\alpha \neq 1$, and $\frac{2}{\pi} \log |t|$ for $\alpha = 1$. The normal distribution is merely the special case where $\alpha = 2$. For $\alpha < 2$, a critical property emerges: the tails decay as a power law rather than exponentially. The rigorous derivation requires the machinery of *regular variation theory*[49].

Regular Variation and Tail Behaviour. A function $L : (0, \infty) \rightarrow (0, \infty)$ is *slowly varying* at infinity if, for all $\lambda > 0$:

$$\lim_{x \rightarrow \infty} \frac{L(\lambda x)}{L(x)} = 1 \quad (12)$$

Examples include constants, $\log x$, $\log \log x$, and products thereof. The defining property is that L grows (or decays) more slowly than any power of x . A function $f(x) \sim x^\rho L(x)$ is said to be *regularly varying* with index ρ .

The connection between characteristic function behaviour near the origin and tail behaviour at infinity is established by a Tauberian theorem of Karamata type[45, 46, 49, 48]. The precise statement requires care. Let $\phi(t)$ be the characteristic function of a random variable X . The relevant result is:

Theorem (Bingham, Goldie & Teugels, 1987, Theorem 8.1.6; symmetric case)³³.

Let X be a symmetric random variable with characteristic function $\phi(t)$. If for some $\alpha \in (0, 2)$ and slowly varying function L :

$$1 - \operatorname{Re}(\phi(t)) \sim |t|^\alpha L(1/|t|) \quad \text{as } t \rightarrow 0 \quad (13)$$

³³The constant $\frac{2\Gamma(1-\alpha)\cos(\pi\alpha/2)}{\pi}$ is the symmetric-case constant linking transform behaviour to two-sided tail asymptotics. See Feller[48], Vol. II, §XVII.5, equation (5.15). The qualitative result—regularly varying tails with exponent $-\alpha$ —does not depend on writing this exact constant.

then:

$$P(|X| > x) \sim \frac{2\Gamma(1-\alpha)\cos(\pi\alpha/2)}{\pi} \cdot x^{-\alpha}L(x) \quad \text{as } x \rightarrow \infty \quad (14)$$

This is the “hard” Tauberian direction—from transform behaviour to tail behaviour—and requires the regularity condition that L be slowly varying. The converse (tail behaviour implies transform behaviour) is an Abelian theorem and holds without additional conditions.

For Lévy stable distributions, we can verify the hypothesis directly. From the characteristic function, taking the real part of $\log \phi(t)$ for $\alpha \neq 1$:

$$\log |\phi(t)| = \operatorname{Re}(\log \phi(t)) = -c^\alpha |t|^\alpha \quad (15)$$

Hence $|\phi(t)| = e^{-c^\alpha |t|^\alpha}$, and for small $|t|$:

$$1 - \operatorname{Re}(\phi(t)) \approx 1 - |\phi(t)| \approx c^\alpha |t|^\alpha \quad (16)$$

The middle step—that $\operatorname{Re}(\phi(t)) \approx |\phi(t)|$ —requires justification. Writing $\phi(t) = |\phi(t)|e^{i\theta(t)}$ where $\theta(t) = \mu t + O(|t|^\alpha)$, we have:

$$1 - \operatorname{Re}(\phi(t)) = \underbrace{(1 - |\phi(t)|)}_{\sim c^\alpha |t|^\alpha} + \underbrace{|\phi(t)|(1 - \cos \theta(t))}_{\sim \theta(t)^2/2} \quad (17)$$

Since $\theta(t) = \mu t + O(|t|^\alpha)$, the second term is $O(|t|^{2\min(1,\alpha)})$. For $\alpha > 1$ this is $O(t^2)$, dominated by $|t|^\alpha$ because $\alpha < 2$ (raising a small number to a smaller exponent yields a larger result: e.g., $0.1^{1.5} \approx 0.032 > 0.1^2 = 0.01$). For $\alpha \leq 1$ it is $O(|t|^{2\alpha})$, dominated by $|t|^\alpha$ because $2\alpha > \alpha$. In both cases the first term dominates, and $1 - \operatorname{Re}(\phi(t)) \sim c^\alpha |t|^\alpha$ to leading order, which is all the Tauberian theorem requires.

Crucially, the slowly varying function L is simply a *constant*—stable distributions are the canonical “pure power law” case with no logarithmic corrections. The Tauberian theorem then yields:

$$P(|X| > x) \sim C_\alpha \cdot x^{-\alpha} \quad \text{as } x \rightarrow \infty \quad (18)$$

where $C_\alpha = \frac{2c^\alpha \Gamma(1-\alpha)\cos(\pi\alpha/2)}{\pi}$ in the symmetric case. Note that $\Gamma(1-\alpha)$ has a pole at $\alpha = 1$ (the Cauchy distribution), which is why the characteristic function requires separate treatment there (cf. the $\Phi(\alpha, t)$ term changing form at $\alpha = 1$). For $1 < \alpha < 2$, both $\Gamma(1-\alpha) < 0$ and $\cos(\pi\alpha/2) < 0$, so their product is positive as required for a probability. For skewed stable laws ($\beta \neq 0$), one-sided tails satisfy $P(X > x) \sim C_+ x^{-\alpha}$ and $P(X < -x) \sim C_- x^{-\alpha}$ with generally different constants; the exponent remains α . Since L is constant (no logarithmic corrections), differentiating the tail probability is straightforward, and the PDF satisfies:

$$f(x) \sim \alpha C_\alpha \cdot |x|^{-(\alpha+1)} \quad \text{as } |x| \rightarrow \infty \quad (19)$$

The constant is complicated, but the key result is the power-law decay $|x|^{-(\alpha+1)}$. We absorb the constants and write:

$$f(x) \sim K_\alpha \cdot |x|^{-(\alpha+1)} \quad \text{as } |x| \rightarrow \infty \quad (20)$$

Which Moments Exist. This derivation makes explicit why stable distributions have power-law tails: the characteristic function’s $|t|^\alpha$ behaviour near the origin, combined with the Karamata Tauberian machinery, forces the tail decay. The absence of a slowly varying factor (i.e., $L \equiv \text{const}$) is what makes stable distributions “pure” power laws, distinguishing them from distributions with logarithmic corrections. Given this tail behaviour, we can determine which moments exist. The n -th moment requires integrating $x^n f(x)$; substituting the power-law tail:

$$E[X^n] = \int_{-\infty}^{\infty} x^n f(x) dx \sim \int |x|^n \cdot |x|^{-(\alpha+1)} dx = \int |x|^{n-\alpha-1} dx \quad (21)$$

For large $|x|$, the integral $\int |x|^k dx$ converges if and only if $k < -1$ (since $\int_1^\infty x^k dx = \frac{x^{k+1}}{k+1} \Big|_1^\infty$, which is finite only when $k+1 < 0$). Applying this to our exponent $n - \alpha - 1$: the integral converges when $n - \alpha - 1 < -1$, i.e., when $n < \alpha$. Therefore:

- For $\alpha = 2$ (Gaussian): The power-law analysis does not apply; exponential tails ensure all moments converge.
- For $1 < \alpha < 2$: Only $E[X^n]$ with $n < \alpha$ converges. Since $\alpha < 2$, the second moment $E[X^2]$ diverges, hence variance is infinite. Since $\alpha < 2 < 3$, the third moment diverges, hence skewness is undefined. And so on for kurtosis and all higher moments.
- For $\alpha \leq 1$: Even the first moment ($n = 1$) diverges. The mean does not exist.

B.2 Statistical Methodology

This appendix moves from tail theory to finite-sample inference. Where Appendix B.1 establishes what follows mathematically under stable-tail assumptions, this section specifies how we test the required assumptions in dependent market data.

A Brief Primer on Moments. *Moments* are numerical summaries that describe the shape of a probability distribution. They are computed by taking expected values of powers of a random variable:

- **First moment** (mean, μ): $E[X]$ — the central location.
- **Second central moment** (variance, σ^2): $E[(X - \mu)^2]$ — the spread or dispersion.
- **Third standardised moment** (skewness): $E[(X - \mu)^3]/\sigma^3$ — asymmetry. Negative skewness means the left tail (large losses) is heavier than the right.
- **Fourth standardised moment** (kurtosis): $E[(X - \mu)^4]/\sigma^4$ — tail heaviness. Higher kurtosis means more probability mass in extreme outcomes.

Deductive Test Logic. The rejection logic for the Gaussian mean-variance CAPM package is conjunctive. Define

$$\mathcal{M} = A_1 \cap A_2 \cap A_3 \cap A_4, \quad (22)$$

where A_1 is finite and decision-estimable second moments, A_2 is mean-variance sufficiency, A_3 is approximate stationarity (time-invariance)/ergodicity over the estimation window, and A_4 is CAPM pricing-error structure ($E[\epsilon_{i,t} | \mathcal{I}_{t-1}] = 0$ and $\alpha_i = 0$). If any A_j fails, $\mathcal{M}$ fails.

Operationally, let H_j be the null that A_j holds, and $H_{\mathcal{M}} = H_1 \cap H_2 \cap H_3 \cap H_4$. We evaluate:

- H_1 via tail diagnostics³⁴ and precision diagnostics for covariance estimation under heavy tails.
- H_2 via incremental-value tests: if higher-moment or downside-risk terms add significant explanatory or out-of-sample utility value beyond mean-variance terms, mean-variance sufficiency fails.
- H_3 via stability diagnostics³⁵ on moments/tail parameters and break detection in regime dynamics.

³⁴Tail diagnostics include tail-index estimation, exceedance-rate checks at extreme quantiles, and tail-focused QQ/PP diagnostics.

³⁵Stability diagnostics include rolling-window parameter estimates and formal structural-break tests for regime shifts.

- H_4 via residual diagnostics³⁶ and HAC-robust tests of $\alpha_i = 0$ and residual unpredictability.

Let $p_1, \dots, p_4$ be the resulting p-values (or family-level p-values). We control family-wise error with Holm-Bonferroni at level α_F and reject $H_{\mathcal{M}}$ iff at least one H_j is rejected after correction. This yields a logically closed decision rule: conjunction fails if any necessary component fails, with finite-sample Type-I error controlled at α_F .

What cannot be made absolute is metaphysical certainty: with finite stochastic data, no empirical procedure eliminates Type-II error or model-class incompleteness, and no finite test can disprove every conceivable alternative model (the “prove a universal negative” problem). The strongest valid claim is therefore controlled statistical falsification of $\mathcal{M}$, not apodictic proof about all conceivable market models. Beyond that boundary we leave statistics and enter philosophy: the Cartesian floor (*cogito ergo sum*) marks the limit condition of certainty, not a defect of this specific pipeline.

Executed Validation Outcome. We implemented this exact pipeline in `validation_pipeline.py` and executed it on daily excess returns for SPY plus nine sector ETFs (XLB, XLE, XLF, XLI, XLK, XLP, XLU, XLV, XLY), 2002-01-03 to 2024-12-30 ($n = 5,787$). At $\alpha_F = 0.05$ with Holm control, A_1 , A_2 , and A_4 are rejected, while A_3 is not (group-level Holm-adjusted p-values: A_1 below machine precision, $A_2 = 5.99 \times 10^{-4}$, $A_3 = 0.838$, $A_4 = 3.92 \times 10^{-270}$). In this run, A_1 fails primarily on the “decision-estimable” component (strong conditional heteroskedasticity), even though the Hill tail estimate is above 2. This does not conflict with earlier stable-tail evidence: those claims concern a different model/data pairing and show that failure of $\mathcal{M}$ can arise through multiple channels. Therefore $H_{\mathcal{M}}$ is rejected in this sample.

Component	Holm-adjusted p	Reject at 5%?
A_1	$< 10^{-300}$	Yes
A_2	5.99×10^{-4}	Yes
A_3	0.838	No
A_4	3.92×10^{-270}	Yes

Table 5: Executed conjunction results for $\mathcal{M} = A_1 \cap A_2 \cap A_3 \cap A_4$ on 2002–2024 daily U.S. data. Rejection of any necessary component rejects $\mathcal{M}$.

The Kolmogorov-Smirnov Test. The Kolmogorov-Smirnov (K-S) test[40, 41] asks: “How well does a theoretical distribution fit observed data?” It compares the *empirical cumulative distribution function* (ECDF)—for each value x , what fraction of data points fall below x —against the *theoretical CDF* according to the proposed distribution. The test statistic D is the maximum vertical distance between these curves:

$$D = \sup_x |F_n(x) - F(x)| \quad (23)$$

where $F_n(x)$ is the empirical CDF and $F(x)$ is the theoretical CDF.³⁷ A smaller D means a better fit. If F is fully specified *a priori* (no parameters estimated from the sample), then under the null $D\sqrt{n}$ follows the Kolmogorov distribution[40], whose CDF is:

$$K(x) = 1 - 2 \sum_{k=1}^{\infty} (-1)^{k-1} e^{-2k^2 x^2} \quad (24)$$

³⁶Residual diagnostics test serial correlation, conditional heteroskedasticity, and violations of conditional-mean-zero restrictions.

³⁷The supremum differs from the maximum in that it exists even when the maximum is not attained, e.g. for open intervals. For CDFs on finite data, the distinction is immaterial.

In our setting, parameters are estimated from the same sample, so this reference distribution is not exact and plug-in p-values are anti-conservative. Formal inference should be bootstrap-calibrated:

$$\hat{p}_{\text{boot}} = \frac{1 + \sum_{b=1}^B \mathbf{1}\{D_b \geq D_{\text{obs}}\}}{B + 1}, \quad (25)$$

where each bootstrap draw simulates data from the fitted model, re-fits parameters, and recomputes D .

For financial returns this is still incomplete unless dependence is handled explicitly. Volatility clustering violates the i.i.d. premise of the vanilla one-sample K-S test.

Protocol for Valid K-S Inference with These Data.

1. Fit a conditional mean-volatility filter (e.g., ARMA-GARCH type) under each candidate innovation law (Gaussian, stable, etc.).
2. Compute standardised residuals $z_t = (r_t - \hat{\mu}_t)/\hat{\sigma}_t$.
3. Verify whitening diagnostics on z_t and z_t^2 ; if dependence remains, re-specify the filter and repeat.
4. Estimate innovation-distribution parameters on z_t by MLE and compute D_{obs} against the fitted innovation CDF.
5. Run a parametric bootstrap that preserves the full pipeline: simulate from fitted model, re-fit filter and distribution, recompute D_b .
6. Compute bootstrap p-values with the finite-sample correction above; report Monte Carlo uncertainty and use sufficiently large B .
7. If testing multiple families, control multiplicity (Holm-Bonferroni) before drawing acceptance/rejection conclusions.

Only this filtered-and-reestimated bootstrap procedure yields valid K-S inference for dependent return data.

Executed Dependence-Aware K-S Results. Applying this protocol with an AR(1)-GARCH(1,1) filter and $B = 1000$ bootstrap re-estimations to SPY excess returns ($n = 5,786$ standardised residuals), we obtain: normal innovations $D = 0.0386$, $\hat{p}_{\text{boot}} \leq 10^{-3}$; Student-t innovations $D = 0.0236$, $\hat{p}_{\text{boot}} \leq 10^{-3}$ with $\hat{\nu} \approx 6.03$. Ljung-Box diagnostics on filtered residuals are acceptable ($p = 0.271$ for z_t , $p = 0.095$ for z_t^2 under normal; $p = 0.256$ and $p = 0.273$ under Student-t), so rejection is not driven by obvious leftover serial dependence. The t model improves fit distance but remains rejected by this criterion.

Constructive Contradiction: Black Monday under the Gaussian Null. To show incompatibility in explicit contradiction form, define

$$H_G : r_t \stackrel{\text{i.i.d.}}{\sim} N(\mu, \sigma^2), \quad (26)$$

with (μ, σ) calibrated on S&P 500 log returns from 1950-01-04 to 1987-10-16. Let r_* be the observed Black Monday return on 1987-10-19. In our executed pipeline, $r_* = -0.228997$ (about -20.47% simple return), $\hat{\mu} = 2.98 \times 10^{-4}$, $\hat{\sigma} = 0.00787$, and a conservative 99% upper bound is $\sigma_U = 0.00802$. Under H_G ,

$$q = P(r_t \leq r_*) = \Phi\left(\frac{r_* - \mu}{\sigma}\right), \quad (27)$$

so with $\sigma = \sigma_U$ we get $z \approx -28.59$ and

$$\log_{10} q \approx -179.31. \quad (28)$$

Over $n = 9,496$ calibration days, the probability of at least one such event is approximately nq (valid because $q \ll 1$), yielding

$$\log_{10} P(\exists t \leq n : r_t \leq r_*) \approx -175.33. \quad (29)$$

This is effectively zero on any scientific significance scale. Hence the fixed-parameter i.i.d. Gaussian null is contradicted by the realised path; to evade contradiction one must leave that null class (e.g., time-varying volatility, jumps, or non-Gaussian innovations). For a formal boxed lemma-theorem statement of the same argument, see Appendix B.3.

Interpreting Our Results. For the raw-return fits, Gaussian gives $D = 0.0917$ and Lévy stable gives $D = 0.0256$ —a $3.6\times$ reduction in CDF distance. These remain informative descriptive distances. The dependence-aware bootstrap inference above confirms that Gaussian innovations are rejected even after conditioning on time-varying volatility; it also indicates that a simple Student-t innovation law, while better, is still insufficient.

The K-S test is most sensitive near the distribution’s centre and less sensitive in the tails—precisely where fat-tail risk matters. More tail-sensitive tests exist (Anderson-Darling, for instance), so we interpret K-S conservatively: it establishes relative fit, not ontological truth. The result is therefore comparative, not absolute: in the dependence-aware pipeline, Student-t dominates Gaussian; in the raw-return descriptive fit, Lévy stable dominates Gaussian by distance.

Computational Complexity. The timing disparity in Table 4 reflects fundamental mathematics, not implementation. Fitting a normal distribution requires computing two sufficient statistics (μ and σ) in closed form. Fitting a Lévy stable distribution requires numerical integration of the characteristic function for each likelihood evaluation. Scipy’s implementation uses Zolotarev’s method[43], which for $\alpha \neq 1$ requires numerically integrating:

$$f(x; \alpha, \beta) = \frac{\alpha}{\pi|\alpha - 1||x - \zeta|} \int_{-\theta_0}^{\pi/2} V(\theta) \exp(-V(\theta)) d\theta \quad (30)$$

where $\zeta = -\beta \tan(\pi\alpha/2)$, $\theta_0 = \frac{1}{\alpha} \arctan(\beta \tan(\pi\alpha/2))$, and

$$V(\theta) = \left(\frac{\cos(\alpha\theta_0)}{\sin(\alpha(\theta + \theta_0))} \right)^{\frac{\alpha}{\alpha-1}} \frac{\cos(\alpha\theta_0 + (\alpha - 1)\theta)}{\cos \theta} \quad (31)$$

This integral has no closed form and cannot be trivially parallelised. The Gaussian’s computational tractability is constitutive of its dominance: when the “correct” model takes orders of magnitude longer to fit, institutional pressure favours believing the fast model is “good enough.”

Taleb[50, 53] has documented how fat-tailed reality is forced onto the Procrustean bed of thin-tailed models. We have tried to play Theseus[54]—fitting the bed to the sleeper, not the reverse. That the result is attached to a paper about tulips is, we hope, not disqualifying; airport bookshops have set a generous precedent for mathematical arguments in unconventional packaging.

Full source is collected in Appendix E, Section E.0.1.

B.3 Formal Gaussian Contradiction Theorem

This appendix states the Black Monday incompatibility argument in explicit lemma-theorem form.

Let

$$H_G : r_t \stackrel{\text{i.i.d.}}{\sim} N(\mu, \sigma^2), \quad (32)$$

with (μ, σ) calibrated on S&P 500 log returns from 1950-01-04 through 1987-10-16. Let $r_* = -0.228997$ be the observed 1987-10-19 log return. Using the conservative bound $\sigma_U = 0.00802$ (99% upper confidence bound for σ),

$$q := P_{H_G}(r_t \leq r_*) = \Phi\left(\frac{r_* - \mu}{\sigma_U}\right) \quad (33)$$

satisfies

$$\log_{10} q \approx -179.31. \quad (34)$$

Proof. Substituting $\hat{\mu} = 2.98 \times 10^{-4}$ and $\sigma_U = 0.00802$ yields

$$z = \frac{r_* - \hat{\mu}}{\sigma_U} \approx -28.5868, \quad (35)$$

hence $q = \Phi(z)$ and therefore $\log_{10} q \approx -179.31$.

If per-period event probability is q and observations are independent, then over n periods

$$P(\exists t \leq n : E_t) = 1 - (1 - q)^n \leq nq, \quad (36)$$

with first-order expansion $1 - (1 - q)^n = nq + O(n^2 q^2)$ for $q \ll 1$.

Proof. The inequality is the union bound. The expansion follows from the binomial series.

Under H_G above, with $n = 9,496$ calibration days and conservative σ_U , the probability of at least one Black-Monday-scale event in an n -day horizon is bounded by

$$\log_{10} P_{H_G}(\exists t \leq n : r_t \leq r_*) - 175.33. \quad (37)$$

Therefore the realised Black Monday path is incompatible with the fixed-parameter i.i.d. Gaussian null at any practical significance level.

Proof. By Lemma B.1, $\log_{10} q \approx -179.31$. By Lemma B.2,

$$\log_{10} P(\exists t \leq n) \approx \log_{10}(nq) = \log_{10} n + \log_{10} q. \quad (38)$$

With $n = 9,496$, this gives $\log_{10} P \approx 3.98 - 179.31 = -175.33$. Hence the fixed-parameter i.i.d. Gaussian null is rejected; any viable model must leave that null class (e.g., stochastic volatility, jumps, or non-Gaussian innovations).

C Data and Computation Pipelines

C.1 Distribution Fitting Code

The following Python script fetches total market index data, fits normal and Lévy stable distributions to daily returns, and reports exploratory plug-in and i.i.d.-bootstrap K-S diagnostics. For formal inference with dependent returns, use the filtered-and-reestimated bootstrap protocol in Appendix B.2.

The distribution fitting script is provided in `appendix_e_distribution_fitting.py`. The source is included in the supplementary materials accompanying this paper.

Script: `appendix_e_distribution_fitting.py`
Outputs: `images/returns_distribution.png`; console fit statistics
Dependencies: `numpy, pandas, scipy, yfinance, matplotlib`

Full source is collected in Appendix E, Section E.0.2.

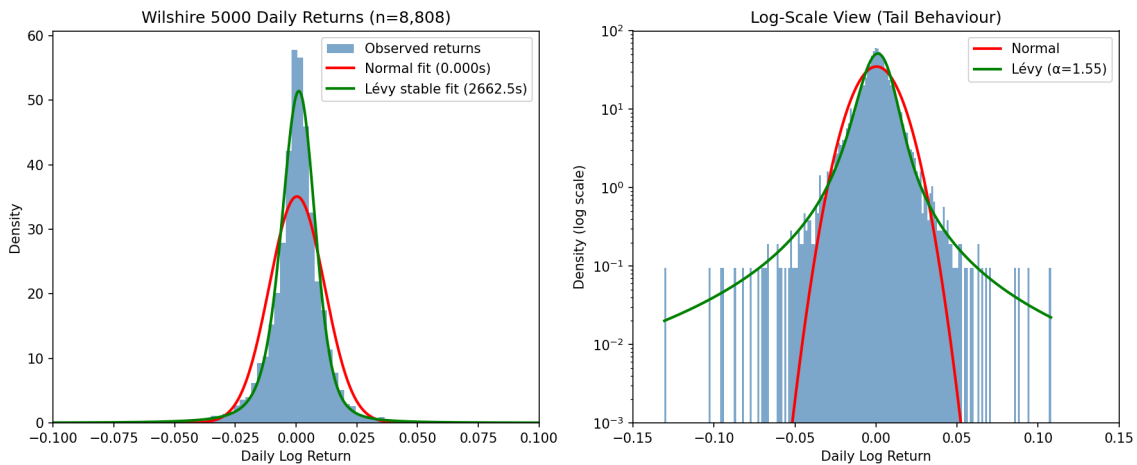


Figure 13: Output of the above script: Wilshire 5000 daily returns ($n=8,808$) with Normal (red) and Lévy stable (green, $\alpha = 1.55$) fits. Fitting took <0.001 s vs 2,663s respectively.

C.2 VOC Dataset Pipeline

This appendix provides the Python script used to fetch, clean, and compute returns from Petram’s[7] VOC share price dataset (1602–1698). The output, `data/voc_returns.csv`, is the market portfolio time series used in the CAPM and portfolio optimisation analyses in Sections 5 and 6.

The script fetches directly from the Google Sheets CSV export endpoint, requiring no API key. Dates are constructed using Python’s `datetime.date` rather than pandas `Timestamp`, since the latter cannot represent dates before 1677-09-21 (nanosecond precision limit). Missing dividends are treated as zero and flagged with a boolean column. No price gaps are interpolated; months without observations produce NaN returns.

Run with:

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
  pkgs.python313.withPackages (ps: with ps; [numpy pandas]))' \
  -c python appendix_f_voc_dataset.py
```

Full source is collected in Appendix E, Section E.0.3.

Script: `appendix_f_voc_dataset.py`
Outputs: `data/voc_returns.parquet`, `data/voc_returns.csv`
Dependencies: `numpy, pandas`

C.3 Tulip Dataset Pipeline

This appendix provides the Python script used to extract and process Smant’s[6] tulip price dataset, built on Garber[2, 3] and Posthumus[10]. The script produces two outputs:

- **data/tulip_index.csv** — the composite price index with returns (primary series for CAPM analysis)
- **data/tulip_raw.csv** — individual variety observations in long format (201 observations across 44 canonical varieties, Garber subset)

The composite index is Smant’s pre-computed aggregate, normalised to 100 on 05/02/1637 (the Alkmaar estate auction). Two series are available: **index_excl**, which excludes arbitrarily-dated observations, is used as the primary series for return calculations; **index_all** is retained for robustness. Dates are parsed as `datetime.date` objects for the same pre-1677 reason noted in Appendix C.2. Returns are computed between consecutive observations and **days_elapsed** is recorded to support annualisation.

A note on variety counts: Smant’s full dataset contains 246 observations across 70 apparent variety names. After excluding the 41 rows flagged **note=‘Added’** (Goldgar and other non-Garber additions) and stripping editorial annotations from variety names (e.g. bracketed date corrections and source attributions), the Garber canonical subset comprises 201 observations across 44 varieties. This differs from the 161/39 figure in Garber’s published table[2], reflecting Smant’s de-duplication and corrections to source attributions and weights. Section 4.2 retains Garber’s published figures as the cited source; Table 2 reflects the reconciled counts from the dataset.

Run with:

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
  pkgs.python313.withPackages (ps: with ps; [xlrld pandas numpy]))' \
  -c python appendix_g_tulip_dataset.py
```

Full source is collected in Appendix E, Section E.0.4.

Script: `appendix_g_tulip_dataset.py`
Outputs: `data/tulip_index.parquet`, `data/tulip_raw.csv`
Dependencies: `xlrld`, `pandas`, `numpy`

C.4 VOC Commodity Price Dataset

This appendix provides the Python script used to extract and process annual Amsterdam commodity prices from De Zwart’s[84] “VOC Prices in the Dutch-Asian Trade, 1608–1800” dataset, published by the International Institute of Social History (IISG) under a CC BY-SA 4.0 licence. All outputs of this script are derivative works under the same licence terms.

The dataset contains annual Amsterdam sale prices and Asian source prices for 16 commodities traded by the VOC. We extract the six commodities with complete Amsterdam price coverage for our 1630–1638 analysis window:

- **Spices:** Cinnamon, Cloves, Mace, Nutmeg, Pepper — the core VOC monopoly goods and the commodities occasionally used as in-kind dividend payments to VOC shareholders.
- **Sugar** — a significant trade commodity but not a VOC monopoly.

All prices are in Dutch florins (guilders) per pond. From these we construct an equal-weighted spice index (excluding sugar) and compute annual simple and log returns. Outputs:

- **data/commodity_prices.parquet** — canonical output with prices and returns for all six commodities plus the spice index.

- `data/commodity_prices_share.csv` — human-readable sharing copy with embedded attribution and licence notice.

For completeness, we also ran a commodity-augmented version of the dinner-party optimisation using the broad monthly commodity index from the aligned dataset. Holding fixed the same headline assumptions from the main text (cutoff 25 November 1636, $R_f = 5\%$, $\beta_{\text{tulip}} = 3.5$, pre-spike tulip variance, and zero tulip–VOC covariance), extending the risky set from two assets (tulips, VOC) to three assets (tulips, VOC, commodities) shifts the tangency portfolio from approximately 8.5% tulips to 4.06% tulips, with weights 75.94% VOC and 20.00% commodities (Sharpe = 2.497).

We do not elevate this specification to the headline result because the tulip–commodity covariance is identified on only three overlapping observations in the pre-cutoff window; it is therefore a fragile but directionally inconvenient robustness check, and we report it here rather than in Section 6.

A note on data quality: the nutmeg Amsterdam price is constant across 1630–1637 (2.4524 fl/pond), which is almost certainly an interpolation by De Zwart rather than a reflection of market conditions. See the Data Description Annex distributed with the source dataset for De Zwart’s interpolation and imputation methodology.

Run with:

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
  pkgs.python313.withPackages (ps: with ps; [numpy pandas openpyxl
    pyarrow]))' \
  -c python appendix_h_commodity_dataset.py
```

Full source is collected in Appendix E, Section E.0.5.

Script: `appendix_h_commodity_dataset.py`
Outputs: `data/commodity_prices.parquet`, `data/commodity_prices_share.csv`
Dependencies: `numpy`, `pandas`, `openpyxl`, `pyarrow`

C.5 Data Figures Pipeline

This appendix documents the Python script used to generate all data figures for Sections 4 and 5 of the paper. The script reads the cleaned datasets produced by Appendices C.2–C.4 and the CAPM results from Appendix C.6, and produces eight figures:

- `fig.voc_price.png` — VOC share price level, 1632–1638 (Figure 2)
- `fig.tulip_index.png` — Tulip composite index, Dec 1634 – Feb 1637 (Figure 1)
- `fig.overlay.png` — Normalised price series overlay (Figure 3)
- `fig.cumulative_returns.png` — Growth of one guilder comparison (Figure 4)
- `fig.beta_scatter.png` — Return scatter with OLS beta line (Figure 5)
- `fig.volatility.png` — Annualised volatility comparison with log scale (Figure 6)
- `fig.sml.png` — Security Market Line (Figure 7)
- `fig.alpha_sensitivity.png` — CAPM alpha across assumed betas (Figure 8)

All figures use a consistent academic style: serif fonts, muted colour palette, percentage-formatted axes, and italic subtitles with key parameter values.

Run with:

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
pkgs.python313.withPackages (ps: with ps; [numpy pandas pyarrow
matplotlib scipy]))' \
-c python appendix_h_figures.py
```

Full source is collected in Appendix E, Section E.0.6.

Script: `appendix_h_figures.py`
Outputs: 8 figures to `images/` (listed above)
Dependencies: `numpy, pandas, pyarrow, matplotlib, scipy`

C.6 CAPM and MVO Analysis Pipeline

This appendix documents the Python script that executes the full CAPM and mean-variance optimisation pipeline described in Section 5, using the pre-crash dataset (up to January 1637). The script computes:

- Market Risk Premium from VOC total returns (Section 5.4)
- OLS beta estimate for tulips vs the market (Section 5.3)
- CAPM alpha for tulips, pre-crash and full-cycle (Section 5.2)
- Sensitivity tables: alpha and tangency weights across $\beta = 0.5, 1.0, 1.5, 2.0$
- The dinner-party analysis at $\beta = 3.5$ (Section 5.3)
- Single-factor and sample-covariance tangency portfolios (Section 5.2)

All results are written to `data/capm_results.json` for downstream use by the figures pipeline (Appendix C.5) and the diagnostics pipeline (Appendix D.1).

Run with:

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
pkgs.python313.withPackages (ps: with ps; [numpy pandas pyarrow scipy
]))' \
-c python appendix_i_capm_analysis.py
```

Full source is collected in Appendix E, Section E.0.7.

Script: `appendix_i_capm_analysis.py`
Outputs: `data/capm_results.json, data/capm_results.txt`
Dependencies: `numpy, pandas, pyarrow, scipy`

D Diagnostics

D.1 Diagnostic Outputs (CAPM/MVO)

To accompany the core analysis, we generate a compact diagnostics pack containing: (i) an input summary table, (ii) covariance matrices for the two-asset and commodity-augmented three-asset specifications, (iii) alpha sensitivity output, (iv) an efficient-frontier figure (Figure 9), and (v) an out-of-sample path chart (Figure 10). These are produced by `appendix_k_diagnostics_outputs.py` and written to `data/` and `images/`.

Because the parameter choices in the main specification intentionally sit near aggressive but defensible ends of sparse historical estimates, we include both a point-estimate positioning panel (Figure 14) and a bootstrap visual diagnostic for recommended tulip weight (Figure 15).

Run with:

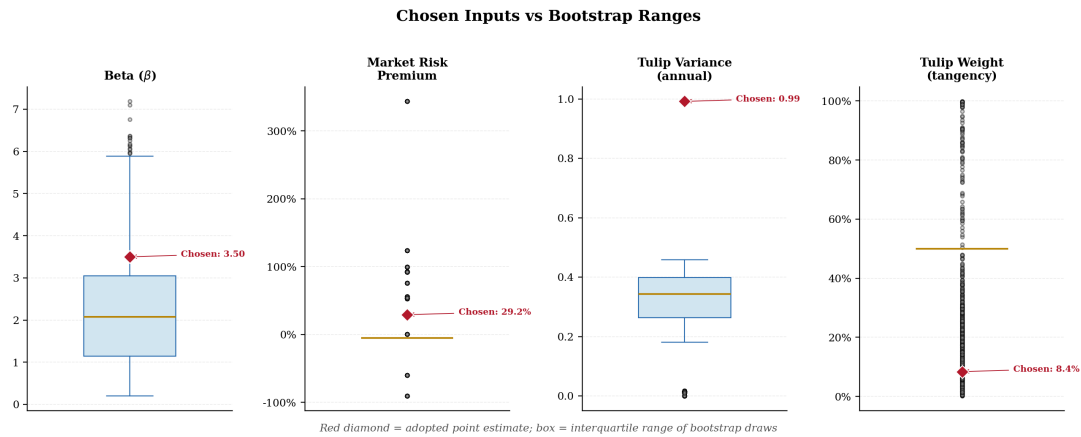


Figure 14: Point-estimate positioning panel. Each box shows the interquartile range of bootstrap draws for a key input parameter; whiskers extend to $1.5 \times$ IQR. The red diamond marks the adopted point estimate for the dinner-party analysis. The tulip variance panel illustrates the most extreme positioning: the adopted pre-spike variance (0.99) sits far above the bootstrap median, reflecting our choice to use a narrow variance window that excluded the January 1637 explosion. The fourth panel shows the *resulting* tangency weight—an output of the optimisation, not a selected input—to illustrate how input uncertainty propagates to the allocation decision.

```
nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
  pkgs.python313.withPackages (ps: with ps; [numpy pandas pyarrow
    matplotlib scipy])' \
  -c python appendix_k_diagnostics_outputs.py
```

Full source is collected in Appendix E, Section E.0.8.

Script: appendix_k_diagnostics_outputs.py

Outputs: data/input_summary_table.csv, data/weight_bootstrap_summary.csv, 4 figures to images/

Dependencies: numpy, pandas, pyarrow, matplotlib

E Collected Script Sources

This appendix consolidates full source listings referenced in the pipeline and diagnostics appendices.

E.0.1 Statistical Methodology Script

```
1 #!/usr/bin/env python3
2 """
3 Dependence-aware validation pipeline for CAPM/MPT assumptions.
4
5 Implements:
6 - Valid K-S inference under conditional heteroskedasticity via AR(1)
  - GARCH(1,1)
7   filter + full parametric bootstrap re-estimation.
8 - A1..A4 component tests and conjunction decision for M = A1 `` A2
  `` A3 `` A4.
9
10 Outputs JSON with all statistics and decisions.
11 """
```

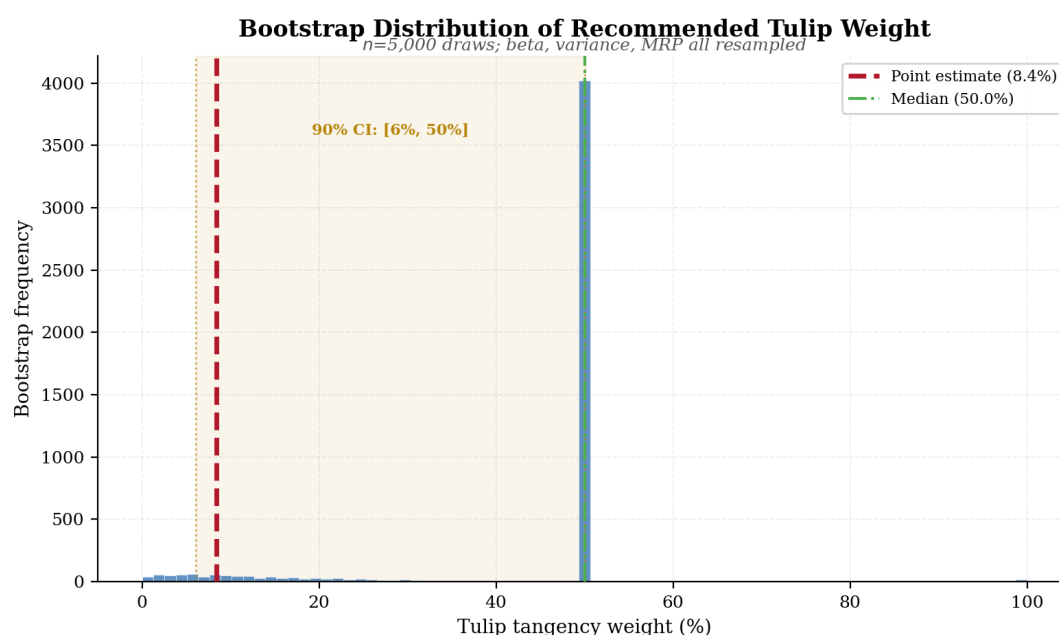


Figure 15: Bootstrap distribution ($n = 5,000$) of recommended tulip tangency weights under resampled inputs. Beta, tulip variance, MRP, and the risk-free rate are all resampled. The dashed red line marks the dinner-party point estimate (8.4%); the dash-dot green line marks the bootstrap median (50.0%). The shaded band shows the 90% confidence interval. The bimodal shape reflects the sensitivity of the long-only optimiser to the sign of the resampled MRP: when the resampled MRP is negative, the optimiser flips toward 50/50 equal-weight.

```

12
13 from __future__ import annotations
14
15 import argparse
16 import json
17 import time
18 from dataclasses import asdict, dataclass
19 from typing import Any
20
21 import numpy as np
22 import pandas as pd
23 import statsmodels.api as sm
24 import yfinance as yf
25 from arch import arch_model
26 from scipy import stats
27 from statsmodels.stats.diagnostic import acorr_ljungbox,
    breaks_cusumolsresid, het_arch
28 from statsmodels.stats.multitest import multipletests
29
30
31 MARKET_TICKER = "SPY"
32 SECTOR_TICKERS = ["XLB", "XLE", "XLF", "XLI", "XLK", "XLP", "XLU", "
    XLV", "XLY"]
33 RF_TICKER = "^IRX"
34
35
36 @dataclass
37 class KSResult:

```

```

38     dist: str
39     nobs: int
40     d_obs: float
41     p_boot: float
42     mc_se: float
43     ljung_box_p_resid: float
44     ljung_box_p_resid_sq: float
45     params: dict[str, float]
46     fit_time_seconds: float
47     bootstrap_time_seconds: float
48
49
50 def _extract_close(df: pd.DataFrame, ticker: str) -> pd.Series:
51     if df.empty:
52         raise RuntimeError(f"No rows downloaded for {ticker}")
53     cols = df.columns
54
55     if isinstance(cols, pd.MultiIndex):
56         if ("Close", ticker) in cols:
57             out = df[("Close", ticker)]
58         elif ("Adj Close", ticker) in cols:
59             out = df[("Adj Close", ticker)]
60         elif "Close" in cols.get_level_values(0):
61             out = df["Close"].iloc[:, 0]
62         elif "Adj Close" in cols.get_level_values(0):
63             out = df["Adj Close"].iloc[:, 0]
64         else:
65             out = df.iloc[:, 0]
66     else:
67         if "Close" in cols:
68             out = df["Close"]
69         elif "Adj Close" in cols:
70             out = df["Adj Close"]
71         else:
72             out = df.iloc[:, 0]
73
74     return out.dropna().astype(float)
75
76
77 def download_price_series(ticker: str, start: str, end: str) -> pd.
Series:
78     df = yf.download(ticker, start=start, end=end, auto_adjust=True,
79                       progress=False)
80     return _extract_close(df, ticker)
81
82 def build_excess_returns(
83     start: str, end: str
84 ) -> tuple[pd.DataFrame, pd.Series, list[str]]:
85     warnings: list[str] = []
86
87     tickers = [MARKET_TICKER] + SECTOR_TICKERS
88     prices: dict[str, pd.Series] = {}
89     for t in tickers:
90         try:
91             prices[t] = download_price_series(t, start, end)
92         except Exception as exc: # noqa: BLE001
93             warnings.append(f"Failed to download {t}: {exc}")

```

```

94
95 missing = [t for t in tickers if t not in prices]
96 if MARKET_TICKER in missing:
97     raise RuntimeError("Market series missing; cannot continue")
98
99 usable_assets = [t for t in SECTOR_TICKERS if t in prices]
100 if len(usable_assets) < 5:
101     raise RuntimeError(
102         f"Too few assets downloaded ({len(usable_assets)}). Need
          at least 5 sector series."
103     )
104
105 all_cols = [MARKET_TICKER] + usable_assets
106 rets = pd.concat(
107     {t: np.log(prices[t] / prices[t].shift(1)) for t in all_cols
108     },
109     axis=1,
110 ).dropna(how="any")
111
112 try:
113     rf_raw = download_price_series(RF_TICKER, start, end)
114     rf_daily = (rf_raw / 100.0) / 252.0
115     rf = rf_daily.reindex(rets.index).ffill().fillna(0.0)
116 except Exception as exc: # noqa: BLE001
117     warnings.append(
118         f"Failed to download {RF_TICKER}; using rf=0 fallback.
          Reason: {exc}"
119     )
120     rf = pd.Series(0.0, index=rets.index)
121
122 excess = rets.sub(rf, axis=0)
123 return excess, rf, warnings
124
125 def black_monday_gaussian_case() -> dict[str, Any]:
126     """
127     Contradiction test for i.i.d. Gaussian model calibrated pre-1987
128     crash.
129
130     Uses S&P 500 log returns:
131     - Calibration sample: up to 1987-10-16
132     - Event: 1987-10-19 (Black Monday)
133
134     Reports one-day tail and horizon exceedance probabilities under:
135     - sigma_hat (sample std)
136     - conservative sigma_upper_99 (99% upper CI bound)
137     """
138     px = download_price_series("^GSPC", "1950-01-01", "1987-10-21")
139     ret = np.log(px / px.shift(1)).dropna()
140
141     if pd.Timestamp("1987-10-19") not in ret.index:
142         raise RuntimeError("Black Monday return not found in
          downloaded series")
143
144     pre = ret.loc[:"1987-10-16"]
145     r_crash = float(ret.loc["1987-10-19"])
146     n = int(len(pre))

```

```

147
148 mu_hat = float(pre.mean())
149 s_hat = float(pre.std(ddof=1))
150
151 # Conservative upper bound: upper endpoint from a 99% two-sided
    CI for sigma
152 chi2_lower = stats.chi2.ppf(0.005, df=n - 1)
153 sigma_upper_99 = float(np.sqrt((n - 1) * s_hat * s_hat /
    chi2_lower))
154
155 def _tail_block(sig: float) -> dict[str, float]:
156     z = (r_crash - mu_hat) / sig
157     log_q = float(stats.norm.logcdf(z))
158     log10_q = float(log_q / np.log(10.0))
159     # For tiny q, p(any in n) " n q; reported in log10 scale
160     log10_p_any = float(np.log10(n) + log10_q)
161     return {
162         "z_score": float(z),
163         "log10_one_day_tail_prob": log10_q,
164         "log10_prob_at_least_one_in_n_days_approx": log10_p_any,
165     }
166
167 return {
168     "series": "^GSPC",
169     "calibration_start": str(pre.index.min().date()),
170     "calibration_end": "1987-10-16",
171     "event_date": "1987-10-19",
172     "n_calibration_days": n,
173     "event_log_return": float(r_crash),
174     "event_simple_return": float(np.exp(r_crash) - 1.0),
175     "mu_hat": mu_hat,
176     "sigma_hat": s_hat,
177     "sigma_upper_99": sigma_upper_99,
178     "under_sigma_hat": _tail_block(s_hat),
179     "under_sigma_upper_99": _tail_block(sigma_upper_99),
180 }
181
182
183 def hill_tail_test(series: pd.Series, tail_frac: float = 0.05) ->
dict[str, float]:
184     x = np.abs(series - series.median()).to_numpy()
185     x = np.sort(x[x > 0])
186     n = len(x)
187     k = max(80, int(tail_frac * n))
188     if n <= k + 1:
189         raise RuntimeError("Insufficient observations for Hill
    estimator")
190     u = x[-k - 1]
191     tail = x[-k:]
192     hill_inv = np.mean(np.log(tail / u))
193     alpha_hat = 1.0 / hill_inv
194     se = alpha_hat / np.sqrt(k)
195     z = (alpha_hat - 2.0) / se
196     p_one_sided = stats.norm.cdf(z) # H1: alpha < 2
197     return {
198         "alpha_hat": float(alpha_hat),
199         "se": float(se),
200         "k": int(k),

```

```

201         "p_h0_alpha_ge_2": float(p_one_sided),
202     }
203
204
205 def holm_adjust(p_values: list[float], alpha: float) -> tuple[list[
float], bool]:
206     if not p_values:
207         return [], False
208     _, adj, _, _ = multipletests(p_values, alpha=alpha, method="holm
")
209     adj_list = [float(x) for x in adj]
210     return adj_list, any(x < alpha for x in adj_list)
211
212
213 def fit_capm(y: pd.Series, m: pd.Series) -> Any:
214     df = pd.concat([y, m], axis=1).dropna()
215     yv = df.iloc[:, 0]
216     xv = sm.add_constant(df.iloc[:, 1])
217     return sm.OLS(yv, xv).fit(cov_type="HAC", cov_kws={"maxlags":
5})
218
219
220 def test_a1(market_excess: pd.Series, alpha: float) -> dict[str, Any
]:
221     hill = hill_tail_test(market_excess)
222     arch_stat = het_arch(market_excess.to_numpy(), nlags=20)
223     p_vals = [hill["p_h0_alpha_ge_2"], float(arch_stat[1])]
224     p_adj, reject = holm_adjust(p_vals, alpha)
225     return {
226         "description": "A1: finite and decision-estimable second
moments",
227         "tail_test": hill,
228         "arch_lm_p": float(arch_stat[1]),
229         "raw_p_values": p_vals,
230         "holm_adjusted_p_values": p_adj,
231         "reject_a1": reject,
232         "group_p": float(min(p_adj)) if p_adj else 1.0,
233     }
234
235
236 def test_a2(excess: pd.DataFrame, alpha: float) -> dict[str, Any]:
237     m = excess[MARKET_TICKER]
238     raw_p: list[float] = []
239     per_asset: list[dict[str, Any]] = []
240
241     for asset in [c for c in excess.columns if c != MARKET_TICKER]:
242         df = pd.DataFrame({"y": excess[asset], "m": m}).dropna()
243         df["m2"] = df["m"] ** 2
244         df["m3"] = df["m"] ** 3
245         x = sm.add_constant(df[["m", "m2", "m3"]])
246         fit = sm.OLS(df["y"], x).fit(cov_type="HAC", cov_kws={"
maxlags": 5})
247         wald = fit.wald_test("m2 = 0, m3 = 0", scalar=True)
248         p = float(wald.pvalue)
249         raw_p.append(p)
250         per_asset.append({"asset": asset, "wald_p": p})
251
252     p_adj, reject = holm_adjust(raw_p, alpha)

```

```

253     for row, p in zip(per_asset, p_adj, strict=True):
254         row["holm_p"] = p
255
256     return {
257         "description": "A2: mean-variance sufficiency",
258         "per_asset": per_asset,
259         "raw_p_values": raw_p,
260         "holm_adjusted_p_values": p_adj,
261         "reject_a2": reject,
262         "group_p": float(min(p_adj)) if p_adj else 1.0,
263     }
264
265
266 def test_a3(excess: pd.DataFrame, alpha: float) -> dict[str, Any]:
267     m = excess[MARKET_TICKER]
268     raw_p: list[float] = []
269     per_asset: list[dict[str, Any]] = []
270
271     for asset in [c for c in excess.columns if c != MARKET_TICKER]:
272         df = pd.DataFrame({"y": excess[asset], "m": m}).dropna()
273         x = sm.add_constant(df[["m"]])
274         fit = sm.OLS(df["y"], x).fit()
275         stat, p, _ = breaks_cusumolsresid(fit.resid, ddof=int(fit.
276             df_model) + 1)
277         p = float(p)
278         raw_p.append(p)
279         per_asset.append({"asset": asset, "cusum_stat": float(stat),
280             "cusum_p": p})
281
282     p_adj, reject = holm_adjust(raw_p, alpha)
283     for row, p in zip(per_asset, p_adj, strict=True):
284         row["holm_p"] = p
285
286     return {
287         "description": "A3: stationarity/time-invariance and
288             stability",
289         "per_asset": per_asset,
290         "raw_p_values": raw_p,
291         "holm_adjusted_p_values": p_adj,
292         "reject_a3": reject,
293         "group_p": float(min(p_adj)) if p_adj else 1.0,
294     }
295
296
297 def test_a4(excess: pd.DataFrame, alpha: float) -> dict[str, Any]:
298     m = excess[MARKET_TICKER]
299     raw_p: list[float] = []
300     per_asset: list[dict[str, Any]] = []
301
302     for asset in [c for c in excess.columns if c != MARKET_TICKER]:
303         df = pd.DataFrame({"y": excess[asset], "m": m}).dropna()
304         x = sm.add_constant(df[["m"]])
305         fit = sm.OLS(df["y"], x).fit(cov_type="HAC", cov_kws={"
306             maxlags": 5})
307
308         p_alpha = float(fit.pvalues["const"])
309         lb_p = float(

```

```

306         acorr_ljungbox(fit.resid, lags=[10], return_df=True)["
            lb_pvalue"].iloc[0]
307     )
308     arch_p = float(het_arch(fit.resid, nlags=10)[1])
309
310     raw_p.extend([p_alpha, lb_p, arch_p])
311     per_asset.append(
312         {
313             "asset": asset,
314             "alpha_hat": float(fit.params["const"]),
315             "alpha_p": p_alpha,
316             "ljungbox_p": lb_p,
317             "arch_lm_p": arch_p,
318         }
319     )
320
321     p_adj, reject = holm_adjust(raw_p, alpha)
322
323     idx = 0
324     for row in per_asset:
325         row["alpha_holm_p"] = p_adj[idx]
326         row["ljungbox_holm_p"] = p_adj[idx + 1]
327         row["arch_holm_p"] = p_adj[idx + 2]
328         idx += 3
329
330     return {
331         "description": "A4: CAPM pricing-error structure",
332         "per_asset": per_asset,
333         "raw_p_values": raw_p,
334         "holm_adjusted_p_values": p_adj,
335         "reject_a4": reject,
336         "group_p": float(min(p_adj)) if p_adj else 1.0,
337     }
338
339
340 def _ks_stat_from_resid(z: np.ndarray, dist: str, nu: float | None =
    None) -> float:
341     if dist == "normal":
342         return float(stats.kstest(z, "norm").statistic)
343     if dist == "t":
344         if nu is None or nu <= 2.0:
345             raise RuntimeError("Invalid nu for t distribution")
346         scale = np.sqrt((nu - 2.0) / nu)
347         cdf = lambda x: stats.t.cdf(x, df=nu, loc=0.0, scale=scale)
348         return float(stats.kstest(z, cdf).statistic)
349     raise ValueError(f"Unsupported dist: {dist}")
350
351
352 def fit_garch(series: pd.Series, dist: str) -> Any:
353     am = arch_model(
354         series.to_numpy() * 100.0,
355         mean="AR",
356         lags=1,
357         vol="GARCH",
358         p=1,
359         q=1,
360         dist=dist,
361         rescale=False,

```



```

362     )
363     return am.fit(dis="off")
364
365
366 def valid_ks_with_bootstrap(
367     series: pd.Series, dist: str, b_reps: int, seed: int
368 ) -> KSResult:
369     t0 = time.time()
370     fit = fit_garch(series, dist)
371     fit_time = time.time() - t0
372
373     z = pd.Series(fit.std_resid).dropna().to_numpy()
374     lb_z = float(acorr_ljungbox(z, lags=[10], return_df=True)["
375         lb_pvalue"].iloc[0])
376     lb_z2 = float(acorr_ljungbox(z**2, lags=[10], return_df=True)["
377         lb_pvalue"].iloc[0])
378
379     nu = float(fit.params.get("nu", np.nan)) if dist == "t" else
380         None
381     d_obs = _ks_stat_from_resid(z, dist, nu)
382
383     t1 = time.time()
384     d_boot = np.empty(b_reps)
385     rng = np.random.default_rng(seed)
386     for b in range(b_reps):
387         np.random.seed(int(rng.integers(0, 2**31 - 1)))
388         sim = fit.model.simulate(fit.params, nobs=len(series), burn
389             =300)
390         yb = np.asarray(sim["data"], dtype=float)
391         fit_b = arch_model(
392             yb, mean="AR", lags=1, vol="GARCH", p=1, q=1, dist=dist,
393             rescale=False
394         ).fit(dis="off")
395         zb = pd.Series(fit_b.std_resid).dropna().to_numpy()
396         nu_b = float(fit_b.params.get("nu", np.nan)) if dist == "t"
397             else None
398         d_boot[b] = _ks_stat_from_resid(zb, dist, nu_b)
399
400     boot_time = time.time() - t1
401     p_boot = float((1 + np.sum(d_boot >= d_obs)) / (b_reps + 1))
402     mc_se = float(np.sqrt(max(p_boot * (1.0 - p_boot), 1e-12) / (
403         b_reps + 1)))
404
405     params: dict[str, float] = {}
406     for k, v in fit.params.items():
407         params[str(k)] = float(v)
408
409     return KSResult(
410         dist=dist,
411         nobs=int(len(z)),
412         d_obs=float(d_obs),
413         p_boot=p_boot,
414         mc_se=mc_se,
415         ljung_box_p_resid=lb_z,
416         ljung_box_p_resid_sq=lb_z2,
417         params=params,
418         fit_time_seconds=float(fit_time),
419         bootstrap_time_seconds=float(boot_time),

```

```

413     )
414
415
416 def run_pipeline(
417     start: str, end: str, alpha: float, b_reps: int, seed: int
418 ) -> dict[str, Any]:
419     t0 = time.time()
420
421     excess, rf, warnings_list = build_excess_returns(start, end)
422     market = excess[MARKET_TICKER]
423
424     a1 = test_a1(market, alpha)
425     a2 = test_a2(excess, alpha)
426     a3 = test_a3(excess, alpha)
427     a4 = test_a4(excess, alpha)
428
429     group_ps = [a1["group_p"], a2["group_p"], a3["group_p"], a4["
430                  group_p"]]
431     _, group_adj, _, _ = multipletests(group_ps, alpha=alpha, method
432                                       ="holm")
433     group_adj = [float(x) for x in group_adj]
434
435     reject_flags = [a1["reject_a1"], a2["reject_a2"], a3["reject_a3"
436                  ], a4["reject_a4"]]
437     reject_m = any(reject_flags)
438
439     ks_normal = valid_ks_with_bootstrap(market, dist="normal",
440                                       b_reps=b_reps, seed=seed)
441     ks_t = valid_ks_with_bootstrap(market, dist="t", b_reps=b_reps,
442                                   seed=seed + 1)
443
444     try:
445         gaussian_case = black_monday_gaussian_case()
446     except Exception as exc: # noqa: BLE001
447         gaussian_case = {"error": str(exc)}
448
449     out: dict[str, Any] = {
450         "config": {
451             "start": start,
452             "end": end,
453             "alpha": alpha,
454             "bootstrap_reps": b_reps,
455             "seed": seed,
456             "market_ticker": MARKET_TICKER,
457             "assets": [c for c in excess.columns if c !=
458                       MARKET_TICKER],
459         },
460         "data": {
461             "n_obs": int(len(excess)),
462             "date_start": str(excess.index.min().date()),
463             "date_end": str(excess.index.max().date()),
464             "rf_mean_daily": float(rf.reindex(excess.index).mean()),
465             "warnings": warnings_list,
466         },
467         "a_tests": {
468             "A1": a1,
469             "A2": a2,
470             "A3": a3,

```

```

465         "A4": a4,
466     },
467     "conjunction": {
468         "group_p_values": {
469             "A1": float(group_ps[0]),
470             "A2": float(group_ps[1]),
471             "A3": float(group_ps[2]),
472             "A4": float(group_ps[3]),
473         },
474         "group_p_values_holm": {
475             "A1": float(group_adj[0]),
476             "A2": float(group_adj[1]),
477             "A3": float(group_adj[2]),
478             "A4": float(group_adj[3]),
479         },
480         "reject_flags": {
481             "A1": bool(reject_flags[0]),
482             "A2": bool(reject_flags[1]),
483             "A3": bool(reject_flags[2]),
484             "A4": bool(reject_flags[3]),
485         },
486         "reject_M": bool(reject_m),
487     },
488     "ks_validation": {
489         "normal": asdict(ks_normal),
490         "student_t": asdict(ks_t),
491     },
492     "gaussian_incompatibility_case_study": gaussian_case,
493     "timing_seconds": {
494         "total": float(time.time() - t0),
495     },
496 }
497 return out
498
499
500 def main() -> None:
501     parser = argparse.ArgumentParser(
502         description="Run dependence-aware validation pipeline"
503     )
504     parser.add_argument("--start", default="2002-01-01")
505     parser.add_argument("--end", default="2024-12-31")
506     parser.add_argument("--alpha", type=float, default=0.05)
507     parser.add_argument("--bootstrap-reps", type=int, default=300)
508     parser.add_argument("--seed", type=int, default=42)
509     parser.add_argument("--out", default="validation_results.json")
510     args = parser.parse_args()
511
512     results = run_pipeline(
513         start=args.start,
514         end=args.end,
515         alpha=args.alpha,
516         b_reps=args.bootstrap_reps,
517         seed=args.seed,
518     )
519
520     with open(args.out, "w", encoding="utf-8") as f:
521         json.dump(results, f, indent=2)
522

```

```

523     print(f"Wrote {args.out}")
524     print(json.dumps(results["conjunction"], indent=2))
525     print(json.dumps(results["ks_validation"], indent=2))
526
527
528 if __name__ == "__main__":
529     main()

```

E.0.2 Distribution Fitting Script

```

1  """
2  Fitting Normal vs Levy Stable Distributions to Market Returns
3
4  Generates Figure for "Buy the Tulip" paper.
5  Demonstrates computational cost of proper distribution fitting.
6  """
7  import time
8  import numpy as np
9  import pandas as pd
10 import yfinance as yf
11 import matplotlib.pyplot as plt
12 from scipy import stats
13 from scipy.stats import levy_stable
14
15 print("=" * 60)
16 print("Distribution Fitting: Normal vs L'vy Stable")
17 print("=" * 60)
18
19 # Fetch Wilshire 5000 Total Market Index (~W5000)
20 # Falls back to VTI (Vanguard Total Market ETF) if unavailable
21 print("\nFetching market data...")
22 t0 = time.time()
23 try:
24     data = yf.download("^W5000", start="1990-01-01", end="2024-12-31",
25                        auto_adjust=True, progress=False)
26     if len(data) == 0:
27         raise ValueError("No data")
28     ticker_name = "Wilshire 5000"
29 except:
30     data = yf.download("VTI", start="2001-06-01", end="2024-12-31",
31                        auto_adjust=True, progress=False)
32     ticker_name = "VTI Total Market"
33 t_fetch = time.time() - t0
34 print(f" Data fetched in {t_fetch:.2f}s")
35 print(f" Ticker: {ticker_name}")
36 print(f" Observations: {len(data)}")
37
38 # Calculate daily log returns (auto_adjust=True means 'Close' is
39 # already adjusted)
40 close_col = 'Close' if 'Close' in data.columns else data.columns[0]
41 returns = np.log(data[close_col] / data[close_col].shift(1)).dropna()
42
43 returns = returns.values.flatten()
44 n_returns = len(returns)
45 print(f" Returns computed: {n_returns}")
46
47 # Compute empirical moments

```

```

44 print("\n" + "-" * 60)
45 print("EMPIRICAL MOMENTS")
46 print("-" * 60)
47 mu_emp = np.mean(returns)
48 sigma_emp = np.std(returns)
49 skew_emp = stats.skew(returns)
50 kurt_emp = stats.kurtosis(returns) + 3 # scipy returns excess
    kurtosis
51
52 print(f" Mean:      {mu_emp:.6f}")
53 print(f" Std Dev:    {sigma_emp:.6f}")
54 print(f" Skewness:    {skew_emp:.4f} (normal = 0)")
55 print(f" Kurtosis:    {kurt_emp:.2f} (normal = 3)")
56
57 # Fit normal distribution
58 print("\n" + "-" * 60)
59 print("FITTING NORMAL DISTRIBUTION")
60 print("-" * 60)
61 t0 = time.time()
62 norm_params = stats.norm.fit(returns)
63 t_norm = time.time() - t0
64 print(f" Time: {t_norm:.4f} seconds")
65 print(f" ' = {norm_params[0]:.6f}")
66 print(f" ' = {norm_params[1]:.6f}")
67
68 # Fit Levy stable distribution (slow - uses MLE)
69 print("\n" + "-" * 60)
70 print("FITTING L' VY STABLE DISTRIBUTION")
71 print("-" * 60)
72 print(" (This requires numerical integration - please wait...)")
73 t0 = time.time()
74 levy_params = levy_stable.fit(returns)
75 t_levy = time.time() - t0
76 alpha, beta, loc, scale = levy_params
77 print(f" Time: {t_levy:.2f} seconds ({t_levy/60:.1f} minutes)")
78 print(f" ' (stability) = {alpha:.4f} (normal = 2)")
79 print(f" ' (skewness) = {beta:.4f}")
80 print(f" loc = {loc:.6f}")
81 print(f" scale = {scale:.6f}")
82
83 # Timing comparison
84 print("\n" + "-" * 60)
85 print("TIMING COMPARISON")
86 print("-" * 60)
87 print(f" Normal fit:      {t_norm:.4f} seconds")
88 print(f" L' vy stable fit: {t_levy:.2f} seconds")
89 print(f" Ratio:           {t_levy/t_norm:.0f}x slower")
90 print(f"\n This {t_levy/t_norm:.0f}x computational cost helps
    explain")
91 print(f" why Gaussian assumptions persist despite empirical
    falsification.")
92
93 # Goodness of fit (Kolmogorov-Smirnov test)
94 print("\n" + "-" * 60)
95 print("GOODNESS OF FIT (Kolmogorov-Smirnov)")
96 print("-" * 60)
97 ks_norm = stats.kstest(returns, 'norm', norm_params)
98 ks_levy = stats.kstest(returns, levy_stable.cdf, levy_params)

```

```

99 print(f"   Normal:           D = {ks_norm.statistic:.4f}, p = {ks_norm.
    pvalue:.2e}")
100 print(f"   L' vy stable: D = {ks_levy.statistic:.4f}, p = {ks_levy.
    pvalue:.2e}")
101
102 # Create figure
103 print("\n" + "-" * 60)
104 print("GENERATING FIGURE")
105 print("-" * 60)
106
107 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
108
109 # Left panel: histogram with fits
110 ax1 = axes[0]
111 x = np.linspace(returns.min(), returns.max(), 1000)
112
113 # Plot histogram
114 ax1.hist(returns, bins=100, density=True, alpha=0.7,
115         color='steelblue', label='Observed returns')
116
117 # Plot normal fit
118 norm_pdf = stats.norm.pdf(x, *norm_params)
119 ax1.plot(x, norm_pdf, 'r-', lw=2, label=f'Normal fit ({t_norm:.3f}s)
    ')
120
121 # Plot Levy stable fit
122 levy_pdf = levy_stable.pdf(x, *levy_params)
123 ax1.plot(x, levy_pdf, 'g-', lw=2, label=f'L' vy stable fit ({t_levy
    :.1f}s)')
124
125 ax1.set_xlabel('Daily Log Return')
126 ax1.set_ylabel('Density')
127 ax1.set_title(f'{ticker_name} Daily Returns (n={n_returns:,})')
128 ax1.legend()
129 ax1.set_xlim(-0.1, 0.1)
130
131 # Right panel: log-scale tails
132 ax2 = axes[1]
133
134 # Log-scale histogram to show tails
135 counts, bins, _ = ax2.hist(returns, bins=200, density=True,
136                             alpha=0.7, color='steelblue')
137 ax2.plot(x, norm_pdf, 'r-', lw=2, label='Normal')
138 ax2.plot(x, levy_pdf, 'g-', lw=2, label=f'L' vy ( '{alpha:.2f})')
139 ax2.set_yscale('log')
140 ax2.set_xlabel('Daily Log Return')
141 ax2.set_ylabel('Density (log scale)')
142 ax2.set_title('Log-Scale View (Tail Behaviour)')
143 ax2.legend()
144 ax2.set_xlim(-0.15, 0.15)
145 ax2.set_ylim(1e-3, 1e2)
146
147 plt.tight_layout()
148 plt.savefig('images/returns_distribution.png', dpi=150, bbox_inches=
    'tight')
149 print(f"   Saved to: images/returns_distribution.png")
150
151 print("\n" + "=" * 60)

```

```

152 print("SUMMARY TABLE (for LaTeX)")
153 print("=" * 60)
154 print(f"""
155 \\begin{{table}}[h]
156 \\centering
157 \\begin{{tabular}}{{|l|r|r|}}
158 \\hline
159 \\textbf{{Metric}} & \\textbf{{Normal}} & \\textbf{{L' vy Stable}}
160 \\\\\\\\thickhline
161 Fit time (seconds) & {t_norm:.4f} & {t_levy:.2f} \\\\\\\\hline
162 Skewness (theoretical) & 0 & {beta:.4f} \\\\\\\\hline
163 Kurtosis (theoretical) & 3 & $\\infty$ \\\\\\\\hline
164 K-S statistic & {ks_norm.statistic:.4f} & {ks_levy.statistic:.4f}
165 \\\\\\\\hline
166 K-S p-value & {ks_norm.pvalue:.2e} & {ks_levy.pvalue:.2e} \\\\\\\\
167 hline
168 \\end{{tabular}}
169 \\caption{{Distribution fitting results. Empirical: skewness = {
170     skew_emp:.4f}, kurtosis = {kurt_emp:.2f}. L' vy $\\alpha$ = {
171     alpha:.4f}.}}
172 \\label{{tab:fit}}
173 \\end{{table}}
174 """)
175 print("Done.")

```

E.0.3 VOC Dataset Script

```

1 """
2 appendix_f_voc_dataset.py
3 -----
4 Fetches Petram's VOC share price dataset (1602-1698) from Google
5 Sheets,
6 computes monthly total returns over the window 1630-1638, and writes
7 :
8
9     data/voc_returns.parquet    -- canonical output (typed schema,
10     compressed)
11     data/voc_returns_share.csv -- human-readable sharing copy with
12     header block
13
14 Source:
15     Petram, L. O. (2011). The World's First Stock Exchange. PhD
16     thesis,
17     Universiteit van Amsterdam.
18     Dataset: https://docs.google.com/spreadsheets/d/1XI6Muw-ubqPktgibFP2W3V3UheAez6hVU4YWz9jCVQ/
19
20 Units:
21     Prices are expressed as a percentage of the original 1602
22     subscription
23     price (3,150 guilders per full Amsterdam chamber share). A price
24     of 400
25     means the share traded at 400% of par, i.e. 12,600 guilders.
26
27     Dividends are recorded in the same units (% of par). Some
28     dividend

```

```

21     entries are Petram's reconstructed estimates from archival
22         sources
23     rather than directly observed cash payments; some were paid in
24         kind
25     (spices, bonds). We trust Petram's estimates and note this
26         limitation.
27
28 Return conventions:
29     - Missing dividends are treated as zero. A boolean column
30       'dividend_imputed' flags every such observation.
31     - Simple total return:  $r_t = (P_t - P_{t-1} + D_t) / P_{t-1}$ 
32     - Log return (ex-div):  $\log(P_t / P_{t-1})$ 
33     - Log total return:  $\log((P_t + D_t) / P_{t-1})$ 
34     - Simple returns used for portfolio optimisation arithmetic.
35     - Log returns used for OLS beta estimation.
36
37     Gaps in the price series produce NaN returns 'not
38         interpolated.
39
40 Date handling:
41     pandas Timestamp cannot represent dates before 1677-09-21 (
42         nanosecond
43         precision limit). Dates are stored as:
44     - date_str: ISO 8601 string YYYY-MM-DD (human-readable)
45     - date_ord: integer ordinal datetime.date.toordinal() (
46         machine-sortable)
47     Parquet stores both as native types. Read back via:
48     import datetime
49     date = datetime.date.fromordinal(row['date_ord'])
50
51 Window:
52     1630-01 to 1638-12. Pre-crash filter (up to 1637-01) applied in
53     '5 .
54
55 Run with:
56     nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
57     pkgs.python313.withPackages (ps: with ps; [numpy pandas
58     pyarrow]))' \
59     -c python appendix_f_voc_dataset.py
60
61 """
62
63 import datetime
64 import os
65 import numpy as np
66 import pandas as pd
67
68 # -----# 1. Fetch data
69 # -----
70 SHEET_ID = "1XIk6Muw-ubqPktgibFP2W3V3UheAez6hVU4YWz9jCVQ"
71 CSV_URL = f"https://docs.google.com/spreadsheets/d/{SHEET_ID}/export
72     ?format=csv&gid=0"
73
74 print("Fetching VOC price data from Google Sheets...")
75 print(f"    URL: {CSV_URL}")
76
77 try:
78     raw = pd.read_csv(CSV_URL)
79 except Exception as e:

```



```

70     raise RuntimeError(
71         f"Failed to fetch spreadsheet. Check the sheet is publicly "
72         f"accessible.\n Error: {e}"
73     )
74
75 print(f"    Fetched {len(row)} rows, columns: {list(row.columns)}")
76
77 # -----# 2. Normalise column
78     names
79 # -----
80 raw.columns = [
81     c.strip().lower().replace(" ", "_").replace("-", "_") for c in
82     raw.columns
83 ]
84
85 def find_col(df, candidates):
86     for c in candidates:
87         if c in df.columns:
88             return c
89     raise KeyError(f"Could not find any of {candidates} in columns:
90                    {list(df.columns)}")
91
92 year_col = find_col(row, ["year"])
93 month_col = find_col(row, ["month"])
94 price_col = find_col(
95     row,
96     [
97         "average_share_price_ex_dividend",
98         "average_share_price_exdividend",
99         "price",
100        "share_price",
101    ],
102 )
103 div_col = find_col(row, ["dividend", "div"])
104
105 # -----# 3. Parse dates via
106     datetime.date (pandas Timestamp pre-1677 limit)
107 # -----
108 year = pd.to_numeric(row[year_col], errors="coerce")
109 month = pd.to_numeric(row[month_col], errors="coerce")
110
111 def make_date(y, m):
112     try:
113         return datetime.date(int(y), int(m), 1)
114     except (ValueError, TypeError):
115         return None
116
117 dates = [make_date(y, m) for y, m in zip(year, month)]
118 n_bad = sum(1 for d in dates if d is None)
119 if n_bad:
120     print(f"    Warning: {n_bad} rows with unparseable year/month
121           dropped")
122
123 df = pd.DataFrame(

```

```

123     {
124         "date_str": [d.isoformat() if d else None for d in dates],
125         "date_ord": [d.toordinal() if d else None for d in dates],
126         "price": pd.to_numeric(raw[price_col], errors="coerce").
            values,
127         "dividend_raw": pd.to_numeric(raw[div_col], errors="coerce")
            .values,
128     }
129 )
130
131 df = df[df["date_ord"].notna()].copy()
132 df["date_ord"] = df["date_ord"].astype(int)
133 df = df.sort_values("date_ord").reset_index(drop=True)
134
135 print(f"   Date range: {df['date_str'].iloc[0]} to {df['date_str'].
        iloc[-1]}")
136 print(f"   Total observations:   {len(df)}")
137 print(f"   With price data:         {df['price'].notna().sum()}")
138 print(f"   With dividend data:      {df['dividend_raw'].notna().sum()}")
139
140 # -----# 4. Window filter:
141 #         1630-01-01 to 1638-12-31
142 # -----
143 W_START = datetime.date(1630, 1, 1).toordinal()
144 W_END = datetime.date(1638, 12, 31).toordinal()
145
146 df = df[(df["date_ord"] >= W_START) & (df["date_ord"] <= W_END)].
    copy()
147 df = df.reset_index(drop=True)
148
149 print(f"\nAfter window filter (1630-01 to 1638-12):")
150 print(f"   Observations:   {len(df)}")
151 print(f"   With price:         {df['price'].notna().sum()}")
152 print(f"   With dividend: {df['dividend_raw'].notna().sum()}")
153
154 # -----# 5. Dividend imputation:
155 #         missing '0, flagged
156 # -----
157 df["dividend_imputed"] = df["dividend_raw"].isna()
158 df["dividend"] = df["dividend_raw"].fillna(0.0)
159
160 n_imp = df["dividend_imputed"].sum()
161 print(f"\nDividend imputation: {n_imp} of {len(df)} observations
        treated as zero.")
162
163 # -----# 6. Return computation
164 # -----
165 price = df["price"]
166 div = df["dividend"]
167 prev_price = price.shift(1)
168
169 df["simple_return_exdiv"] = (price - prev_price) / prev_price
170 df["simple_total_return"] = (price - prev_price + div) / prev_price
171 df["log_return_exdiv"] = np.log(price / prev_price)
172 df["log_total_return"] = np.log((price + div) / prev_price)
173
174 n_ret = df["simple_total_return"].notna().sum()
175 print(f"Return observations computed: {n_ret} of {len(df)}")

```

```

174
175 # -----# 7. Summary statistics
176 # -----
177 priced = df[df["price"].notna()]
178 print(
179     f"\nFull window: {len(priced)} price observations, "
180     f"{priced['price'].min():.1f}--{priced['price'].max():.1f} % of
        par"
181 )
182
183 M_START = datetime.date(1634, 1, 1).toordinal()
184 M_END = datetime.date(1637, 1, 31).toordinal()
185 mania = df[(df["date_ord"] >= M_START) & (df["date_ord"] <= M_END)]
186 m_ret = mania[mania["simple_total_return"].notna()]
187
188 print(
189     f"Pre-crash window (1634-01 to 1637-01): "
190     f"{mania['price'].notna().sum()} price obs, "
191     f"{len(m_ret)} computable returns"
192 )
193
194 if len(m_ret) > 1:
195     ann = 12
196     mu, sd = m_ret["simple_total_return"].mean(), m_ret["
        simple_total_return"].std()
197     print(f" Mean monthly total return: {mu:.4f} ({mu * ann:.2%}
        ann.)")
198     print(f" Std monthly total return: {sd:.4f} ({sd * (ann**0.5)
        :.2%} ann.)")
199
200 # -----# 8. Write Parquet (
        canonical)
201 # -----
202 os.makedirs("data", exist_ok=True)
203
204 PARQUET_PATH = "data/voc_returns.parquet"
205
206 parquet_cols = [
207     "date_str",
208     "date_ord",
209     "price",
210     "dividend_raw",
211     "dividend",
212     "dividend_imputed",
213     "simple_return_exdiv",
214     "simple_total_return",
215     "log_return_exdiv",
216     "log_total_return",
217 ]
218
219 df[parquet_cols].to_parquet(PARQUET_PATH, index=False, compression="
    snappy")
220 print(f"\nCanonical output: {PARQUET_PATH}")
221 print(f" {len(df)} rows | columns: {parquet_cols}")
222 print(f" Read back: pd.read_parquet('{PARQUET_PATH}')

```

```

225 # -----# 9. Write sharing CSV (
      human-readable, self-documenting)
226 # -----
227 SHARE_PATH = "data/voc_returns_share.csv"
228
229 HEADER = ""\
230 # VOC Share Price Returns | Petram (2011) via Google Sheets
231 # Source:  https://docs.google.com/spreadsheets/d/1XI6Muv-
      ubqPktgibFP2W3V3UheAez6hVU4YWz9jCVQ/
232 # Built by: appendix_f_voc_dataset.py (INV-paper repo)
233 # Window:  1630-01 to 1638-12
234 #
235 # Columns:
236 #   date          ISO 8601 string, first of month (YYYY-MM-DD)
237 #   price          VOC share price, % of 1602 par (3150 guilders
      /share)
238 #   dividend       Dividend paid this month, % of par (0 if none
      recorded)
239 #   dividend_imputed TRUE if dividend was missing and set to zero
240 #   simple_total_return (P_t - P_{t-1} + D_t) / P_{t-1} [NaN if
      no prior price]
241 #   log_return_exdiv  log(P_t / P_{t-1}) [NaN if
      no prior price]
242 #   log_total_return  log((P_t + D_t) / P_{t-1}) [NaN if
      no prior price]
243 #
244 # Notes:
245 #   - Prices denominated as % of original 1602 subscription price
246 #   - Missing dividends treated as zero (see dividend_imputed flag)
247 #   - Some dividends paid in kind (spices/bonds); Petram's estimates
      used
248 #   - Only months with price observations appear; gaps not
      interpolated
249 #   - See Petram (2011) for full archival methodology
250 ""
251
252 share_cols = [
253     "date_str",
254     "price",
255     "dividend",
256     "dividend_imputed",
257     "simple_total_return",
258     "log_return_exdiv",
259     "log_total_return",
260 ]
261
262 share_df = df[parquet_cols].copy()
263 share_df = share_df.rename(columns={"date_str": "date"})
264
265 # Only rows with price or dividend data
266 has_data = share_df["price"].notna() | (share_df["dividend"] > 0)
267 share_df = share_df[has_data][
268     [
269         "date",
270         "price",
271         "dividend",
272         "dividend_imputed",
273         "simple_total_return",

```

```

274         "log_return_exdiv",
275         "log_total_return",
276     ]
277 ]
278
279 with open(SHARE_PATH, "w") as f:
280     f.write(HEADER)
281 share_df.to_csv(SHARE_PATH, mode="a", index=False, float_format="%.6
    f")
282
283 print(f"Sharing CSV:      {SHARE_PATH}")
284 print(f"    {len(share_df)} rows (price/dividend observations only)")
285
286 # -----# 10. Sanity check
287     printout
288 # -----
289 print(f"\nData rows:")
290 pd.set_option("display.float_format", "{:.4f}".format)
291 pd.set_option("display.max_rows", 30)
292 print(share_df.to_string(index=False))

```

E.0.4 Tulip Dataset Script

```

1  """
2  appendix_g_tulip_dataset.py
3  -----
4  Reads Smant's tulip price XLS file, extracts the composite price
5  index
6  and raw variety observations, computes returns, and writes:
7
8      data/tulip_index.parquet      -- composite index, canonical
9      data/tulip_raw.parquet       -- raw variety observations,
10      canonical
11      data/tulip_index_share.csv    -- sharing copy with header block
12      data/tulip_raw_share.csv     -- sharing copy with header block
13
14 Source:
15     Smant, D. (2012). Tulipmania. Personal research compilation.
16     Data: https://drive.google.com/file/d/1KaQxxo0b9NDkPWbUv\_bE7Q-4cU2xpvae/
17     Built on: Garber (1989, 2000), Posthumus (1929).
18
19 Index series (Graph sheet):
20     Two composite series, both normalised to 100 on 05/02/1637 (
21     Alkmaar
22     estate auction):
23     index_all  -- all 32 observations including arbitrarily-dated
24     ones
25     index_excl -- excluding arbitrarily-dated observations (
26     PRIMARY)
27     Returns computed between consecutive observations; days_elapsed
28     recorded for annualisation (series is irregular, not evenly
29     spaced).
30
31 Raw observations (Prices&Index sheet):
32     246 total observations across 70 apparent variety names.

```

```

27     Garber canonical subset (note != 'Added'): 201 obs / 44
        varieties
28     after stripping editorial annotations from variety names.
29
30 Date handling:
31     Dates stored as:
32         date_str: ISO 8601 string    DD/MM/YYYY -> YYYY-MM-DD
33         date_ord: integer ordinal    datetime.date.toordinal()
34     Read back: datetime.date.fromordinal(row['date_ord'])
35
36 Run with:
37     nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
38         pkgs.python313.withPackages (ps: with ps; [xlrld pandas numpy
39             pyarrow]))' \
40     -c python appendix_g_tulip_dataset.py [--local]
41
42 import argparse
43 import datetime
44 import os
45 import urllib.request
46
47 import numpy as np
48 import pandas as pd
49 import xlrld
50
51 # -----# 0. Configuration
52 # -----
53 FILE_ID = "1KaQxxo0b9NDkPWbUv_bE7Q-4cU2xpvae"
54 DL_URL = f"https://drive.google.com/uc?export=download&id={FILE_ID}"
55 LOCAL = "data/tulip_prices.xls"
56
57 parser = argparse.ArgumentParser()
58 parser.add_argument(
59     "--local", action="store_true", help="Use already-downloaded
        data/tulip_prices.xls"
60 )
61 args = parser.parse_args()
62
63 os.makedirs("data", exist_ok=True)
64
65 # -----# 1. Acquire XLS file
66 # -----
67 if args.local and os.path.exists(LOCAL):
68     print(f"Using local file: {LOCAL}")
69 else:
70     print(f"Downloading tulip price data from Google Drive...")
71     urllib.request.urlretrieve(DL_URL, LOCAL)
72     print(f"    Saved to {LOCAL} ({os.path.getsize(LOCAL) // 1024} KB)
73         ")
74
75 wb = xlrld.open_workbook(LOCAL)
76 print(f"    Sheets: {wb.sheet_names()}")
77
78 # -----# 2. Date parsing
79 # -----
80 def parse_date(s):

```

```

81     """Parse DD/MM/YYYY string to datetime.date, or None."""
82     try:
83         return datetime.datetime.strptime(str(s).strip(), "%d/%m/%Y"
84                                           ).date()
85     except (ValueError, TypeError):
86         return None
87
88 # -----# 3. Extract composite
89 # -----
90 print("\nExtracting composite index from Graph sheet...")
91
92 ws_graph = wb.sheet_by_name("Graph")
93
94 # Column layout:
95 #   col 5:  DATE string (index_all series)
96 #   col 6:  INDEX value (index_all)
97 #   col 10: INDEX value (index_excl  ‘‘  same dates, different
98 #           weighting)
99
100 index_rows = []
101 for r in range(3, ws_graph.nrows):
102     date_str = ws_graph.cell_value(r, 5)
103     if date_str == "":
104         continue
105     date = parse_date(date_str)
106     if date is None:
107         print(f"  Warning: unparseable date {date_str!r} at row {r}"
108               )
109         continue
110     idx_all = ws_graph.cell_value(r, 6)
111     idx_excl = ws_graph.cell_value(r, 10) if ws_graph.ncols > 10
112     else ""
113     index_rows.append(
114         {
115             "date_str": date.isoformat(),
116             "date_ord": date.toordinal(),
117             "index_all": float(idx_all) if idx_all != "" else np.nan
118             ,
119             "index_excl": float(idx_excl) if idx_excl != "" else np.
120                 nan,
121         }
122     )
123
124 idx_df = pd.DataFrame(index_rows).sort_values("date_ord").
125     reset_index(drop=True)
126
127 print(f"  Observations: {len(idx_df)}")
128 print(f"  Date range:   {idx_df['date_str'].iloc[0]} to {idx_df['
129     date_str'].iloc[-1]}")
130
131 # Verify normalisation on peak date
132 peak_ord = datetime.date(1637, 2, 5).toordinal()
133 peak = idx_df[idx_df["date_ord"] == peak_ord]
134 if len(peak):
135     print(

```

```

129         f" Peak (05/02/1637): index_all={peak['index_all'].values
130           [0]:.2f}, "
131         f"index_excl={peak['index_excl'].values[0]:.2f} (both
132           should be 100.0)"
133     )
134 # -----# 4. Compute returns on
135     index
136 # -----
137 prev_excl = idx_df["index_excl"].shift(1)
138 prev_all = idx_df["index_all"].shift(1)
139 prev_ord = idx_df["date_ord"].shift(1)
140
141 idx_df["days_elapsed"] = idx_df["date_ord"] - prev_ord
142 idx_df["simple_return"] = (idx_df["index_excl"] - prev_excl) /
143     prev_excl
144 idx_df["log_return"] = np.log(idx_df["index_excl"] / prev_excl)
145 idx_df["simple_return_all"] = (idx_df["index_all"] - prev_all) /
146     prev_all
147 idx_df["log_return_all"] = np.log(idx_df["index_all"] / prev_all)
148
149 n_ret = idx_df["simple_return"].notna().sum()
150 print(f" Returns computed: {n_ret} of {len(idx_df)}")
151
152 # Pre-crash summary
153 precrash_end = datetime.date(1637, 2, 4).toordinal()
154 precrash = idx_df[idx_df["date_ord"] <= precrash_end]
155 precrash_ret = precrash[precrash["simple_return"].notna()]
156
157 print(f"\nPre-crash window (up to 1637-02-04):")
158 print(f" Observations: {len(precrash)}")
159 print(f" Computable returns: {len(precrash_ret)}")
160 if len(precrash_ret) > 1:
161     ann_ret = precrash_ret["simple_return"] / (precrash_ret["
162         days_elapsed"] / 365.25)
163     print(f" Mean annualised return (index_excl): {ann_ret.mean()
164         :.2%}")
165     print(
166         f" Index range: {precrash['index_excl'].min():.2f} to "
167         f"{precrash['index_excl'].max():.2f} (normalised, peak=100)"
168     )
169
170 # -----# 5. Write index Parquet (
171     canonical)
172 # -----
173 IDX_PARQUET = "data/tulip_index.parquet"
174
175 idx_cols = [
176     "date_str",
177     "date_ord",
178     "index_all",
179     "index_excl",
180     "days_elapsed",
181     "simple_return",
182     "log_return",
183     "simple_return_all",
184     "log_return_all",
185 ]

```



```

179
180 idx_df[idx_cols].to_parquet(IDX_PARQUET, index=False, compression="
    snappy")
181 print(f"\nCanonical index: {IDX_PARQUET} ({len(idx_df)} rows)")
182
183 # -----# 6. Extract raw variety
    observations from Prices&Index sheet
184 # -----
185 print("\nExtracting raw observations from Prices&Index sheet...")
186
187 ws_pi = wb.sheet_by_name("Prices&Index")
188
189 # Column layout:
190 #   col 0: note ('Added' or blank)
191 #   col 1: date string
192 #   col 2: variety name
193 #   col 3: price in florins
194 #   col 4: weight in aas (numeric, or string like 'I bulb' for per-
    bulb)
195 #   col 5: source 1 / comments
196 #   col 6: source 2
197 #   col 7: contract type
198
199 raw_obs = []
200 for r in range(5, ws_pi.nrows):
201     variety = str(ws_pi.cell_value(r, 2)).strip()
202     if not variety:
203         continue
204     date = parse_date(ws_pi.cell_value(r, 1))
205     price_raw = ws_pi.cell_value(r, 3)
206     wt_raw = ws_pi.cell_value(r, 4)
207     try:
208         price = float(price_raw)
209     except (ValueError, TypeError):
210         price = np.nan
211     try:
212         weight = float(wt_raw)
213         per_aas = price / weight if (price and weight) else np.nan
214     except (ValueError, TypeError):
215         weight = per_aas = np.nan
216     raw_obs.append(
217         {
218             "date_str": date.isoformat() if date else None,
219             "date_ord": date.toordinal() if date else None,
220             "date_certain": date is not None,
221             "variety": variety,
222             "price_fl": price,
223             "weight_aas": weight,
224             "price_per_aas": per_aas,
225             "source1": str(ws_pi.cell_value(r, 5)).strip(),
226             "source2": str(ws_pi.cell_value(r, 6)).strip(),
227             "contract": str(ws_pi.cell_value(r, 7)).strip(),
228             "note": str(ws_pi.cell_value(r, 0)).strip(),
229         }
230     )
231
232 raw_df = pd.DataFrame(raw_obs)
233 # Sort by ordinal (None last)

```

```

234 raw_df = raw_df.sort_values(
235     "date_ord", key=lambda s: s.fillna(99999999).astype(int)
236 ).reset_index(drop=True)
237
238 print(f" Total observations: {len(raw_df)}")
239 print(f" With certain date: {raw_df['date_certain'].sum()}")
240 print(f" Unique variety names: {raw_df['variety'].nunique()}")
241 print(f" Garber subset (note != Added): {(raw_df['note'] != 'Added
    ').sum()} obs")
242
243 # -----# 7. Write raw Parquet (
    canonical)
244 # -----
245 RAW_PARQUET = "data/tulip_raw.parquet"
246
247 raw_cols = [
248     "date_str",
249     "date_ord",
250     "date_certain",
251     "variety",
252     "price_fl",
253     "weight_aas",
254     "price_per_aas",
255     "source1",
256     "source2",
257     "contract",
258     "note",
259 ]
260
261 raw_df[raw_cols].to_parquet(RAW_PARQUET, index=False, compression="
    snappy")
262 print(f"Canonical raw: {RAW_PARQUET} ({len(raw_df)} rows)")
263
264 # -----# 8. Write sharing CSVs
265 # -----
266 IDX_SHARE = "data/tulip_index_share.csv"
267 RAW_SHARE = "data/tulip_raw_share.csv"
268
269 IDX_HEADER = ""\
270 # Tulip Composite Price Index | Smant (2012) via Google Drive
271 # Source: https://drive.google.com/file/d/1KaQxxo0b9NDkPWbUv_bE7Q
    -4cU2xpvae/
272 # Built on: Garber (1989, 2000), Posthumus (1929)
273 # Built by: appendix_g_tulip_dataset.py (INV-paper repo)
274 #
275 # Index normalised to 100.0 on 05/02/1637 (Alkmaar estate auction
    peak).
276 # PRIMARY series: index_excl (excludes arbitrarily-dated
    observations).
277 # index_all retained for robustness checks.
278 #
279 # Columns:
280 # date ISO 8601 date string (YYYY-MM-DD)
281 # index_all Composite index, all observations
282 # index_excl Composite index, excl. arbitrarily-dated obs (
    PRIMARY)
283 # days_elapsed Calendar days since previous observation

```

```

284 #   simple_return      (index_excl_t - index_excl_{t-1}) / index_excl_
      {t-1}
285 #   log_return         log(index_excl_t / index_excl_{t-1})
286 #
287 # Notes:
288 #   - Series is irregular (not evenly spaced); annualise using
      days_elapsed
289 #   - No dividend equivalent; total return = price return throughout
290 #   - See Smant (2012) and Garber (1989, 2000) for full methodology
291 """
292
293 RAW_HEADER = """\
294 # Tulip Raw Price Observations | Smant (2012) via Google Drive
295 # Source:  https://drive.google.com/file/d/1KaQxxo0b9NDkPWbUv_bE7Q
      -4cU2xpvae/
296 # Built on: Garber (1989, 2000), Posthumus (1929), Goldgar (2007)
297 # Built by: appendix_g_tulip_dataset.py (INV-paper repo)
298 #
299 # 246 total observations; 201 in the Garber canonical subset (note
      != 'Added').
300 # 41 rows flagged note='Added' are Goldgar/other additions beyond
      Garber.
301 #
302 # Columns:
303 #   date              ISO 8601 date string (blank if date uncertain)
304 #   date_certain      TRUE if exact date known; FALSE if month/season
      only
305 #   variety           Tulip variety name (includes Smant editorial
      annotations)
306 #   price_fl          Price in Dutch florins (guilders)
307 #   weight_aas        Weight in aas (blank for per-bulb prices)
308 #   price_per_aas     price_fl / weight_aas (blank if per-bulb)
309 #   source1           Primary source citation
310 #   source2           Secondary source citation
311 #   contract          Contract type (spot, forward, etc.)
312 #   note              'Added' = Goldgar/other addition; blank =
      Garber original
313 #
314 # Notes:
315 #   - Prices quoted per bulb OR per aas depending on variety/source
316 #   - ~20% of observations have uncertain dates (date_certain=FALSE)
317 #   - See Smant (2012) and Garber (1989, 2000) for full methodology
318 """
319
320 # Index sharing CSV
321 with open(IDX_SHARE, "w") as f:
322     f.write(IDX_HEADER)
323 idx_share = idx_df[
324     [
325         "date_str",
326         "index_all",
327         "index_excl",
328         "days_elapsed",
329         "simple_return",
330         "log_return",
331     ]
332 ].copy()
333 idx_share = idx_share.rename(columns={"date_str": "date"})

```

```

334 idx_share.to_csv(IDX_SHARE, mode="a", index=False, float_format="%.6
    f")
335 print(f"\nSharing index CSV:  {IDX_SHARE}  ({len(idx_share)} rows)")
336
337 # Raw sharing CSV
338 with open(RAW_SHARE, "w") as f:
339     f.write(RAW_HEADER)
340 raw_share = raw_df[
341     [
342         "date_str",
343         "date_certain",
344         "variety",
345         "price_fl",
346         "weight_aas",
347         "price_per_aas",
348         "source1",
349         "source2",
350         "contract",
351         "note",
352     ]
353 ].copy()
354 raw_share = raw_share.rename(columns={"date_str": "date"})
355 raw_share.to_csv(RAW_SHARE, mode="a", index=False, float_format="%.6
    f")
356 print(f"Sharing raw CSV:      {RAW_SHARE}  ({len(raw_share)} rows)")
357
358 # -----# 9. Sanity check printout
359 # -----
360 print(f"\nComposite index:")
361 pd.set_option("display.float_format", "{:.4f}".format)
362 pd.set_option("display.max_rows", 35)
363 print(idx_share.to_string(index=False))

```

E.0.5 Commodity Dataset Script

```

1  """
2  appendix_h_commodity_dataset.py
3  -----
4  Extracts annual Amsterdam commodity prices from De Zwart's "VOC
    Prices in
5  the Dutch-Asian Trade, 1608-1800" dataset, computes annual returns
    over
6  the window 1625-1645 (with focus on 1630-1638), and writes:
7
8      data/commodity_prices.parquet    -- canonical output (typed
        schema, compressed)
9      data/commodity_prices_share.csv  -- human-readable sharing copy
        with header block
10
11 Source:
12     De Zwart, P. (2022). "VOC Prices in the Dutch-Asian Trade,
        1608-1800."
13     IISH Data Collection, V1. https://hdl.handle.net/10622/PEKT40
14
15     Licensed under CC BY-SA 4.0
16     (https://creativecommons.org/licenses/by-sa/4.0/).
17

```

```

18     This script and its outputs are derivative works of the above
19     dataset.
20
21     Any redistribution must preserve this attribution and licence
22     notice.
23
24 Units:
25     All Amsterdam prices are in Dutch florins (guilders) per pond.
26     Asian source prices are in the same units (converted by De Zwart
27     ).
28
29 Periodicity:
30     Annual. The summary spreadsheet provides one observation per
31     commodity
32     per year, derived from underlying transaction-level records in
33     the
34     individual commodity workbooks. See the Data Description Annex
35     for
36     De Zwart's interpolation and imputation methodology.
37
38 Commodities extracted:
39     Cinnamon, Cloves, Mace, Nutmeg, Pepper, Sugar ‘‘ the six
40     commodities
41     with complete Amsterdam price coverage for 1630-1638. These are
42     the
43     core VOC trade goods and the spices occasionally used as in-kind
44     dividend payments to VOC shareholders.
45
46 Window:
47     1625-01 to 1645-12 (broad); 1630-01 to 1638-12 (primary analysis
48     ).
49     The broader window provides context for return estimation.
50
51 Run with:
52     nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
53     pkgs.python313.withPackages (ps: with ps; [numpy pandas
54     openpyxl pyarrow]))' \
55     -c python appendix_h_commodity_dataset.py
56
57 """
58
59 import os
60
61 import numpy as np
62 import pandas as pd
63
64 # -----# 1. Configuration
65 # -----
66 SUMMARY_FILE = "data/iisg_voc_commodities/Zwart Dutch-Asiatic_P_1608
67 -1800.xlsx"
68
69 # Commodities with complete Amsterdam coverage in the 1630s
70 TARGET_COMMODITIES = ["Cinnamon", "Cloves", "Mace", "Nutmeg", "
71 Pepper", "Sugar"]
72
73 # Window
74 W_BROAD_START = 1625
75 W_BROAD_END = 1645
76 W_PRIMARY_START = 1630
77 W_PRIMARY_END = 1638

```

```

64
65 os.makedirs("data", exist_ok=True)
66
67 # -----# 2. Extract Amsterdam
   prices from summary spreadsheet
68 # -----
69 print("Reading De Zwart VOC commodity summary spreadsheet...")
70 print(f"   File: {SUMMARY_FILE}")
71
72 xls = pd.ExcelFile(SUMMARY_FILE)
73 all_sheets = [s for s in xls.sheet_names if s != "Sources"]
74 print(f"   Available commodity sheets: {all_sheets}")
75
76 records = []
77 for sheet in TARGET_COMMODITIES:
78     df = pd.read_excel(xls, sheet_name=sheet, skiprows=1)
79
80     # Column layout: year | source_price | amsterdam_price | markup
81     # Some sheets have fewer columns (e.g. Porcelain has only 2);
82     # we only target the 6 commodities known to have 4 columns.
83     cols = list(df.columns)
84
85     # Find Amsterdam column by name or position
86     adam_idx = None
87     for i, c in enumerate(cols):
88         if "amsterdam" in str(c).lower():
89             adam_idx = i
90             break
91     if adam_idx is None:
92         adam_idx = 2 # fallback: third column
93
94     source_idx = 1 # second column is always source/Asian price
95
96     year_vals = pd.to_numeric(df.iloc[:, 0], errors="coerce")
97     adam_vals = pd.to_numeric(df.iloc[:, adam_idx], errors="coerce")
98     source_vals = pd.to_numeric(df.iloc[:, source_idx], errors="
   coerce")
99
100     mask = (year_vals >= W_BROAD_START) & (year_vals <= W_BROAD_END)
101     for yr, adam, src in zip(year_vals[mask], adam_vals[mask],
   source_vals[mask]):
102         if pd.notna(yr):
103             records.append(
104                 {
105                     "year": int(yr),
106                     "commodity": sheet,
107                     "amsterdam_price_fl": adam if pd.notna(adam)
   else np.nan,
108                     "source_price_fl": src if pd.notna(src) else np.
   nan,
109                 }
110             )
111
112 prices_df = pd.DataFrame(records)
113 print(
114     f"\nExtracted {len(prices_df)} observations across {len(
   TARGET_COMMODITIES)} commodities"
115 )

```

```

116 print(f"   Year range: {prices_df['year'].min()}-{prices_df['year'].
      max()}")
117
118 # -----# 3. Pivot to wide format
      and compute returns
119 # -----
120 # Wide format: one row per year, one column per commodity
121 wide = prices_df.pivot_table(
122     values="amsterdam_price_fl", index="year", columns="commodity",
      aggfunc="first"
123 )
124 wide = wide.sort_index()
125
126 # Compute simple annual returns per commodity
127 returns = wide.pct_change(fill_method=None)
128 returns.columns = [f"{c}_return" for c in returns.columns]
129
130 # Log returns
131 log_returns = np.log(wide / wide.shift(1))
132 log_returns.columns = [f"{c}_log_return" for c in log_returns.
      columns]
133
134 # Equal-weighted spice index (Cinnamon, Cloves, Mace, Nutmeg, Pepper
      ' not Sugar)
135 spice_cols = [c for c in wide.columns if c != "Sugar"]
136 wide["spice_index"] = wide[spice_cols].mean(axis=1)
137 returns["spice_index_return"] = wide["spice_index"].pct_change()
138 log_returns["spice_index_log_return"] = np.log(
139     wide["spice_index"] / wide["spice_index"].shift(1)
140 )
141
142 # Combine into single dataframe
143 combined = wide.join(returns).join(log_returns)
144 combined = combined.reset_index()
145
146 print(f"\nWide table: {len(combined)} rows x {len(combined.columns)}
      columns")
147
148 # -----# 4. Summary statistics
149 # -----
150 primary = combined[
151     (combined["year"] >= W_PRIMARY_START) & (combined["year"] <=
      W_PRIMARY_END)
152 ]
153 print(f"\nPrimary window ({W_PRIMARY_START}-{W_PRIMARY_END}):")
154 print(f"   Observations: {len(primary)}")
155
156 for c in TARGET_COMMODITIES:
157     ret_col = f"{c}_return"
158     vals = primary[ret_col].dropna()
159     if len(vals) > 1:
160         print(
161             f"   {c:12s}: mean return {vals.mean():+.2%}, "
162             f"std {vals.std():.2%}, n={len(vals)}"
163         )
164
165 spice_ret = primary["spice_index_return"].dropna()
166 if len(spice_ret) > 1:

```

```

167     print(
168         f"    {'SPICE INDEX':12s}: mean return {spice_ret.mean()
169             :+.2%}, "
170         f"std {spice_ret.std():.2%}, n={len(spice_ret)}"
171     )
172 # -----# 5. Source price summary
173 # (Asian purchase prices)
174 # -----
175 source_wide = prices_df.pivot_table(
176     values="source_price_fl", index="year", columns="commodity",
177     aggfunc="first"
178 )
179 source_window = source_wide.loc[W_PRIMARY_START:W_PRIMARY_END]
180 print(f"\nAsian source prices available ({W_PRIMARY_START}-{
181     W_PRIMARY_END}):")
182 for c in TARGET_COMMODITIES:
183     if c in source_window.columns:
184         n_src = source_window[c].notna().sum()
185         print(f"    {c:12s}: {n_src}/{len(source_window)} years with
186             source price")
187
188 # -----# 6. Write Parquet (
189 # canonical)
190 # -----
191 PARQUET_PATH = "data/commodity_prices.parquet"
192
193 combined.to_parquet(PARQUET_PATH, index=False, compression="snappy")
194 print(f"\nCanonical output: {PARQUET_PATH}")
195 print(f"    {len(combined)} rows | {len(combined.columns)} columns")
196
197 # -----# 7. Write sharing CSV (
198 # human-readable, self-documenting)
199 # -----
200 SHARE_PATH = "data/commodity_prices_share.csv"
201
202 HEADER = ""
203 # VOC Commodity Prices (Amsterdam) | De Zwart (2022) via IISG
204 # Source: https://hdl.handle.net/10622/PEKT40
205 # Licence: CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa
206     /4.0/)
207 # Built by: appendix_h_commodity_dataset.py (INV-paper repo)
208 # Window: {broad_start} to {broad_end} (primary: {primary_start}-{
209     primary_end})
210 #
211 # Citation:
212 # De Zwart, P. (2022). "VOC Prices in the Dutch-Asian Trade,
213     1608-1800."
214 # IISH Data Collection, V1. https://hdl.handle.net/10622/PEKT40
215 #
216 # Commodities: Cinnamon, Cloves, Mace, Nutmeg, Pepper, Sugar
217 # All Amsterdam prices in Dutch florins (guilders) per pond.
218 # spice_index: equal-weighted mean of Cinnamon, Cloves, Mace, Nutmeg
219     , Pepper
220 # (excludes Sugar, which is not a VOC monopoly spice)
221 #
222 # Returns are simple annual returns: (P_t - P_{{t-1}}) / P_{{t-1}}
223 # Log returns: log(P_t / P_{{t-1}})

```



```

214 # First year of each series has NaN returns (no prior price).
215 #
216 # Notes:
217 #   - Periodicity is annual; sub-annual resolution not available
218 #   - Some prices are De Zwart's interpolations/imputations (see
    Data Description Annex)
219 #   - Nutmeg Amsterdam price is constant 1630-1637 (likely
    interpolated)
220 #   - This file is a derivative work under CC BY-SA 4.0
221 """ .format(
222     broad_start=W_BROAD_START,
223     broad_end=W_BROAD_END,
224     primary_start=W_PRIMARY_START,
225     primary_end=W_PRIMARY_END,
226 )
227
228 with open(SHARE_PATH, "w") as f:
229     f.write(HEADER)
230 combined.to_csv(SHARE_PATH, mode="a", index=False, float_format="%.6
    f")
231
232 print(f"Sharing CSV:      {SHARE_PATH}")
233 print(f"    {len(combined)} rows")
234
235 # -----# 8. Sanity check printout
236 # -----
237 print(f"\nAmsterdam prices ({W_PRIMARY_START}-{W_PRIMARY_END}):")
238 pd.set_option("display.float_format", "{:.4f}".format)
239 pd.set_option("display.max_columns", 20)
240 pd.set_option("display.width", 140)
241 display_cols = ["year"] + TARGET_COMMODITIES + ["spice_index"]
242 print(primary[display_cols].to_string(index=False))
243
244 print(f"\nAnnual returns ({W_PRIMARY_START}-{W_PRIMARY_END}):")
245 ret_cols = (
246     ["year"] + [f"{c}_return" for c in TARGET_COMMODITIES] + [
        "spice_index_return"
    ]
247 )
248 print(primary[ret_cols].to_string(index=False))

```

E.0.6 Figures Pipeline Script

```

1 """
2 appendix_h_figures.py
3 -----
4 Generates all data figures for '4''5 (Sources, Data, and Results
    ) of the paper.
5
6 Outputs (written to images/):
7     fig_voc_price.png           -- VOC share price level, 1632-1638
8     fig_tulip_index.png         -- Tulip composite index, Dec 1634
9     - Feb 1637
10    fig_overlay.png              -- Both series normalised to 100 at
11    Dec 1634
12    fig_beta_scatter.png         -- Return scatter with OLS beta
13    line + CI
14    fig_volatility.png           -- Annualised volatility comparison

```

```

12     fig_sml.png                -- Security Market Line (CAPM
        canonical chart)
13     fig_cumulative_returns.png -- Growth of 1 guilder in each
        asset
14     fig_alpha_sensitivity.png  -- Alpha vs assumed beta
15
16 Run with:
17     nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
18         pkgs.python313.withPackages (ps: with ps; [numpy pandas
        pyarrow matplotlib scipy]))' \
19         -c python appendix_h_figures.py
20 ""
21
22 import datetime
23 import json
24 import os
25 import warnings
26
27 import matplotlib
28
29 matplotlib.use("Agg")
30 import matplotlib.pyplot as plt
31 import matplotlib.patches as mpatches
32 import matplotlib.ticker as mticker
33 import numpy as np
34 import pandas as pd
35
36 warnings.filterwarnings("ignore")
37
38 os.makedirs("images", exist_ok=True)
39
40 # -----# Shared academic style
41 # -----
42 plt.rcParams.update(
43     {
44         "font.family": "serif",
45         "font.size": 10,
46         "axes.titlesize": 12,
47         "axes.titleweight": "bold",
48         "axes.labelsize": 10,
49         "axes.spines.top": False,
50         "axes.spines.right": False,
51         "axes.grid": True,
52         "grid.alpha": 0.25,
53         "grid.linestyle": "--",
54         "grid.linewidth": 0.6,
55         "figure.dpi": 200,
56         "savefig.dpi": 200,
57         "savefig.bbox": "tight",
58         "legend.fontsize": 8.5,
59         "legend.framealpha": 0.85,
60         "legend.edgecolor": "#cccccc",
61         "xtick.labelsize": 9,
62         "ytick.labelsize": 9,
63     }
64 )
65
66 # Colour palette ‘‘ consistent across all figures

```

```

67 VOC_BLUE = "#2166ac"
68 TULIP_RED = "#b2182b"
69 GREY = "#555555"
70 LIGHT_GREY = "#999999"
71 DINNER_GOLD = "#b8860b"
72 COMMODITY_GREEN = "#4daf4a"
73 BOND_TEAL = "#35978f"
74 HIGHLIGHT_AMBER = "#f4a582"
75
76 CRASH_DATE = datetime.date(1637, 2, 5)
77 DINNER_PARTY_DATE = datetime.date(1636, 11, 25)
78
79 # -----# Load data
80 # -----
81 voc = pd.read_parquet("data/voc_returns.parquet")
82 tulip = pd.read_parquet("data/tulip_index.parquet")
83
84 voc["date"] = voc["date_ord"].apply(datetime.date.fromordinal)
85 tulip["date"] = tulip["date_ord"].apply(datetime.date.fromordinal)
86
87 voc_priced = voc[voc["price"].notna()].sort_values("date").copy()
88 voc_div = voc[voc["dividend_raw"].notna()].copy()
89 tulip_data = tulip.copy()
90
91 tulip_pre = tulip[tulip["date"] <= CRASH_DATE].copy()
92 tulip_post = tulip[tulip["date"] > CRASH_DATE].copy()
93
94 # CAPM results
95 capm = json.load(open("data/capm_results.json"))
96
97 # -----# Helpers
98 # -----
99
100
101 def ord_to_x(dates):
102     return [d.toordinal() for d in dates]
103
104
105 def make_date_ticks(start, end, freq="yearly"):
106     ticks, labels = [], []
107     d = start.replace(month=1, day=1)
108     step_months = {"yearly": 12, "quarterly": 3, "monthly": 1}[freq]
109     while d <= end:
110         if start <= d <= end:
111             ticks.append(d.toordinal())
112             if freq == "yearly":
113                 labels.append(str(d.year))
114             elif freq == "quarterly":
115                 labels.append(f"{d.year}-Q{(d.month - 1) // 3 + 1}")
116             else:
117                 labels.append(f"{d.year}-{d.month:02d}")
118         m = d.month + step_months
119         y = d.year + (m - 1) // 12
120         m = (m - 1) % 12 + 1
121         d = d.replace(year=y, month=m, day=1)
122     return ticks, labels
123
124

```

```

125 def pct_formatter(x, _):
126     return f"{x:.0%}"
127
128
129 def pct_formatter_1(x, _):
130     return f"{x:.1%}"
131
132
133 crash_ord = CRASH_DATE.toordinal()
134 dinner_ord = DINNER_PARTY_DATE.toordinal()
135
136
137 # -----# Figure 1: VOC share
138 # -----
139 print("Generating fig_voc_price.png ...")
140
141 fig, ax = plt.subplots(figsize=(8, 4.2))
142
143 xs = ord_to_x(voc_priced["date"])
144 ax.plot(
145     xs,
146     voc_priced["price"],
147     color=VOC_BLUE,
148     linewidth=1.2,
149     linestyle="--",
150     alpha=0.45,
151     zorder=1,
152 )
153 ax.scatter(
154     xs,
155     voc_priced["price"],
156     color=VOC_BLUE,
157     s=40,
158     zorder=3,
159     edgecolors="white",
160     linewidths=0.5,
161     label="VOC share price (% of par)",
162 )
163
164 # Dividend vertical bands
165 div_xs = ord_to_x(voc_div["date"])
166 for x in div_xs:
167     ax.axvline(x, color=VOC_BLUE, alpha=0.12, linewidth=6, zorder=0)
168
169 div_patch = mpatches.Patch(color=VOC_BLUE, alpha=0.15, label="
170     Dividend recorded")
171
172 # Mania window shading
173 ax.axvspan(
174     datetime.date(1636, 11, 1).toordinal(),
175     crash_ord,
176     alpha=0.06,
177     color=TULIP_RED,
178     zorder=0,
179 )
180 ax.text(
181     datetime.date(1636, 12, 1).toordinal(),

```

```

181     ax.get_ylim()[1] * 0.95 if ax.get_ylim()[1] > 350 else 365,
182     "Mania\nwindow",
183     ha="center",
184     va="top",
185     fontsize=8,
186     color=TULIP_RED,
187     style="italic",
188 )
189
190 start_d = datetime.date(1632, 1, 1)
191 end_d = datetime.date(1638, 12, 1)
192 ticks, labels = make_date_ticks(start_d, end_d, "yearly")
193 ax.set_xticks(ticks)
194 ax.set_xticklabels(labels)
195 ax.set_xlim(start_d.toordinal() - 30, end_d.toordinal() + 30)
196
197 ax.set_ylabel("Share price (% of 1602 par value)")
198 ax.set_xlabel("Year")
199 ax.set_title(
200     "VOC Share Price, 1632\u20131638",
201     pad=10,
202 )
203 ax.text(
204     0.5,
205     1.01,
206     r"$n=16$ observations; dashed line connects sparse observations"
207     ,
208     transform=ax.transAxes,
209     ha="center",
210     fontsize=9,
211     color=GREY,
212     style="italic",
213 )
214 ax.legend(
215     handles=[ax.collections[0], div_patch],
216     fontsize=8,
217     loc="upper left",
218 )
219 ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x
220     :.0f}%"))
221
222 plt.tight_layout()
223 plt.savefig("images/fig_voc_price.png")
224 plt.close()
225 print("    Saved.")
226
227 # -----# Figure 2: Tulip
228     composite index
229 # -----
230 print("Generating fig_tulip_index.png ...")
231
232 fig, ax = plt.subplots(figsize=(8, 4.2))
233
234 # Pre-crash
235 xs_pre = ord_to_x(tulip_pre["date"])
236 ax.plot(
237     xs_pre,

```

```

236     tulip_pre["index_excl"],
237     color=TULIP_RED,
238     linewidth=1.6,
239     zorder=2,
240     label="Tulip index (pre-crash)",
241 )
242 ax.scatter(
243     xs_pre,
244     tulip_pre["index_excl"],
245     color=TULIP_RED,
246     s=25,
247     zorder=3,
248     edgecolors="white",
249     linewidths=0.4,
250 )
251
252 # Post-crash
253 if len(tulip_post):
254     xs_post = ord_to_x(tulip_post["date"])
255     ax.plot(
256         xs_post,
257         tulip_post["index_excl"],
258         color=TULIP_RED,
259         linewidth=1.6,
260         linestyle=":",
261         zorder=2,
262         label="Tulip index (post-crash)",
263     )
264     ax.scatter(
265         xs_post,
266         tulip_post["index_excl"],
267         color=TULIP_RED,
268         s=25,
269         zorder=3,
270         marker="D",
271         edgecolors="white",
272         linewidths=0.4,
273     )
274
275 # Dinner-party date
276 ax.axvline(dinner_ord, color=DINNER_GOLD, linewidth=1.4, linestyle="
    -.", zorder=1)
277 ax.text(
278     dinner_ord - 2,
279     175,
280     "Dinner party\n25 Nov 1636",
281     ha="right",
282     va="top",
283     fontsize=8,
284     color=DINNER_GOLD,
285     fontweight="bold",
286 )
287
288 # Crash date
289 ax.axvline(crash_ord, color=GREY, linewidth=1.2, linestyle="--",
    zorder=1)
290 ax.text(
291     crash_ord + 1,

```

```

292     175,
293     "Alkmaar auction\n5 Feb 1637",
294     ha="left",
295     va="top",
296     fontsize=8,
297     color=GREY,
298     style="italic",
299 )
300
301 # Index = 100 baseline
302 ax.axhline(100, color=LIGHT_GREY, linewidth=0.7, alpha=0.5, zorder
303           =0)
304 ax.text(
305     tulip["date"].iloc[0].toordinal(),
306     102,
307     "Index = 100 (peak)",
308     fontsize=7,
309     color=LIGHT_GREY,
310     va="bottom",
311 )
312
313 start_d = datetime.date(1634, 10, 1)
314 end_d = datetime.date(1637, 3, 1)
315 ticks, labels = make_date_ticks(start_d, end_d, "quarterly")
316 ax.set_xticks(ticks)
317 ax.set_xticklabels(labels, rotation=30, ha="right")
318 ax.set_xlim(start_d.toordinal() - 5, end_d.toordinal() + 5)
319
320 ax.set_ylabel("Composite index (100 = 5 Feb 1637 peak)")
321 ax.set_xlabel("Date")
322 ax.set_title(
323     "Tulip Composite Price Index, Dec 1634 \u2013 Feb 1637",
324     pad=10,
325 )
326 ax.text(
327     0.5,
328     1.01,
329     r"$n=32$ observations; index$_{\mathrm{excl}}$ (arbitrarily-
330     dated obs excluded)",
331     transform=ax.transAxes,
332     ha="center",
333     fontsize=9,
334     color=GREY,
335     style="italic",
336 )
337 ax.legend(fontsize=8, loc="upper left")
338
339 plt.tight_layout()
340 plt.savefig("images/fig_tulip_index.png")
341 plt.close()
342 print("  Saved.")
343
344 # -----# Figure 3: Overlaid
345 # normalised series
346 # -----
347 print("Generating fig_overlay.png ...")

```

```

347 BASE_DATE = datetime.date(1634, 12, 1)
348
349 voc_sorted = voc_priced.sort_values("date").copy()
350 voc_base_rows = voc_sorted[voc_sorted["date"] <= BASE_DATE]
351 voc_base = voc_base_rows["price"].iloc[-1] if len(voc_base_rows)
    else np.nan
352
353 tulip_base_vals = tulip_data[tulip_data["date"] == BASE_DATE]["
    index_excl"].values
354 tulip_base = tulip_base_vals[0] if len(tulip_base_vals) else np.nan
355
356 fig, ax = plt.subplots(figsize=(9, 4.8))
357
358 # VOC normalised (step)
359 if not np.isnan(voc_base):
360     voc_norm_x = ord_to_x(voc_priced["date"])
361     voc_norm_y = (voc_priced["price"] / voc_base) * 100
362     ax.step(
363         voc_norm_x,
364         voc_norm_y,
365         where="post",
366         color=VOC_BLUE,
367         linewidth=1.5,
368         alpha=0.7,
369         zorder=2,
370     )
371     ax.scatter(
372         voc_norm_x,
373         voc_norm_y,
374         color=VOC_BLUE,
375         s=35,
376         zorder=3,
377         edgecolors="white",
378         linewidths=0.4,
379         label=f"VOC (rebased; {voc_base:.0f}% of par)",
380     )
381
382 # Tulip normalised
383 if not np.isnan(tulip_base):
384     tulip_norm_x = ord_to_x(tulip_data["date"])
385     tulip_norm_y = (tulip_data["index_excl"] / tulip_base) * 100
386     pre_mask = tulip_data["date"] <= CRASH_DATE
387     post_mask = tulip_data["date"] > CRASH_DATE
388     if pre_mask.any():
389         ax.plot(
390             [x for x, m in zip(tulip_norm_x, pre_mask) if m],
391             [y for y, m in zip(tulip_norm_y, pre_mask) if m],
392             color=TULIP_RED,
393             linewidth=1.6,
394             zorder=2,
395             label=f"Tulip index (rebased; index={tulip_base:.1f})",
396         )
397     ax.scatter(
398         [x for x, m in zip(tulip_norm_x, pre_mask) if m],
399         [y for y, m in zip(tulip_norm_y, pre_mask) if m],
400         color=TULIP_RED,
401         s=18,
402         zorder=3,

```



```

403         edgecolors="white",
404         linewidths=0.3,
405     )
406     if post_mask.any():
407         ax.plot(
408             [x for x, m in zip(tulip_norm_x, post_mask) if m],
409             [y for y, m in zip(tulip_norm_y, post_mask) if m],
410             color=TULIP_RED,
411             linewidth=1.6,
412             linestyle=":",
413             zorder=2,
414         )
415         ax.scatter(
416             [x for x, m in zip(tulip_norm_x, post_mask) if m],
417             [y for y, m in zip(tulip_norm_y, post_mask) if m],
418             color=TULIP_RED,
419             s=18,
420             zorder=3,
421             marker="D",
422             edgecolors="white",
423             linewidths=0.3,
424         )
425
426     # Posthumus commodity index
427     _posthumus_path = "data/posthumus_monthly.parquet"
428     if os.path.exists(_posthumus_path):
429         comm = pd.read_parquet(_posthumus_path)
430         comm["date"] = comm.apply(
431             lambda r: datetime.date(int(r["year"]), int(r["month"]), 1),
432             axis=1
433         )
434         comm_base_rows = comm[(comm["year"] == 1634) & (comm["month"] == 12)]
435         if len(comm_base_rows):
436             comm_base_val = comm_base_rows["broad_index"].values[0]
437             if not np.isnan(comm_base_val):
438                 comm_norm_x = ord_to_x(comm["date"])
439                 comm_norm_y = (comm["broad_index"] / comm_base_val) * 100
440                 ax.plot(
441                     comm_norm_x,
442                     comm_norm_y,
443                     color=COMMODITY_GREEN,
444                     linewidth=1.3,
445                     alpha=0.8,
446                     zorder=2,
447                     label="Commodity index (Posthumus broad)",
448                 )
449                 ax.scatter(
450                     comm_norm_x,
451                     comm_norm_y,
452                     color=COMMODITY_GREEN,
453                     s=14,
454                     zorder=3,
455                     marker="s",
456                     edgecolors="white",
457                     linewidths=0.3,

```

```

458
459 ax.axhline(100, color=LIGHT_GREY, linewidth=0.7, alpha=0.4, zorder
    =0)
460
461 # Dinner-party and crash annotations
462 ax.axvline(dinner_ord, color=DINNER_GOLD, linewidth=1.4, linestyle="
    -.", zorder=1)
463 ylim_top = ax.get_ylim()[1]
464 ax.text(
465     dinner_ord - 2,
466     ylim_top * 0.92,
467     "Dinner party\n25 Nov 1636",
468     ha="right",
469     va="top",
470     fontsize=8,
471     color=DINNER_GOLD,
472     fontweight="bold",
473 )
474
475 ax.axvline(crash_ord, color=GREY, linewidth=1.2, linestyle="--",
    zorder=1)
476 ax.text(
477     crash_ord + 2,
478     ylim_top * 0.95,
479     "Alkmaar\n5 Feb 1637",
480     ha="left",
481     va="top",
482     fontsize=8,
483     color=GREY,
484     style="italic",
485 )
486
487 start_d = datetime.date(1632, 1, 1)
488 end_d = datetime.date(1638, 12, 1)
489 ticks, labels = make_date_ticks(start_d, end_d, "yearly")
490 ax.set_xticks(ticks)
491 ax.set_xticklabels(labels)
492 ax.set_xlim(start_d.toordinal() - 30, end_d.toordinal() + 30)
493
494 ax.set_ylabel("Index level (100 = December 1634)")
495 ax.set_xlabel("Year")
496 ax.set_title(
497     "VOC, Tulip, and Commodity Price Series, Normalised to December
        1634",
498     pad=10,
499 )
500 ax.text(
501     0.5,
502     1.01,
503     "VOC step-interpolated; tulip and commodity at observed dates",
504     transform=ax.transAxes,
505     ha="center",
506     fontsize=9,
507     color=GREY,
508     style="italic",
509 )
510 ax.legend(fontsize=8, loc="upper left")
511

```

```

512 plt.tight_layout()
513 plt.savefig("images/fig_overlay.png")
514 plt.close()
515 print("    Saved.")
516
517
518 # -----# Figure 4: Beta scatter
519 # ' matched return pairs
520 # -----
521 print("Generating fig_beta_scatter.png ...")
522
523 pairs = []
524 for _, row in tulip_data.iterrows():
525     if pd.isna(row["simple_return"]):
526         continue
527     t_date = row["date"]
528     t_ret = row["simple_return"]
529     t_days = row["days_elapsed"]
530     if pd.isna(t_days) or t_days <= 0:
531         continue
532
533     prior_t_date = datetime.date.fromordinal(t_date.toordinal() -
534                                             int(t_days))
535
536     voc_at_end = voc_sorted[voc_sorted["date"] <= t_date]
537     voc_at_start = voc_sorted[voc_sorted["date"] <= prior_t_date]
538
539     if voc_at_end.empty or voc_at_start.empty:
540         continue
541
542     p_end = voc_at_end.iloc[-1]["price"]
543     p_start = voc_at_start.iloc[-1]["price"]
544
545     d_in_window = voc[
546         (voc["date"] > prior_t_date)
547         & (voc["date"] <= t_date)
548         & (voc["dividend_raw"].notna())
549     ]["dividend"].sum()
550
551     voc_ret = (p_end - p_start + d_in_window) / p_start
552
553     pairs.append(
554         {
555             "date": t_date,
556             "tulip_r": t_ret,
557             "voc_r": voc_ret,
558             "days": t_days,
559         }
560     )
561
562 pairs_df = pd.DataFrame(pairs).dropna()
563 pairs_df_nonzero = pairs_df[pairs_df["voc_r"].abs() > 1e-9].copy()
564 pairs_plot = pairs_df # Plot ALL pairs (informative + zero-VOC-
565                       return)
566
567 fig, ax = plt.subplots(figsize=(6.5, 5.2))
568
569 if len(pairs_df_nonzero) >= 2:

```

```

567 x = pairs_df_nonzero["voc_r"].values
568 y = pairs_df_nonzero["tulip_r"].values
569
570 # OLS (fitted on informative pairs only)
571 A = np.vstack([np.ones_like(x), x]).T
572 coeffs, *_ = np.linalg.lstsq(A, y, rcond=None)
573 alpha_hat, beta_hat = coeffs
574
575 # Bootstrap CI
576 rng = np.random.default_rng(42)
577 boot_betas = []
578 for _ in range(5000):
579     idx = rng.integers(0, len(x), len(x))
580     xb, yb = x[idx], y[idx]
581     Ab = np.vstack([np.ones_like(xb), xb]).T
582     try:
583         cb, *_ = np.linalg.lstsq(Ab, yb, rcond=None)
584         boot_betas.append(cb[1])
585     except Exception:
586         pass
587 boot_betas = np.array(boot_betas)
588 beta_lo, beta_hi = np.percentile(boot_betas, [2.5, 97.5])
589
590 # Regression line and CI band
591 x_line = np.linspace(x.min() - 0.05, x.max() + 0.05, 200)
592 y_line = alpha_hat + beta_hat * x_line
593
594 rng2 = np.random.default_rng(43)
595 boot_lines = []
596 for _ in range(2000):
597     idx = rng2.integers(0, len(x), len(x))
598     xb, yb = x[idx], y[idx]
599     Ab = np.vstack([np.ones_like(xb), xb]).T
600     try:
601         cb, *_ = np.linalg.lstsq(Ab, yb, rcond=None)
602         boot_lines.append(cb[0] + cb[1] * x_line)
603     except Exception:
604         pass
605 if boot_lines:
606     boot_lines = np.array(boot_lines)
607     y_lo = np.percentile(boot_lines, 2.5, axis=0)
608     y_hi = np.percentile(boot_lines, 97.5, axis=0)
609     ax.fill_between(
610         x_line,
611         y_lo,
612         y_hi,
613         alpha=0.12,
614         color=VOC_BLUE,
615         label="95% bootstrap CI",
616     )
617
618 ax.plot(
619     x_line,
620     y_line,
621     color=GREY,
622     linewidth=1.5,
623     zorder=2,

```

```

624         label=f"OLS:  $\hat{\beta}=\{\text{beta\_hat:.2f}\}$  [ $\{\text{beta\_lo:.2f}$ 
        },  $\{\text{beta\_hi:.2f}\}$ ]",
625     )
626
627     # Data points with era colouring
628     for _, row in pairs_plot.iterrows():
629         if row["date"] > CRASH_DATE:
630             c, marker, ms = LIGHT_GREY, "D", 55
631         elif row["date"] >= DINNER_PARTY_DATE:
632             c, marker, ms = DINNER_GOLD, "*", 100
633         else:
634             c, marker, ms = TULIP_RED, "o", 60
635         ax.scatter(
636             row["voc_r"],
637             row["tulip_r"],
638             color=c,
639             s=ms,
640             zorder=4,
641             edgecolors="white",
642             linewidths=0.6,
643             marker=marker,
644         )
645         ax.annotate(
646             row["date"].strftime("%d/%m/%y"),
647             xy=(row["voc_r"], row["tulip_r"]),
648             xytext=(5, 5),
649             textcoords="offset points",
650             fontsize=6.5,
651             color=GREY,
652             fontstyle="italic",
653         )
654
655     pre_patch = mpatches.Patch(color=TULIP_RED, label="Pre-dinner-
        party")
656     dinner_patch = mpatches.Patch(
657         color=DINNER_GOLD, label="Dinner-party era (\u2265
        25/11/1636)"
658     )
659     post_patch = mpatches.Patch(color=LIGHT_GREY, label="Post-crash"
        )
660
661     ax.axhline(0, color=LIGHT_GREY, linewidth=0.5, alpha=0.5)
662     ax.axvline(0, color=LIGHT_GREY, linewidth=0.5, alpha=0.5)
663
664     ax.set_xlabel("VOC return over matched interval")
665     ax.set_ylabel("Tulip index return over matched interval")
666     ax.set_title(
667         "Tulip vs VOC Returns: Beta Estimation",
668         pad=10,
669     )
670     ax.text(
671         0.5,
672         1.01,
673         f"$n=\{\text{len}(\text{pairs\_df\_nonzero})\}$ informative pairs; "
674         f"OLS  $\hat{\beta}=\{\text{beta\_hat:.2f}\}$ , "
675         f"adopted  $\beta=3.5$ ",
676         transform=ax.transAxes,
677         ha="center",

```

```

678         fontsize=9,
679         color=GREY,
680         style="italic",
681     )
682
683     handles = [pre_patch, dinner_patch, post_patch]
684     if boot_lines is not None and len(boot_lines) > 0:
685         handles = [ax.collections[0], ax.lines[0]] + handles
686     else:
687         handles = [ax.lines[0]] + handles
688     ax.legend(handles=handles, fontsize=7.5, loc="lower right")
689
690     ax.xaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter
691     ))
692     ax.yaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter
693     ))
694
695 plt.tight_layout()
696 plt.savefig("images/fig_beta_scatter.png")
697 plt.close()
698 print("    Saved.")
699
700 # -----# Figure 5: Volatility
701 # comparison ‘‘ FIXED with log scale
702 # -----
703 print("Generating fig_volatility.png ...")
704
705 fig, axes = plt.subplots(
706     1, 2, figsize=(11, 4.8), gridspec_kw={"width_ratios": [1.5, 1]}
707 )
708
709 # --- Left panel: annualised volatility bars (LOG SCALE) ---
710 ax = axes[0]
711
712 SCALE_DAYS = 30
713
714 tulip_ret_df = tulip_data[
715     tulip_data["simple_return"].notna() & (tulip_data["days_elapsed"]
716     ] > 0)
717 ].copy()
718 tulip_ret_df["scaled_return"] = tulip_ret_df["simple_return"] * (
719     SCALE_DAYS / tulip_ret_df["days_elapsed"]
720 )
721 tulip_vol_ann = tulip_ret_df["scaled_return"].std() * (12**0.5)
722
723 tulip_pre_ret = tulip_ret_df[tulip_ret_df["date"] <= CRASH_DATE].
724     copy()
725 tulip_pre_vol = (
726     tulip_pre_ret["scaled_return"].std() * (12**0.5)
727     if len(tulip_pre_ret) > 1
728     else np.nan
729 )
730
731 tulip_prespike = tulip_ret_df[
732     (tulip_ret_df["date"] >= datetime.date(1636, 5, 1))
733     & (tulip_ret_df["date"] <= datetime.date(1636, 11, 12))
734     & (tulip_ret_df["days_elapsed"] > 1)

```

```

731 ].copy()
732 if len(tulip_prespike) > 1:
733     tulip_prespike["log_ret"] = np.log(1 + tulip_prespike["
        simple_return"])
734     _ppyr = 365 / tulip_prespike["days_elapsed"].mean()
735     tulip_prespike_vol = tulip_prespike["log_ret"].std() * np.sqrt(
        _ppyr)
736 else:
737     tulip_prespike_vol = np.nan
738
739 voc_ret_df = voc[voc["simple_total_return"].notna()].copy()
740 voc_vol_ann = voc_ret_df["simple_total_return"].std() * (12**0.5)
741
742 categories = [
743     "VOC\n(monthly,\n$n$=3)",
744     "Tulip\nfull series\n($n$=31)",
745     "Tulip\npre-crash\n($n$=28)",
746     "Tulip\npre-spike\n(MVO input,\n$n$=4)",
747 ]
748 values = [voc_vol_ann, tulip_vol_ann, tulip_pre_vol,
        tulip_prespike_vol]
749 colors = [VOC_BLUE, TULIP_RED, HIGHLIGHT_AMBER, DINNER_GOLD]
750
751 bars = ax.bar(
752     range(len(categories)),
753     values,
754     color=colors,
755     edgecolor="white",
756     linewidth=0.8,
757     zorder=2,
758     width=0.65,
759 )
760
761 # Value labels on bars
762 for bar, v in zip(bars, values):
763     if not np.isnan(v):
764         ax.text(
765             bar.get_x() + bar.get_width() / 2,
766             v * 1.15,
767             f"{v:.0%}",
768             ha="center",
769             va="bottom",
770             fontsize=9,
771             fontweight="bold",
772         )
773
774 ax.set_yscale("log")
775 ax.set_xticks(range(len(categories)))
776 ax.set_xticklabels(categories, fontsize=8.5)
777 ax.set_ylabel("Annualised volatility (log scale)")
778 ax.set_title("Annualised Volatility Comparison", pad=8)
779
780 # Format y-axis ticks as percentages
781 ax.yaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter))
782 ax.set_ylim(0.05, max(v for v in values if not np.isnan(v)) * 3)
783
784 # --- Right panel: summary statistics table ---
785 ax2 = axes[1]

```

```

786 ax2.axis("off")
787
788 if len(pairs_df) >= 2:
789     corr = np.corrcoef(pairs_df["voc_r"], pairs_df["tulip_r"])[0, 1]
790     cov_val = np.cov(pairs_df["voc_r"], pairs_df["tulip_r"])[0, 1]
791     beta_cov = (
792         cov_val / (pairs_df["voc_r"].var()) if pairs_df["voc_r"].var
793         () > 0 else np.nan
794     )
795     n_pairs_tab = len(pairs_df)
796 else:
797     corr = cov_val = beta_cov = np.nan
798     n_pairs_tab = 0
799
800 rows = [
801     ["", "VOC", "Tulip"],
802     ["$n$ returns", str(len(voc_ret_df)), str(len(tulip_ret_df))],
803     [
804         "Ann. vol.",
805         f"{voc_vol_ann:.0%}" if not np.isnan(voc_vol_ann) else "\
806         u2013",
807         f"{tulip_vol_ann:.0%}" if not np.isnan(tulip_vol_ann) else "\
808         u2013",
809     ],
810     ["Matched pairs", f"{n_pairs_tab}", ""],
811     ["Correlation  $\rho$ ", f"{corr:.3f}" if not np.isnan(corr) else "\
812     u2013", ""],
813     ["Covariance", f"{cov_val:.4f}" if not np.isnan(cov_val) else "\
814     u2013", ""],
815     [
816         "$\beta$ (cov/var)",
817         f"{beta_cov:.2f}" if not np.isnan(beta_cov) else "\u2013",
818         "",
819     ],
820     ["", "", ""],
821     ["Note:", "$n$ very small;", ""],
822     ["", "see \u00a775.3 for", ""],
823     ["", "sensitivity range", ""],
824 ]
825
826 table = ax2.table(
827     cellText=[[r[0], r[1], r[2]] for r in rows],
828     loc="center",
829     cellLoc="left",
830 )
831 table.auto_set_font_size(False)
832 table.set_fontsize(8.5)
833 table.scale(1, 1.4)
834
835 for j in range(3):
836     table[0, j].set_facecolor("#d0d8e8")
837     table[0, j].set_text_props(fontweight="bold")
838
839 # Alternate row shading
840 for i in range(1, len(rows)):
841     for j in range(3):
842         if i % 2 == 0:
843             table[i, j].set_facecolor("#f7f7f7")

```



```

839
840 ax2.set_title("Summary Statistics", pad=12, fontweight="bold")
841
842 plt.tight_layout()
843 plt.savefig("images/fig_volatility.png")
844 plt.close()
845 print("  Saved.")
846
847
848 # -----# Figure 6: Security
      Market Line (NEW ' canonical CAPM chart)
849 # -----
850 print("Generating fig_sml.png ...")
851
852 dp = capm["dinner_party"]
853 rf = capm["parameters"]["rf_annual"]
854 mrp = capm["mrp"]["mrp_annual"]
855
856 fig, ax = plt.subplots(figsize=(7, 5))
857
858 # SML line:  $E(R) = R_f + \beta \cdot MRP$ 
859 beta_range = np.linspace(-0.5, 5.0, 200)
860 sml_returns = rf + beta_range * mrp
861
862 ax.plot(
863     beta_range,
864     sml_returns,
865     color=GREY,
866     linewidth=2,
867     label="Security Market Line",
868     zorder=2,
869 )
870
871 # Shade the region below the SML (underperformance zone)
872 ax.fill_between(
873     beta_range,
874     sml_returns,
875     -0.5,
876     alpha=0.04,
877     color=TULIP_RED,
878     zorder=0,
879 )
880
881 # VOC (beta=1 by definition, the market)
882 ax.scatter(
883     [1.0],
884     [rf + 1.0 * mrp],
885     color=VOC_BLUE,
886     s=100,
887     zorder=4,
888     edgecolors="white",
889     linewidths=0.8,
890     label=f"VOC (market,  $\beta=1$ )",
891 )
892 ax.annotate(
893     f"VOC\n $E(R) = \{rf + mrp : .1\% \}$ ",
894     xy=(1.0, rf + mrp),
895     xytext=(1.3, rf + mrp + 0.08),

```

```

896     fontsize=8,
897     color=VOC_BLUE,
898     fontweight="bold",
899     arrowprops=dict(arrowstyle="->", color=VOC_BLUE, lw=0.8),
900 )
901
902 # Tulip at adopted beta = 3.5: CAPM-implied position
903 tulip_capm_ret = rf + dp["beta"] * mrp
904 ax.scatter(
905     [dp["beta"]],
906     [tulip_capm_ret],
907     color=TULIP_RED,
908     s=100,
909     zorder=4,
910     marker="^",
911     edgecolors="white",
912     linewidths=0.8,
913     label=f"Tulip CAPM-implied ($\\beta$={dp['beta']} )",
914 )
915 ax.annotate(
916     f"CAPM implied\\nE(R)={tulip_capm_ret:.1%}",
917     xy=(dp["beta"], tulip_capm_ret),
918     xytext=(dp["beta"] + 0.4, tulip_capm_ret + 0.08),
919     fontsize=8,
920     color=TULIP_RED,
921     fontweight="bold",
922     arrowprops=dict(arrowstyle="->", color=TULIP_RED, lw=0.8),
923 )
924
925 # Tulip realised return (shows alpha gap)
926 tulip_realised = dp["r_realised_ann"]
927 ax.scatter(
928     [dp["beta"]],
929     [tulip_realised],
930     color=DINNER_GOLD,
931     s=100,
932     zorder=4,
933     marker="v",
934     edgecolors="white",
935     linewidths=0.8,
936     label=f"Tulip realised ($R$={tulip_realised:.1%})",
937 )
938 ax.annotate(
939     f"Realised\\nR={tulip_realised:.1%}",
940     xy=(dp["beta"], tulip_realised),
941     xytext=(dp["beta"] + 0.4, tulip_realised - 0.12),
942     fontsize=8,
943     color=DINNER_GOLD,
944     fontweight="bold",
945     arrowprops=dict(arrowstyle="->", color=DINNER_GOLD, lw=0.8),
946 )
947
948 # Alpha arrow between CAPM-implied and realised
949 ax.annotate(
950     "",
951     xy=(dp["beta"] - 0.08, tulip_realised),
952     xytext=(dp["beta"] - 0.08, tulip_capm_ret),
953     arrowprops=dict(arrowstyle="<->", color=TULIP_RED, lw=1.5),

```

```

954 )
955 alpha_val = tulip_realised - tulip_capm_ret
956 ax.text(
957     dp["beta"] - 0.22,
958     (tulip_realised + tulip_capm_ret) / 2,
959     "$\\alpha$=" + f"{alpha_val:.1%}",
960     fontsize=9,
961     color=TULIP_RED,
962     fontweight="bold",
963     ha="right",
964     va="center",
965     bbox=dict(boxstyle="round,pad=0.2", fc="white", ec=TULIP_RED,
966               alpha=0.8),
967 )
968 # Risk-free rate
969 ax.scatter(
970     [0],
971     [rf],
972     color=BOND_TEAL,
973     s=80,
974     zorder=4,
975     marker="s",
976     edgecolors="white",
977     linewidths=0.8,
978     label=f"Lekdijk bond ($R_f$={rf:.0%})",
979 )
980
981 ax.set_xlabel("Beta ($\\beta$)")
982 ax.set_ylabel("Expected / realised return (annual)")
983 ax.set_title(
984     "Security Market Line \u2014 Dinner-Party Date (25 Nov 1636)",
985     pad=10,
986 )
987 ax.text(
988     0.5,
989     1.01,
990     f"$R_f$={rf:.0%}, MRP={mrp:.1%}, adopted $\\beta_{{{tulip}}}$={dp
991         ['beta']}",
992     transform=ax.transAxes,
993     ha="center",
994     fontsize=9,
995     color=GREY,
996     style="italic",
997 )
998 ax.yaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter))
999 ax.set_xlim(-0.5, 5.0)
1000 ax.set_ylim(-0.1, 1.6)
1001 ax.legend(fontsize=8, loc="upper left")
1002
1003 plt.tight_layout()
1004 plt.savefig("images/fig_sml.png")
1005 plt.close()
1006 print(" Saved.")
1007
1008

```

```

1009 # -----# Figure 7: Cumulative
      returns ' growth of 1 guilder (NEW)
1010 # -----
1011 print("Generating fig_cumulative_returns.png ...")
1012
1013 fig, ax = plt.subplots(figsize=(8, 4.5))
1014
1015 # Tulip: rebased to 1.0 at first observation
1016 tulip_sorted = tulip_data.sort_values("date")
1017 tulip_first = tulip_sorted.iloc[0]
1018 tulip_dates = ord_to_x(tulip_sorted["date"])
1019 tulip_growth = tulip_sorted["index_excl"] / tulip_first["index_excl"]
      ]
1020
1021 pre_mask = tulip_sorted["date"] <= CRASH_DATE
1022 post_mask = tulip_sorted["date"] > CRASH_DATE
1023
1024 ax.plot(
1025     [x for x, m in zip(tulip_dates, pre_mask) if m],
1026     [y for y, m in zip(tulip_growth, pre_mask) if m],
1027     color=TULIP_RED,
1028     linewidth=1.6,
1029     label="Tulip bulbs",
1030     zorder=2,
1031 )
1032 ax.scatter(
1033     [x for x, m in zip(tulip_dates, pre_mask) if m],
1034     [y for y, m in zip(tulip_growth, pre_mask) if m],
1035     color=TULIP_RED,
1036     s=15,
1037     zorder=3,
1038     edgecolors="white",
1039     linewidths=0.3,
1040 )
1041 if post_mask.any():
1042     ax.plot(
1043         [x for x, m in zip(tulip_dates, post_mask) if m],
1044         [y for y, m in zip(tulip_growth, post_mask) if m],
1045         color=TULIP_RED,
1046         linewidth=1.6,
1047         linestyle=":",
1048         zorder=2,
1049     )
1050     ax.scatter(
1051         [x for x, m in zip(tulip_dates, post_mask) if m],
1052         [y for y, m in zip(tulip_growth, post_mask) if m],
1053         color=TULIP_RED,
1054         s=15,
1055         zorder=3,
1056         marker="D",
1057         edgecolors="white",
1058         linewidths=0.3,
1059     )
1060
1061 # VOC: rebased to 1.0 at nearest date to tulip first observation
1062 first_tulip_date = tulip_first["date"]
1063 voc_base_row = voc_priced[voc_priced["date"] <= first_tulip_date]
1064 if len(voc_base_row):

```

```

1065     voc_base_price = voc_base_row.iloc[-1]["price"]
1066     voc_xs = ord_to_x(voc_priced["date"])
1067     voc_growth = voc_priced["price"] / voc_base_price
1068     ax.step(
1069         voc_xs,
1070         voc_growth,
1071         where="post",
1072         color=VOC_BLUE,
1073         linewidth=1.5,
1074         alpha=0.8,
1075         label="VOC shares",
1076         zorder=2,
1077     )
1078     ax.scatter(
1079         voc_xs,
1080         voc_growth,
1081         color=VOC_BLUE,
1082         s=25,
1083         zorder=3,
1084         edgecolors="white",
1085         linewidths=0.4,
1086     )
1087
1088     # Risk-free bond: continuous 5% annual growth
1089     bond_start_ord = first_tulip_date.toordinal()
1090     bond_end = datetime.date(1638, 9, 1)
1091     bond_days = np.arange(bond_start_ord, bond_end.toordinal() + 1)
1092     bond_growth = (1 + rf) ** ((bond_days - bond_start_ord) / 365.25)
1093     ax.plot(
1094         bond_days,
1095         bond_growth,
1096         color=BOND_TEAL,
1097         linewidth=1.3,
1098         linestyle="-. ",
1099         alpha=0.8,
1100         label=f"Lekdijk bond ({rf:.0%} p.a.)",
1101         zorder=1,
1102     )
1103
1104     ax.axhline(1.0, color=LIGHT_GREY, linewidth=0.6, alpha=0.4, zorder
1105                =0)
1106
1107     # Annotations
1108     ax.axvline(
1109         dinner_ord, color=DINNER_GOLD, linewidth=1.2, linestyle="-. ",
1110         zorder=1, alpha=0.7
1111     )
1112     ax.axvline(crash_ord, color=GREY, linewidth=1.0, linestyle="--",
1113                zorder=1, alpha=0.7)
1114
1115     start_d = datetime.date(1634, 10, 1)
1116     end_d = datetime.date(1638, 9, 1)
1117     ticks, labels = make_date_ticks(start_d, end_d, "yearly")
1118     ax.set_xticks(ticks)
1119     ax.set_xticklabels(labels)
1120     ax.set_xlim(start_d.toordinal() - 5, end_d.toordinal() + 30)
1121     ax.set_ylabel("Growth of 1 guilder")

```

```

1120 ax.set_xlabel("Year")
1121 ax.set_title(
1122     "Cumulative Returns: Tulips vs VOC vs Risk-Free Bond",
1123     pad=10,
1124 )
1125 ax.text(
1126     0.5,
1127     1.01,
1128     f"Rebased to 1.0 at {first_tulip_date.strftime('%b %Y')}; "
1129     "VOC step-interpolated, bond at {:.0%} continuous".format(rf),
1130     transform=ax.transAxes,
1131     ha="center",
1132     fontsize=9,
1133     color=GREY,
1134     style="italic",
1135 )
1136 ax.legend(fontsize=8, loc="upper left")
1137
1138 plt.tight_layout()
1139 plt.savefig("images/fig_cumulative_returns.png")
1140 plt.close()
1141 print("  Saved.")
1142
1143
1144 # -----# Figure 8: Alpha
1145 # sensitivity bar chart (NEW)
1146 # -----
1147 print("Generating fig_alpha_sensitivity.png ...")
1148
1149 alpha_data = capm["alpha_sensitivity"]
1150 betas = [r["beta"] for r in alpha_data]
1151 alphas_pre = [r["alpha_pre"] for r in alpha_data]
1152 alphas_full = [r["alpha_full"] for r in alpha_data]
1153
1154 # Add the dinner-party beta
1155 dp = capm["dinner_party"]
1156 betas.append(dp["beta"])
1157 alphas_pre.append(dp["alpha"])
1158 # Full-cycle alpha at beta=3.5: compute from the data
1159 r_full = capm["tulip_returns"]["r_annualised_fullcycle"]
1160 e_r_full = rf + dp["beta"] * mrp
1161 alphas_full.append(r_full - e_r_full if r_full is not None else None)
1162
1163
1164 fig, ax = plt.subplots(figsize=(8, 4.5))
1165
1166 x_pos = np.arange(len(betas))
1167 width = 0.35
1168
1169 bars_pre = ax.bar(
1170     x_pos - width / 2,
1171     alphas_pre,
1172     width,
1173     color=DINNER_GOLD,
1174     edgecolor="white",
1175     linewidth=0.5,
1176     label="Pre-crash  $\alpha$  (to 25 Nov 1636)",
1177     zorder=2,

```

```

1176 )
1177 bars_full = ax.bar(
1178     x_pos + width / 2,
1179     [a if a is not None else 0 for a in alphas_full],
1180     width,
1181     color=TULIP_RED,
1182     edgecolor="white",
1183     linewidth=0.5,
1184     label="Full-cycle  $\alpha$  (incl. crash)",
1185     zorder=2,
1186 )
1187
1188 # Value labels
1189 for bar in bars_pre:
1190     h = bar.get_height()
1191     ax.text(
1192         bar.get_x() + bar.get_width() / 2,
1193         h + (0.02 if h >= 0 else -0.04),
1194         f"{h:.0%}",
1195         ha="center",
1196         va="bottom" if h >= 0 else "top",
1197         fontsize=8,
1198         color=DINNER_GOLD,
1199         fontweight="bold",
1200     )
1201 for bar, a in zip(bars_full, alphas_full):
1202     if a is None:
1203         continue
1204     h = bar.get_height()
1205     ax.text(
1206         bar.get_x() + bar.get_width() / 2,
1207         h + (-0.04 if h < 0 else 0.02),
1208         f"{h:.0%}",
1209         ha="center",
1210         va="top" if h < 0 else "bottom",
1211         fontsize=8,
1212         color=TULIP_RED,
1213         fontweight="bold",
1214     )
1215
1216 ax.axhline(0, color=GREY, linewidth=0.8, zorder=1)
1217
1218 # Highlight the adopted beta
1219 adopted_idx = len(betas) - 1
1220 ax.get_xticklabels()
1221 beta_labels = [f" $\beta$ ={b}" for b in betas]
1222 beta_labels[-1] = f" $\beta$ ={betas[-1]}\n(adopted)"
1223
1224 ax.set_xticks(x_pos)
1225 ax.set_xticklabels(beta_labels, fontsize=9)
1226
1227 ax.set_ylabel("Alpha ( $\alpha$ )")
1228 ax.yaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter))
1229 ax.set_title(
1230     "CAPM Alpha Across Assumed Beta Values",
1231     pad=10,
1232 )
1233 ax.text(

```

```

1234     0.5,
1235     1.01,
1236     f"$R_f$={rf:.0%}, MRP={mrp:.1%}; "
1237     "positive  $\alpha$  = asset outperformed CAPM expectation",
1238     transform=ax.transAxes,
1239     ha="center",
1240     fontsize=9,
1241     color=GREY,
1242     style="italic",
1243 )
1244 ax.legend(fontsize=8, loc="upper right")
1245
1246 # Shade adopted beta column
1247 ax.axvspan(
1248     adopted_idx - 0.5,
1249     adopted_idx + 0.5,
1250     alpha=0.08,
1251     color=DINNER_GOLD,
1252     zorder=0,
1253 )
1254
1255 plt.tight_layout()
1256 plt.savefig("images/fig_alpha_sensitivity.png")
1257 plt.close()
1258 print("    Saved.")
1259
1260
1261 # -----# Summary
1262 # -----
1263 print("\nAll figures written to images/:")
1264 for f in [
1265     "fig_voc_price.png",
1266     "fig_tulip_index.png",
1267     "fig_overlay.png",
1268     "fig_beta_scatter.png",
1269     "fig_volatility.png",
1270     "fig_sml.png",
1271     "fig_cumulative_returns.png",
1272     "fig_alpha_sensitivity.png",
1273 ]:
1274     path = f"images/{f}"
1275     if os.path.exists(path):
1276         kb = os.path.getsize(path) / 1024
1277         print(f"    {f}    ({kb:.0f} KB)")

```

E.0.7 CAPM/MVO Analysis Script

```

1 """
2 appendix_i_capm_analysis.py
3 -----
4 Executes the full CAPM and mean-variance optimisation pipeline
5   described
6   in Section 5 of the paper, using the pre-crash dataset (up to Jan
7   1637).
8
9 Produces the numbers that populate '6 (Results) and '7 (Conclusion
10 ):

```



```

8   - Market Risk Premium (MRP) from VOC total returns
9   - OLS beta estimate for tulips vs market
10  - CAPM alpha for tulips (pre-crash and full-cycle)
11  - Sensitivity table: alpha across beta = 0.5, 1.0, 1.5, 2.0
12  - Tangency portfolio weights (tulips, VOC, bond)
13  - Sensitivity table: weights across assumed betas
14
15 Writes:
16   data/capm_results.json      -- machine-readable results for
    downstream use
17   data/capm_results.txt      -- human-readable summary
18
19 Run with:
20   nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
21     pkgs.python313.withPackages (ps: with ps; [numpy pandas
22       pyarrow scipy]))' \
23     -c python appendix_i_capm_analysis.py
24
25 Design notes matching the paper:
26
27 CAPM framework ( '5 .1):
28    $E(R_i) = R_f + \beta_i * (E(R_m) - R_f)$ 
29    $\alpha_i = R_{realised\_i} - E(R_i)$ 
30
31 Risk-free rate ( '4 .4):
32    $R_f = 5\%$  per annum = 0.05/12 per month
33   Source: Hoogheemraadschap Lekdijk Bovendams perpetual bond,
    1624.
34
35 Market portfolio ( '4 .3):
36   VOC only. GWC is acknowledged but data too sparse for reliable
37   inclusion; see '4 .3 placeholder for the value-weighting
38   approach
39   if GWC data becomes available.
40
41 Return interval alignment:
42   Both series are irregularly spaced. We use interval-matched
43   returns:
44   for each tulip observation, we compute the contemporaneous VOC
45   return
46   over the same interval using carry-forward pricing (last known
47   price).
48   Only intervals where the VOC price actually changed are used for
49   OLS.
50   For the MRP, we use all VOC computable returns directly.
51
52 Single-factor covariance matrix ( '5 .2):
53    $\sigma_{ij} = \beta_i * \beta_j * \sigma_m^2 + \delta_{ij} * \sigma_{\epsilon_i}^2$ 
54   Bond:  $\beta=0$ ,  $\sigma_{\epsilon}=0$  (risk-free by assumption).
55    $\sigma_m^2$  estimated from VOC returns (annualised, then converted
    back
    to per-interval units consistent with the tulip return series).
56
57 Tangency portfolio ( '5 .2):
58    $w * K^{-1} (\mu - R_f * 1)$ 
59   Normalised to sum to 1 across risky assets.

```

```

55     Bond weight = 1 - sum(risky weights), floored at 0 (no short
56     bond).
57
58     Pre-crash window:
59     All data up to and including 1637-01-31.
60     February crash is out-of-sample throughout.
61     """
62
63     import datetime
64     import json
65     import os
66
67     import numpy as np
68     import pandas as pd
69     import scipy.stats as stats
70
71     # -----# 0. Parameters (from '4
72     .4 and '5 )
73     # -----
74
75     RF_ANNUAL = 0.05 # Risk-free rate, annual ('4 .4)
76     RF_MONTHLY = RF_ANNUAL / 12
77
78     PRECRASH_END = datetime.date(1637, 1, 31) # Pre-crash cutoff
79     CRASH_DATE = datetime.date(1637, 2, 5) # Alkmaar auction
80
81     BETA_SENSITIVITY = [0.5, 1.0, 1.5, 2.0] # '5 .3 sensitivity range
82
83     # Dinner-party framing ('7 headline): an investor on 25 Nov 1636
84     # has just
85     # heard tulips doubled. They sit down with all data available up to
86     # that
87     # date. The crash is 10 weeks away.
88     DINNER_PARTY_DATE = datetime.date(1636, 11, 25)
89     DINNER_PARTY_BETA = 3.5 # OLS point estimate 2.10 + 1 SE 1.42 ''
90     3.52, rounded
91     VARIANCE_CUTOFF = datetime.date(1636, 11, 12) # pre-spike variance
92     window ends here
93
94     os.makedirs("data", exist_ok=True)
95
96     # -----# 1. Load data
97     # -----
98
99     voc = pd.read_parquet("data/voc_returns.parquet")
100     tulip = pd.read_parquet("data/tulip_index.parquet")
101
102     voc["date"] = voc["date_ord"].apply(datetime.date.fromordinal)
103     tulip["date"] = tulip["date_ord"].apply(datetime.date.fromordinal)
104
105     voc_priced = voc[voc["price"].notna()].sort_values("date").copy()
106
107     print("=" * 65)
108     print("CAPM AND MVO ANALYSIS '' INV-paper")
109     print("Pre-crash cutoff: 1637-01-31 | Rf = 5.00% p.a.")
110     print("=" * 65)
111
112     # -----# 2. Market Risk Premium (
113     MRP)
114     #

```

```

106 # Use all VOC computable total returns (monthly, pre-crash window
    ).
107 # Annualise by *12 (returns are over ~1 month each).
108 # MRP = E(Rm) - Rf (annualised).
109 #
110 # With only 3 computable VOC returns, the CI will be very wide.
111 # Report honestly.
112 # -----
113 print("\n '5 .4 / '6 .1 MARKET RISK PREMIUM
    '5 .4 / '6 .1 MARKET RISK PREMIUM
    ")
114
115 voc_pre = voc[voc["simple_total_return"].notna() & (voc["date"] <=
    PRECRASH_END)].copy()
116
117 n_voc = len(voc_pre)
118 voc_monthly_returns = voc_pre["simple_total_return"].values
119
120 # Annualised mean and vol (monthly * 12 and * sqrt(12))
121 mrp_monthly_mean = voc_monthly_returns.mean()
122 mrp_monthly_std = voc_monthly_returns.std(ddof=1) if n_voc > 1 else
    np.nan
123
124 e_rm_annual = mrp_monthly_mean * 12
125 vol_rm_annual = mrp_monthly_std * (12**0.5)
126 mrp_annual = e_rm_annual - RF_ANNUAL
127
128 # Standard error of the mean, t-based CI
129 if n_voc > 1:
130     se_mrp = (mrp_monthly_std / (n_voc**0.5)) * 12
131     t_crit = stats.t.ppf(0.975, df=n_voc - 1)
132     mrp_lo = mrp_annual - t_crit * se_mrp
133     mrp_hi = mrp_annual + t_crit * se_mrp
134 else:
135     se_mrp = mrp_lo = mrp_hi = np.nan
136
137 print(f" VOC return observations (pre-crash): n = {n_voc}")
138 print(f" Individual monthly returns: {[f'{r:.4f}' for r in
    voc_monthly_returns]}")
139 print(
140     f" Mean monthly return: {mrp_monthly_mean:.4f} "
141     f"({mrp_monthly_mean * 12:.2%} annualised)"
142 )
143 print(
144     f" Std monthly return: {mrp_monthly_std:.4f} "
145     f"({vol_rm_annual:.2%} annualised)"
146     if not np.isnan(mrp_monthly_std)
147     else ""
148 )
149 print(f" E(Rm) annual: {e_rm_annual:.4f} ({
    e_rm_annual:.2%})")
150 print(f" Rf annual: {RF_ANNUAL:.2%}")
151 print(f" MRP = E(Rm) - Rf: {mrp_annual:.4f} ({mrp_annual
    :.2%})")
152 if not np.isnan(mrp_lo):
153     print(f" MRP 95% CI (t, n={n_voc}): [{mrp_lo:.2%}, {mrp_hi
    :.2%}]")

```

```

154 print(f"\n NOTE: n={n_voc} is extremely small. MRP estimate is
      illustrative.")
155 print(f" Damodaran modern equity ERP: 4 ‘‘7 %. Our estimate: {
      mrp_annual:.2%}."")
156
157 sigma_m_annual = vol_rm_annual # sigma of market, annualised
158 sigma_m_sq_ann = sigma_m_annual**2 # variance of market, annualised
159
160 # -----# 3. Build matched
      tulip‘‘VOC return pairs (pre-crash)
161 #
162 # For each tulip observation, compute the VOC total return over
      the
163 # same interval using carry-forward pricing. Only include pairs
      where
164 # the VOC price actually changed (non-zero VOC return) ‘‘ flat
      carry-
165 # forward intervals are not informative for beta estimation.
166 # -----
167 print("\n ‘‘‘ ‘5 .3 / ‘6 .1 BETA ESTIMATION
      .....)
      ")
168
169 tulip_pre = tulip[tulip["date"] <= PRECRASH_END].copy()
170 tulip_ret = tulip_pre[
171     tulip_pre["simple_return"].notna() & (tulip_pre["days_elapsed"]
      > 0)
172 ].copy()
173
174 pairs = []
175 for _, row in tulip_ret.iterrows():
176     t_date = row["date"]
177     t_ret = row["simple_return"]
178     t_days = int(row["days_elapsed"])
179     prior_date = datetime.date.fromordinal(t_date.toordinal() -
      t_days)
180
181     # Carry-forward: most recent VOC price at or before each
      boundary
182     voc_end = voc_priced[voc_priced["date"] <= t_date]
183     voc_start = voc_priced[voc_priced["date"] <= prior_date]
184
185     if voc_end.empty or voc_start.empty:
186         continue
187
188     p_end = voc_end.iloc[-1]["price"]
189     p_start = voc_start.iloc[-1]["price"]
190
191     # Add any dividends falling in the interval
192     div_window = voc[
193         (voc["date"] > prior_date)
194         & (voc["date"] <= t_date)
195         & (voc["dividend_raw"].notna())
196     ]["dividend"].sum()
197
198     voc_ret_interval = (p_end - p_start + div_window) / p_start
199
200     pairs.append(

```

```

201     {
202         "date": t_date,
203         "days": t_days,
204         "tulip_r": t_ret,
205         "voc_r": voc_ret_interval,
206         "voc_changed": abs(voc_ret_interval) > 1e-9,
207     }
208 )
209
210 pairs_df = pd.DataFrame(pairs)
211 pairs_nonzero = pairs_df[pairs_df["voc_changed"]].copy()
212
213 print(f" Tulip return observations (pre-crash):      {len(tulip_ret)}")
214 print(f" Matched pairs (all, incl. flat VOC):        {len(pairs_df)}")
215 print(f" Matched pairs (VOC price changed):          {len(
    pairs_nonzero)}")
216
217 if len(pairs_nonzero) >= 2:
218     x = pairs_nonzero["voc_r"].values
219     y = pairs_nonzero["tulip_r"].values
220
221     # OLS: y = alpha + beta * x
222     A = np.vstack([np.ones_like(x), x]).T
223     coeffs, _, _, _ = np.linalg.lstsq(A, y, rcond=None)
224     ols_alpha_const, ols_beta = coeffs
225
226     # Residuals and OLS standard errors
227     y_hat = A @ coeffs
228     resid = y - y_hat
229     n_pairs = len(x)
230     dof = n_pairs - 2
231     s2 = (resid @ resid) / dof if dof > 0 else np.nan
232     var_x = np.var(x, ddof=1)
233     se_beta = (s2 / ((n_pairs - 1) * var_x)) ** 0.5 if var_x > 0
        else np.nan
234     t_stat = ols_beta / se_beta if not np.isnan(se_beta) else np.nan
235     p_val = 2 * stats.t.sf(abs(t_stat), df=dof) if not np.isnan(
        t_stat) else np.nan
236     ci_beta = stats.t.ppf(0.975, df=dof) * se_beta if not np.isnan(
        se_beta) else np.nan
237     beta_lo = ols_beta - ci_beta if not np.isnan(ci_beta) else np.
        nan
238     beta_hi = ols_beta + ci_beta if not np.isnan(ci_beta) else np.
        nan
239
240     # R-squared
241     ss_tot = np.var(y, ddof=1) * (n_pairs - 1)
242     ss_res = resid @ resid
243     r_sq = 1 - ss_res / ss_tot if ss_tot > 0 else np.nan
244
245     print(f"\n OLS results (n={n_pairs} pairs with VOC price change
        ):")
246     print(f"      beta_hat:      {ols_beta:.4f}")
247     print(
248         f"      SE(beta):      {se_beta:.4f}"
249         if not np.isnan(se_beta)

```

```

250         else "      SE(beta):    n/a"
251     )
252     print(
253         f"      t-stat:      {t_stat:.4f}"
254         if not np.isnan(t_stat)
255         else "      t-stat:      n/a"
256     )
257     print(
258         f"      p-value:      {p_val:.4f}" if not np.isnan(p_val) else "
259         p-value:      n/a"
260     )
261     print(
262         f"      95% CI:      [{beta_lo:.4f}, {beta_hi:.4f}]"
263         if not np.isnan(beta_lo)
264         else "      95% CI:      n/a (insufficient df)"
265     )
266     print(
267         f"      R2 :      {r_sq:.4f}" if not np.isnan(r_sq) else "
268         R2 :      n/a"
269     )
270     print(f"\n NOTE: CI is very wide “ OLS point estimate is
271           noisy.")
272     print(f" Sensitivity analysis across beta = {BETA_SENSITIVITY}
273           is the")
274     print(f" operative approach per ‘5 .3.”)
275
276     beta_ols = ols_beta
277 else:
278     print(f" Insufficient pairs for OLS (n={len(pairs_nonzero)}).")
279     beta_ols = np.nan
280     ols_beta = se_beta = t_stat = p_val = beta_lo = beta_hi = r_sq =
281         np.nan
282     n_pairs = len(pairs_nonzero)
283
284 # Correlation (all matched pairs, not just non-zero VOC)
285 if len(pairs_df) >= 2:
286     rho_all = np.corrcoef(pairs_df["voc_r"], pairs_df["tulip_r"])[0,
287         1]
288     rho_nonzero = (
289         np.corrcoef(pairs_nonzero["voc_r"], pairs_nonzero["tulip_r"]
290             ))[0, 1]
291     if len(pairs_nonzero) >= 2
292     else np.nan
293 )
294     print(
295         f"\n Pearson correlation (all {len(pairs_df)} pairs):
296         "
297         f"rho = {rho_all:.4f}"
298     )
299     print(
300         f" Pearson correlation (VOC-changed {len(pairs_nonzero)}
301         pairs): "
302         f"rho = {rho_nonzero:.4f}"
303     )
304 else:
305     rho_all = rho_nonzero = np.nan

```

```

298 # -----# 4. Pre-crash tulip
    realised return
299 #
300 #   Compute the total index appreciation from the first tulip
    observation
301 #   (Dec 1634) to the last pre-crash observation (Jan 30 1637),
    expressed
302 #   as an annualised return for comparison with CAPM-implied
    returns.
303 # -----
304 print("\n '6.2 TULIP REALISED RETURN (PRE-CRASH)
    '")
305
306 tulip_sorted = tulip_pre.sort_values("date")
307 idx_first = tulip_sorted.iloc[0]["index_excl"]
308 idx_last_row = tulip_sorted[tulip_sorted["date"] <= PRECRASH_END].
    iloc[-1]
309 idx_last = idx_last_row["index_excl"]
310 date_first = tulip_sorted.iloc[0]["date"]
311 date_last = idx_last_row["date"]
312 days_total = (date_last - date_first).days
313 years_total = days_total / 365.25
314
315 # Cumulative simple return over the full pre-crash window
316 r_cum_pre = (idx_last - idx_first) / idx_first
317
318 # Annualised (geometric)
319 r_ann_pre = (1 + r_cum_pre) ** (1 / years_total) - 1 if years_total
    > 0 else np.nan
320
321 print(f" First observation: {date_first} index = {idx_first:.4f}"
    )
322 print(f" Last pre-crash obs: {date_last} index = {idx_last:.4f}")
323 print(f" Window: {days_total} days ({years_total:.2f}
    years)")
324 print(f" Cumulative return: {r_cum_pre:.4f} ({r_cum_pre:.2%})")
325 print(f" Annualised return: {r_ann_pre:.4f} ({r_ann_pre:.2%})")
326
327 # Full-cycle (including crash): last observation
328 idx_last_full = tulip.iloc[-1]["index_excl"]
329 date_last_full = tulip.iloc[-1]["date"]
330 days_full = (date_last_full - date_first).days
331 years_full = days_full / 365.25
332 r_cum_full = (idx_last_full - idx_first) / idx_first
333 r_ann_full = (1 + r_cum_full) ** (1 / years_full) - 1 if years_full
    > 0 else np.nan
334
335 print(f"\n Full-cycle last obs:{date_last_full} index = {
    idx_last_full:.4f}")
336 print(f" Full-cycle cumulative return: {r_cum_full:.4f} ({
    r_cum_full:.2%})")
337 print(f" Full-cycle annualised return: {r_ann_full:.4f} ({
    r_ann_full:.2%})")
338
339 # -----# 5. CAPM Alpha '
    sensitivity table
340 #
341 #   For each assumed beta, compute:

```

```

342 #      E(R_tulip) = Rf + beta * MRP      (CAPM-implied, annualised)
343 #      alpha_pre  = R_realised_pre - E(R_tulip)      (pre-crash)
344 #      alpha_full = R_realised_full - E(R_tulip)      (full-cycle)
345 # -----
346 print("\n '6.2 CAPM ALPHA ' SENSITIVITY TABLE
      '
      ")
347 print(f" Rf = {RF_ANNUAL:.2%} | MRP = {mrp_annual:.2%}")
348 print(f" Tulip realised (pre-crash ann.): {r_ann_pre:.2%}")
349 print(f" Tulip realised (full-cycle ann.): {r_ann_full:.2%}")
350 print()
351 print(f" {'beta':>6} {'E(R_tulip)':>12} {'alpha_pre':>12} {'
      alpha_full':>12}")
352 print(f" {' ' ' ' * 6} {' ' ' ' * 12} {' ' ' ' * 12} {' ' ' ' * 12}")
353
354 alpha_results = []
355 for beta in BETA_SENSITIVITY:
356     e_r_tulip = RF_ANNUAL + beta * mrp_annual
357     alpha_pre = r_ann_pre - e_r_tulip if not np.isnan(r_ann_pre)
358     else np.nan
359     alpha_full = r_ann_full - e_r_tulip if not np.isnan(r_ann_full)
360     else np.nan
361     print(
362         f" {beta:>6.1f} {e_r_tulip:>12.2%} {alpha_pre:>12.2%} {
363             alpha_full:>12.2%}"
364     )
365     alpha_results.append(
366         {
367             "beta": beta,
368             "e_r_tulip": e_r_tulip,
369             "alpha_pre": alpha_pre,
370             "alpha_full": alpha_full,
371         }
372     )
373
374 # OLS beta row if available
375 if not np.isnan(beta_ols):
376     e_r_ols = RF_ANNUAL + beta_ols * mrp_annual
377     a_pre = r_ann_pre - e_r_ols
378     a_full = r_ann_full - e_r_ols
379     print(
380         f" {'OLS':>6} {e_r_ols:>12.2%} {a_pre:>12.2%} {a_full
381             :>12.2%} "
382         f"(beta_hat = {beta_ols:.2f})"
383     )
384
385 # -----# 6. Single-factor
386 covariance matrix
387
388 #      K[i,j] = beta_i * beta_j * sigma_m^2 + delta_ij * sigma_eps_i^2
389 #
390 #      Assets: [tulips, VOC, bond]
391 #      Bond: beta=0, sigma_eps=0 => K[bond, *] = K[*, bond] = 0
392 #      VOC:  beta=1 by definition (it IS the market portfolio here)
393 #      sigma_eps_VOC ' 0 (no residual when it's the single
394 factor)
395 #      Tulip: beta = assumed value; sigma_eps estimated from residual
396 variance
397 #      from the matched pairs regression.

```


E COLLECTED SCRIPT SOURCES


```

487     K2_inv = np.linalg.inv(K2)
488     w_raw = K2_inv @ er
489     # Long-only constraint: if any weight is negative, clip to 0
490     # (no short-selling in 17th century Amsterdam)
491     w_raw = np.maximum(w_raw, 0)
492     w_sum = w_raw.sum()
493     if w_sum > 1e-10:
494         w = w_raw / w_sum
495     else:
496         w = np.array([0.5, 0.5]) # degenerate: equal weight
497
498     w_tulip, w_voc = w[0], w[1]
499
500     # Portfolio expected return and volatility (tangency)
501     mu_p = w @ mu2
502     sig_p = (w @ K2 @ w) ** 0.5
503     sharpe = (mu_p - RF_ANNUAL) / sig_p if sig_p > 0 else np.nan
504
505     print(
506         f" {beta:>5.1f} {e_r_tulip:>12.2%} {e_r_voc:>10.2%}"
507         f"{w_tulip:>9.2%} {w_voc:>9.2%} {sharpe:>8.3f}"
508     )
509
510     tangency_results.append(
511         {
512             "beta": beta,
513             "e_r_tulip": e_r_tulip,
514             "e_r_voc": e_r_voc,
515             "w_tulip": w_tulip,
516             "w_voc": w_voc,
517             "mu_p": mu_p,
518             "sigma_p": sig_p,
519             "sharpe": sharpe,
520         }
521     )
522
523 except np.linalg.LinAlgError:
524     print(
525         f" {beta:>5.1f} {e_r_tulip:>12.2%} {e_r_voc:>10.2%}"
526         f"[singular matrix ‘ cannot invert]"
527     )
528     tangency_results.append({"beta": beta, "error": "singular"})
529
530 # OLS beta if available
531 if not np.isnan(beta_ols):
532     e_r_ols = RF_ANNUAL + beta_ols * mrp_annual
533     K2_ols = build_covariance_matrix(beta_ols, sigma_m_sq_ann,
534                                     sigma_eps_sq_tulip)[
535         :2, :2
536     ]
537     er_ols = np.array([e_r_ols, e_r_voc]) - RF_ANNUAL
538     try:
539         w_raw_ols = np.linalg.inv(K2_ols) @ er_ols
540         w_raw_ols = np.maximum(w_raw_ols, 0)
541         ws = w_raw_ols.sum()

```

```

541     w_ols = w_raw_ols / ws if ws > 1e-10 else np.array([0.5,
542               0.5])
543     mu_ols = w_ols @ np.array([e_r_ols, e_r_voc])
544     sig_ols = (w_ols @ K2_ols @ w_ols) ** 0.5
545     sh_ols = (mu_ols - RF_ANNUAL) / sig_ols if sig_ols > 0 else
546         np.nan
547     print(
548         f" {'OLS':>5}  {e_r_ols:>12.2%}  {e_r_voc:>10.2%}  "
549         f"{w_ols[0]:>9.2%}  {w_ols[1]:>9.2%}  {sh_ols:>8.3f}  "
550         f"(beta={beta_ols:.2f})"
551     )
552 except np.linalg.LinAlgError:
553     pass
554
555 # -----# 7b. Sample-covariance
556 MVO (robustness check)
557 #
558 #     The single-factor covariance matrix yields 0% tulip allocation
559 #     because
560 #     the annualised idiosyncratic variance ( $\sigma_{\text{eps}}^2 = 1.36$ ) is
561 #     enormous
562 #     relative to the market variance ( $\sigma_{\text{m}}^2 = 0.025$ ). This is
563 #     an
564 #     artefact of annualising residuals from 4 interval-matched
565 #     pairs over
566 #     irregular intervals of days to months.
567 #
568 #     As a robustness check (flagged in '5.2), we estimate the
569 #     covariance
570 #     matrix directly from the aligned monthly return series. With
571 #     monthly
572 #     returns, the tulip variance is measured at the same frequency
573 #     as VOC,
574 #     producing a more balanced risk comparison.
575 #
576 #     We also sweep across assumed tulip betas for CAPM-implied
577 #     expected
578 #     returns, keeping the sample covariance for risk.
579 # -----
580 print("\n '6.3b SAMPLE-COVARIANCE MVO (ROBUSTNESS)
581       '")
582
583 aligned = pd.read_parquet("data/aligned_monthly.parquet")
584 pre_aligned = aligned[aligned["pre_crash"]].copy()
585
586 # Get months with both tulip and VOC returns
587 co_obs = pre_aligned[
588     pre_aligned["tulip_return"].notna() & pre_aligned["
589         voc_total_return"].notna()
590 ].copy()
591
592 print(f" Co-observed monthly returns (pre-crash): n = {len(co_obs)}
593       ")
594
595 # Even with carry-forward, we need tulip variance from all tulip
596 months
597 tulip_months = pre_aligned[pre_aligned["tulip_return"].notna()].copy
598 ()

```

```

583 voc_months = pre_aligned[pre_aligned["voc_total_return"].notna()].
      copy()
584
585 # Annualise monthly statistics
586 tulip_var_monthly = tulip_months["tulip_return"].var(ddof=1)
587 tulip_var_annual = tulip_var_monthly * 12
588 voc_var_monthly = voc_months["voc_total_return"].var(ddof=1)
589 voc_var_annual = voc_var_monthly * 12
590
591 # Covariance from co-observed months
592 if len(co_obs) >= 2:
593     cov_monthly = co_obs[["tulip_return", "voc_total_return"]].cov()
594     .values
595     cov_annual = cov_monthly * 12
596 else:
597     cov_annual = np.array([[tulip_var_annual, 0.0], [0.0,
598     voc_var_annual]])
599
600 # Use marginal variances on diagonal (more observations), covariance
601 from co-obs
602 K_sample = np.array(
603     [
604         [tulip_var_annual, cov_annual[0, 1]],
605         [cov_annual[1, 0], voc_var_annual],
606     ]
607 )
608
609 print(f"\n Tulip monthly returns: n = {len(tulip_months)}")
610 print(f" Var (monthly): {tulip_var_monthly:.6f}")
611 print(f" Var (annual): {tulip_var_annual:.4f} ( ' = {
612     tulip_var_annual**0.5:.2%})")
613 print(f" VOC monthly returns: n = {len(voc_months)}")
614 print(f" Var (monthly): {voc_var_monthly:.6f}")
615 print(f" Var (annual): {voc_var_annual:.4f} ( ' = {
616     voc_var_annual**0.5:.2%})")
617 print(f" Cov (annual): {cov_annual[0, 1]:.6f}")
618 print(
619     f" Correlation: {cov_annual[0, 1] / (tulip_var_annual**0.5
620     * voc_var_annual**0.5):.4f}"
621     if tulip_var_annual > 0 and voc_var_annual > 0
622     else " Correlation: n/a"
623 )
624
625 print(f"\n Sample covariance matrix K (annualised):")
626 print(f" Tulips VOC")
627 print(f" Tulips {K_sample[0, 0]:10.6f} {K_sample[0, 1]:10.6f}")
628 print(f" VOC {K_sample[1, 0]:10.6f} {K_sample[1, 1]:10.6f}")
629
630 # Also estimate pre-mania tulip variance (before Jan 1637 spike)
631 # An investor at the START of January 1637 would use data through
632 Dec 1636
633 pre_mania_cutoff = (1636, 12)
634 tulip_pre_mania = pre_aligned[
635     pre_aligned["tulip_return"].notna()
636     & (
637         (pre_aligned["year"] < pre_mania_cutoff[0])
638         | (
639             (pre_aligned["year"] == pre_mania_cutoff[0])

```

```

633         & (pre_aligned["month"] <= pre_mania_cutoff[1])
634     )
635 )
636 ]
637 if len(tulip_pre_mania) >= 2:
638     tulip_var_pre_mania_m = tulip_pre_mania["tulip_return"].var(ddof
        =1)
639     tulip_var_pre_mania_a = tulip_var_pre_mania_m * 12
640     print(f"\n Pre-mania tulip variance (up to Dec 1636):")
641     print(f"      n = {len(tulip_pre_mania)} monthly returns")
642     print(f"      Var (monthly): {tulip_var_pre_mania_m:.6f}")
643     print(
644         f"      Var (annual): {tulip_var_pre_mania_a:.4f} ( ' = {
            tulip_var_pre_mania_a**0.5:.2%})"
645     )
646
647     K_pre_mania = np.array([[tulip_var_pre_mania_a, 0.0], [0.0,
        voc_var_annual]])
648 else:
649     tulip_var_pre_mania_a = tulip_var_annual
650     K_pre_mania = K_sample
651
652 # Tangency portfolio with BOTH covariance estimates
653 sample_tangency_results = []
654 premania_tangency_results = []
655
656 for label, K_use, result_list in [
657     ("full pre-crash sample", K_sample, sample_tangency_results),
658     ("pre-mania (to Dec 1636)", K_pre_mania,
        premania_tangency_results),
659 ]:
660     print(f"\n Tangency portfolio ({label}):")
661     print(
662         f"      {'beta':>5}      {'E(R_tulip)':>12}      {'E(R_VOC)':>10}      "
663         f"      {'w_tulip':>9}      {'w_VOC':>9}      {'Sharpe':>8}"
664     )
665     print(f"      {' ' * 5}      {' ' * 12}      {' ' * 10}      {' ' * 9}
        {' ' * 9}      {' ' * 8}")
666
667     for beta in BETA_SENSITIVITY:
668         e_r_tulip = RF_ANNUAL + beta * mrp_annual
669         mu2 = np.array([e_r_tulip, e_r_voc])
670         er = mu2 - RF_ANNUAL
671
672         try:
673             K_inv = np.linalg.inv(K_use)
674             w_raw = K_inv @ er
675             w_raw = np.maximum(w_raw, 0) # long-only
676             w_sum = w_raw.sum()
677             if w_sum > 1e-10:
678                 w = w_raw / w_sum
679             else:
680                 w = np.array([0.5, 0.5])
681
682             mu_p = w @ mu2
683             sig_p = (w @ K_use @ w) ** 0.5
684             sharpe = (mu_p - RF_ANNUAL) / sig_p if sig_p > 0 else np
                .nan

```



```

732 dp_days = (dp_date_last - dp_date_first).days
733 dp_years = dp_days / 365.25
734 dp_r_cum = (dp_idx_last - dp_idx_first) / dp_idx_first
735 dp_r_ann = (1 + dp_r_cum) ** (1 / dp_years) - 1 if dp_years > 0 else
    np.nan
736
737 print(
738     f"\n Tulip index: {dp_date_first} ({dp_idx_first:.2f})  "
739     f"{dp_date_last} ({dp_idx_last:.2f})"
740 )
741 print(f" Window: {dp_days} days ({dp_years:.2f} yr)")
742 print(f" Cumulative return: {dp_r_cum:.2%}")
743 print(f" Annualised return: {dp_r_ann:.2%}")
744
745 # (b) Pre-spike tulip variance (5 returns from May 1636 to Nov 12
    1636)
746 # These are observations 3-7 in the tulip timeline: the "quiet"
    period
747 # before the Nov 25 doubling. Earlier returns (Oct/Dec 1635)
    are excluded
748 # because they predate the active trading phase and include the
    initial
749 # surge from Dec 1634  " Oct 1635 which is not representative
    of ongoing
750 # tulip volatility as the investor would perceive it.
751 VARIANCE_START = datetime.date(1636, 5, 1) # include returns from
    May 1636 onward
752 tulip_dp_ret = tulip_dp[
753     tulip_dp["simple_return"].notna()
754     & (tulip_dp["days_elapsed"] > 0)
755     & (tulip_dp["date"] >= VARIANCE_START)
756     & (tulip_dp["date"] <= VARIANCE_CUTOFF)
757 ]
758 dp_returns = tulip_dp_ret["simple_return"].values
759
760 dp_tulip_var_monthly = np.var(dp_returns, ddof=1)
761 dp_tulip_var_annual = dp_tulip_var_monthly * 12 # treat each as ~
    monthly
762
763 print(f"\n Pre-spike returns (for variance, up to {VARIANCE_CUTOFF
    }):")
764 for _, row in tulip_dp_ret.iterrows():
765     print(
766         f" {row['date']}: {row['simple_return']:+.4f}  "
767         f"({int(row['days_elapsed'])} days)"
768     )
769 print(f" n = {len(dp_returns)} returns")
770 print(f" Var (sample): {dp_tulip_var_monthly:.6f}")
771 print(
772     f" Var (annualised): {dp_tulip_var_annual:.4f}  "
773     f"(' = {dp_tulip_var_annual**0.5:.2%})"
774 )
775
776 # (c) CAPM alpha at beta = 3.5
777 dp_e_r_tulip = RF_ANNUAL + DINNER_PARTY_BETA * mrp_annual
778 dp_alpha = dp_r_ann - dp_e_r_tulip
779
780 print(f"\n CAPM expected return (beta={DINNER_PARTY_BETA}):")

```



```

781 print(
782     f"      E(R_tulip) = Rf + {DINNER_PARTY_BETA} ' MRP "
783     f"= {RF_ANNUAL:.2%} + {DINNER_PARTY_BETA} ' {mrp_annual:.2%} "
784     f"= {dp_e_r_tulip:.2%}"
785 )
786 print(f"      Realised:      {dp_r_ann:.2%}")
787 print(f"      Alpha:          {dp_alpha:.2%}")
788
789 # (d) Tangency portfolio
790 #      K = [[tulip_pre_spike_var_annual, 0], [0, voc_var_annual]]
791 #      mu = [Rf + 3.5 * MRP, Rf + 1.0 * MRP]
792 dp_K = np.array(
793     [
794         [dp_tulip_var_annual, 0.0],
795         [0.0, voc_var_annual],
796     ]
797 )
798 dp_mu = np.array([dp_e_r_tulip, e_r_voc])
799 dp_er = dp_mu - RF_ANNUAL
800
801 dp_K_inv = np.linalg.inv(dp_K)
802 dp_w_raw = dp_K_inv @ dp_er
803 dp_w_raw = np.maximum(dp_w_raw, 0) # long-only
804 dp_w_sum = dp_w_raw.sum()
805 dp_w = dp_w_raw / dp_w_sum if dp_w_sum > 1e-10 else np.array([0.5,
806     0.5])
807
808 dp_w_tulip, dp_w_voc = dp_w[0], dp_w[1]
809 dp_mu_p = dp_w @ dp_mu
810 dp_sig_p = (dp_w @ dp_K @ dp_w) ** 0.5
811 dp_sharpe = (dp_mu_p - RF_ANNUAL) / dp_sig_p if dp_sig_p > 0 else np
812     .nan
813
814 print(f"\n      Tangency portfolio (long-only):")
815 print(f"      Covariance matrix K (annualised):")
816 print(f"      Tulips      VOC")
817 print(f"      Tulips {dp_K[0, 0]:10.6f} {dp_K[0, 1]:10.6f}")
818 print(f"      VOC {dp_K[1, 0]:10.6f} {dp_K[1, 1]:10.6f}")
819 print(f"\n      Expected returns: tulip {dp_e_r_tulip:.2%} | VOC {
820     e_r_voc:.2%}")
821 print(f"      Weights: tulip {dp_w_tulip:.2%} | VOC {
822     dp_w_voc:.2%}")
823 print(f"      Portfolio E(R): {dp_mu_p:.2%}")
824 print(f"      Portfolio ' : {dp_sig_p:.2%}")
825 print(f"      Sharpe ratio: {dp_sharpe:.3f}")
826 print()
827 print(
828     f"      >>> DINNER-PARTY RESULT: CAPM recommends approximately "
829     f"      f"{dp_w_tulip:.0%} allocation to tulips."
830 )
831 print(f"      >>> The crash was 10 weeks later.")
832
833 dinner_party_results = {
834     "date": str(DINNER_PARTY_DATE),
835     "beta": DINNER_PARTY_BETA,
836     "variance_cutoff": str(VARIANCE_CUTOFF),
837     "n_pre_spike_returns": len(dp_returns),
838     "tulip_var_annual": float(dp_tulip_var_annual),

```



```

873 base_sc = next((r for r in sample_tangency_results if r.get("beta")
874 == 1.0), None)
875 if base_sc and "w_tulip" in base_sc:
876     print(f"    Full sample covariance:    tulip {base_sc['w_tulip']:.2%}")
877 # Pre-mania covariance
878 base_pm = next((r for r in premania_tangency_results if r.get("beta")
879 == 1.0), None)
880 if base_pm and "w_tulip" in base_pm:
881     print(f"    Pre-mania covariance:    tulip {base_pm['w_tulip']:.2%}")
882 # -----# 9. Write results to JSON
883 # -----
884 results = {
885     "parameters": {
886         "rf_annual": RF_ANNUAL,
887         "precrash_end": str(PRECRASH_END),
888         "crash_date": str(CRASH_DATE),
889     },
890     "mrp": {
891         "n_observations": n_voc,
892         "mrp_annual": mrp_annual,
893         "e_rm_annual": e_rm_annual,
894         "vol_rm_annual": vol_rm_annual,
895         "mrp_ci_lo": mrp_lo if not np.isnan(mrp_lo) else None,
896         "mrp_ci_hi": mrp_hi if not np.isnan(mrp_hi) else None,
897     },
898     "beta_ols": {
899         "n_pairs": n_pairs,
900         "beta_hat": float(ols_beta) if not np.isnan(ols_beta) else
901             None,
902         "se_beta": float(se_beta) if not np.isnan(se_beta) else None,
903         "t_stat": float(t_stat) if not np.isnan(t_stat) else None,
904         "p_value": float(p_val) if not np.isnan(p_val) else None,
905         "ci_lo": float(beta_lo) if not np.isnan(beta_lo) else None,
906         "ci_hi": float(beta_hi) if not np.isnan(beta_hi) else None,
907         "r_squared": float(r_sq) if not np.isnan(r_sq) else None,
908         "rho_all_pairs": float(rho_all) if not np.isnan(rho_all)
909             else None,
910         "rho_nonzero": float(rho_nonzero) if not np.isnan(
911             rho_nonzero) else None,
912     },
913     "tulip_returns": {
914         "r_cumulative_precrash": float(r_cum_pre) if not np.isnan(
915             r_cum_pre) else None,
916         "r_annualised_precrash": float(r_ann_pre) if not np.isnan(
917             r_ann_pre) else None,
918         "r_cumulative_fullcycle": float(r_cum_full)
919         if not np.isnan(r_cum_full)
920         else None,
921         "r_annualised_fullcycle": float(r_ann_full)
922         if not np.isnan(r_ann_full)
923         else None,
924     },
925 }

```

```

920     "alpha_sensitivity": alpha_results,
921     "tangency_single_factor": tangency_results,
922     "tangency_sample_covariance": sample_tangency_results,
923     "tangency_pre_mania": premania_tangency_results,
924     "dinner_party": dinner_party_results,
925     "sigma_m_sq_annual": sigma_m_sq_ann,
926     "sigma_eps_sq_tulip": sigma_eps_sq_tulip,
927     "sample_covariance": {
928         "tulip_var_annual": float(tulip_var_annual),
929         "voc_var_annual": float(voc_var_annual),
930         "cov_annual": float(cov_annual[0, 1]) if cov_annual.ndim >=
931             2 else None,
932         "K_sample": K_sample.tolist(),
933     },
934 }
935
936 # Convert any remaining NaN/inf to None for JSON
937 def clean(obj):
938     if isinstance(obj, float) and (np.isnan(obj) or np.isinf(obj)):
939         return None
940     if isinstance(obj, dict):
941         return {k: clean(v) for k, v in obj.items()}
942     if isinstance(obj, list):
943         return [clean(v) for v in obj]
944     return obj
945
946 results = clean(results)
947
948 with open("data/capm_results.json", "w") as f:
949     json.dump(results, f, indent=2)
950
951 print(f"\n Results written to data/capm_results.json")
952 print("=" * 65)

```

E.0.8 Diagnostics Output Script

```

1  """
2  appendix_k_diagnostics_outputs.py
3  -----
4  Produces standard diagnostic tables/figures for the CAPM + MVO
5  analysis.
6
7  Outputs:
8  data/input_summary_table.csv
9  data/covariance_matrix_risky_2x2.csv
10 data/covariance_matrix_risky_3x3.csv
11 data/pair_counts_matrix_3x3.csv
12 data/alpha_sensitivity_table.csv
13 data/weight_bootstrap_summary.csv
14 images/fig_efficient_frontier.png
15 images/fig_weight_bootstrap.png
16 images/fig_parameter_position_boxplots.png
17 images/fig_out_of_sample_path.png
18
19 Run with:

```

```

19     nix shell --impure --expr '(let pkgs = import <nixpkgs> {}; in
20         pkgs.python313.withPackages (ps: with ps; [numpy pandas
21             pyarrow matplotlib scipy]))' \
22     -c python appendix_k_diagnostics_outputs.py
23
24 import datetime
25 import json
26 import os
27
28 import matplotlib
29
30 matplotlib.use("Agg")
31 import matplotlib.pyplot as plt
32 import matplotlib.ticker as mticker
33 import numpy as np
34 import pandas as pd
35
36
37 # -----# Shared academic style (
38     matches appendix_h_figures.py)
39 # -----
40 plt.rcParams.update(
41     {
42         "font.family": "serif",
43         "font.size": 10,
44         "axes.titlesize": 12,
45         "axes.titleweight": "bold",
46         "axes.labelsize": 10,
47         "axes.spines.top": False,
48         "axes.spines.right": False,
49         "axes.grid": True,
50         "grid.alpha": 0.25,
51         "grid.linestyle": "--",
52         "grid.linewidth": 0.6,
53         "figure.dpi": 200,
54         "savefig.dpi": 200,
55         "savefig.bbox": "tight",
56         "legend.fontsize": 8.5,
57         "legend.framealpha": 0.85,
58         "legend.edgecolor": "#cccccc",
59         "xtick.labelsize": 9,
60         "ytick.labelsize": 9,
61     }
62 )
63
64 # Colour palette
65 VOC_BLUE = "#2166ac"
66 TULIP_RED = "#b2182b"
67 GREY = "#555555"
68 LIGHT_GREY = "#999999"
69 DINNER_GOLD = "#b8860b"
70 BOND_TEAL = "#35978f"
71 COMMODITY_GREEN = "#4daf4a"
72
73 RF = 0.05
74 CUTOFF = datetime.date(1636, 11, 25)
75 CRASH = datetime.date(1637, 2, 5)

```

```

75 N_BOOT = 5000
76
77
78 def pct_formatter(x, _):
79     return f"{x:.0%}"
80
81
82 def month_end(y: int, m: int) -> datetime.date:
83     if m == 12:
84         return datetime.date(y + 1, 1, 1) - datetime.timedelta(days
85                             =1)
86     return datetime.date(y, m + 1, 1) - datetime.timedelta(days=1)
87
88 def tangency_weights(mu: np.ndarray, cov: np.ndarray, rf: float) ->
89     np.ndarray:
90     ex = mu - rf
91     w_raw = np.linalg.inv(cov) @ ex
92     w_raw = np.maximum(w_raw, 0.0)
93     s = w_raw.sum()
94     if s <= 1e-12:
95         return np.repeat(1.0 / len(mu), len(mu))
96     return w_raw / s
97
98 def portfolio_stats(
99     w: np.ndarray, mu: np.ndarray, cov: np.ndarray
100 ) -> tuple[float, float, float]:
101     mu_p = float(w @ mu)
102     sig_p = float(np.sqrt(w @ cov @ w))
103     sharpe = float((mu_p - RF) / sig_p) if sig_p > 0 else np.nan
104     return mu_p, sig_p, sharpe
105
106
107 def main() -> None:
108     os.makedirs("data", exist_ok=True)
109     os.makedirs("images", exist_ok=True)
110
111     capm = json.load(open("data/capm_results.json"))
112     capm_com = json.load(open("data/capm_results_with_commodities.
113                               json"))
114     aligned = pd.read_parquet("data/aligned_monthly.parquet")
115     tulip = pd.read_parquet("data/tulip_index.parquet")
116
117     aligned = aligned.copy()
118     aligned["month_end"] = aligned.apply(
119         lambda r: month_end(int(r["year"]), int(r["month"])), axis=1
120     )
121
122     dp = capm["dinner_party"]
123     w2 = np.array([dp["w_tulip"], dp["w_voc"]], dtype=float)
124     mu2 = np.array([dp["e_r_tulip"], dp["e_r_voc"]], dtype=float)
125     cov2 = np.array(
126         [[dp["tulip_var_annual"], 0.0], [0.0, dp["voc_var_annual"]
127         ]], dtype=float
128     )
129
130     m3 = capm_com["covariance_annual"]["matrix"]

```

```

129 cov3 = np.array(m3, dtype=float)
130 mu3 = np.array(
131     [
132         capm_com["means_annual"]["tulip"],
133         capm_com["means_annual"]["voc"],
134         capm_com["means_annual"]["commodity_broad"],
135     ],
136     dtype=float,
137 )
138 w3 = np.array(
139     [
140         capm_com["tangency_long_only"]["w_tulip"],
141         capm_com["tangency_long_only"]["w_voc"],
142         capm_com["tangency_long_only"]["w_commodity_broad"],
143     ],
144     dtype=float,
145 )
146
147 # ----- # 1) Input summary
148 # ----- input_rows = [
149     ["Cutoff date", str(CUTOFF), "Dinner-party decision date"],
150     ["Risk-free rate", RF, "Lekdijk bond coupon"],
151     ["MRP", capm["mrp"]["mrp_annual"], "From 3 VOC total return
152         observations"],
153     ["Beta (adopted)", dp["beta"], "OLS point estimate + 1 SE"],
154     ["Tulip expected return", dp["e_r_tulip"], "CAPM implied"],
155     ["Tulip realised return", dp["r_realised_ann"], "To cutoff"
156         ],
157     ["Tulip alpha", dp["alpha"], "Realised minus CAPM implied"],
158     ["Tulip variance", dp["tulip_var_annual"], "Pre-spike window
159         (5 returns)"],
160     ["VOC variance", dp["voc_var_annual"], "From VOC monthly
161         returns"],
162     ["Tulip-VOC covariance", 0.0, "Set to zero"],
163     ["Tangency tulip weight", dp["w_tulip"], "2-asset risky set"
164         ],
165     ["Tangency VOC weight", dp["w_voc"], "2-asset risky set"],
166     ["Tangency portfolio return", dp["mu_p"], "Expected annual
167         return"],
168     ["Tangency portfolio volatility", dp["sigma_p"], "Annual
169         volatility"],
170     ["Tangency Sharpe", dp["sharpe"], "(E[R]-Rf)/sigma"],
171 ]
172 pd.DataFrame(input_rows, columns=["parameter", "value", "note"])
173 .to_csv(
174     "data/input_summary_table.csv", index=False
175 )
176
177 # 2) Covariance/correlation tables
178 pd.DataFrame(cov2, index=["tulip", "voc"], columns=["tulip", "
179     voc"]).to_csv(
180     "data/covariance_matrix_risky_2x2.csv"
181 )
182 pd.DataFrame(
183     cov3,
184     index=["tulip", "voc", "commodity"],
185     columns=["tulip", "voc", "commodity"],

```

```

177     ).to_csv("data/covariance_matrix_risky_3x3.csv")
178
179     pair_counts = pd.DataFrame(
180         [
181             [8, 8, capm_com["covariance_annual"]["pair_counts"]["tulip_commodity"]],
182             [8, 8, capm_com["covariance_annual"]["pair_counts"]["voc_commodity"]],
183             [
184                 capm_com["covariance_annual"]["pair_counts"]["tulip_commodity"],
185                 capm_com["covariance_annual"]["pair_counts"]["voc_commodity"],
186                 capm_com["means_annual"]["commodity_obs_n"],
187             ],
188         ],
189         index=["tulip", "voc", "commodity"],
190         columns=["tulip", "voc", "commodity"],
191     )
192     pair_counts.to_csv("data/pair_counts_matrix_3x3.csv")
193
194     # 3) Alpha table
195     pd.DataFrame(capm["alpha_sensitivity"]).to_csv(
196         "data/alpha_sensitivity_table.csv", index=False
197     )
198
199     # ----- # 4) Efficient
200     Frontier “ polished with CML, annotations, % axes print("Generating
201     # ----- fig_efficient_frontier.png ...)
202
203     ws = np.linspace(0, 1, 401)
204     vols, rets = [], []
205     for wt in ws:
206         w = np.array([wt, 1 - wt])
207         mu_p, sig_p, _ = portfolio_stats(w, mu2, cov2)
208         vols.append(sig_p)
209         rets.append(mu_p)
210
211     mu2_t, sig2_t, sh2_t = portfolio_stats(w2, mu2, cov2)
212     mu3_t, sig3_t, sh3_t = portfolio_stats(w3, mu3, cov3)
213     mu_voc = mu2[1]
214     sig_voc = np.sqrt(cov2[1, 1])
215
216     fig, ax = plt.subplots(figsize=(8, 5.5))
217
218     # Efficient frontier curve
219     ax.plot(
220         vols,
221         rets,
222         color=VOC_BLUE,
223         lw=2,
224         label="Efficient frontier (Tulip + VOC)",
225         zorder=2,
226     )
227
228     # Capital Market Line from Rf through tangency
229     cml_sig = np.linspace(0, max(vols) * 0.7, 100)

```



```

229     cml_ret = RF + sh2_t * cml_sig
230     ax.plot(
231         cml_sig,
232         cml_ret,
233         color=GREY,
234         lw=1.5,
235         linestyle="--",
236         alpha=0.6,
237         label=f"Capital Market Line (Sharpe={sh2_t:.2f})",
238         zorder=1,
239     )
240
241     # Risk-free rate point
242     ax.scatter(
243         [0],
244         [RF],
245         color=BOND_TEAL,
246         s=70,
247         zorder=4,
248         marker="s",
249         edgecolors="white",
250         linewidths=0.6,
251         label=f"$R_f$ = {RF:.0%}",
252     )
253
254     # VOC-only point
255     ax.scatter(
256         [sig_voc],
257         [mu_voc],
258         color=GREY,
259         s=65,
260         zorder=4,
261         edgecolors="white",
262         linewidths=0.6,
263         label=f"VOC only ({mu_voc:.1%}, $\sigma$={sig_voc:.1%})",
264     )
265     ax.annotate(
266         "VOC only",
267         xy=(sig_voc, mu_voc),
268         xytext=(sig_voc + 0.025, mu_voc - 0.04),
269         fontsize=8,
270         color=GREY,
271         arrowprops=dict(arrowstyle="->", color=GREY, lw=0.7),
272     )
273
274     # 2-asset tangency
275     ax.scatter(
276         [sig2_t],
277         [mu2_t],
278         color=TULIP_RED,
279         s=80,
280         zorder=4,
281         edgecolors="white",
282         linewidths=0.6,
283         label=f"Tangency: {dp['w_tulip']:.1%} tulip, {dp['w_voc']:.1%} VOC",
284     )
285     ax.annotate(

```

```

286         f"Tangency\n{dp['w_tulip']:.1%} tulip\nSharpe={sh2_t:.2f}",
287         xy=(sig2_t, mu2_t),
288         xytext=(sig2_t + 0.04, mu2_t + 0.06),
289         fontsize=8,
290         color=TULIP_RED,
291         fontweight="bold",
292         arrowprops=dict(arrowstyle="->", color=TULIP_RED, lw=0.8),
293         bbox=dict(boxstyle="round,pad=0.2", fc="white", ec=TULIP_RED
294             , alpha=0.8),
295     )
296
297     # 3-asset tangency with commodities
298     ax.scatter(
299         [sig3_t],
300         [mu3_t],
301         color=COMMODITY_GREEN,
302         s=80,
303         zorder=4,
304         edgecolors="white",
305         linewidths=0.6,
306         label=f"3-asset tangency (incl. commodities)",
307     )
308     ax.annotate(
309         f"3-asset\nSharpe={sh3_t:.2f}",
310         xy=(sig3_t, mu3_t),
311         xytext=(sig3_t - 0.06, mu3_t + 0.05),
312         fontsize=8,
313         color=COMMODITY_GREEN,
314         fontweight="bold",
315         arrowprops=dict(arrowstyle="->", color=COMMODITY_GREEN, lw
316             =0.8),
317     )
318
319     ax.set_xlabel("Volatility ($\\sigma$, annual)")
320     ax.set_ylabel("Expected return (annual)")
321     ax.xaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter
322         ))
323     ax.yaxis.set_major_formatter(mticker.FuncFormatter(pct_formatter
324         ))
325     ax.set_title(
326         "Efficient Frontier and Tangency Portfolios",
327         pad=10,
328     )
329     ax.text(
330         0.5,
331         1.01,
332         "Dinner-party parameters; long-only constraint; $R_f$=5%",
333         transform=ax.transAxes,
334         ha="center",
335         fontsize=9,
336         color=GREY,
337         style="italic",
338     )
339     ax.legend(fontsize=7.5, loc="lower right")
340     ax.set_xlim(-0.01, max(vols) * 0.65)
341     ax.set_ylim(RF - 0.05, max(rets) * 0.7)
342
343     plt.tight_layout()

```

```

340 plt.savefig("images/fig_efficient_frontier.png")
341 plt.close()
342 print("    Saved.")
343
344 # ----- # 5) Bootstrap
345 # ----- print("Generating
346 # ----- fig_weight_bootstrap.png ...")
347
348 rng = np.random.default_rng(1636)
349 tulip_vals = np.array([0.233, 0.172, -0.117, -0.102, 0.240])
350 voc_vals = (
351     aligned[aligned["month_end"] <= CUTOFF]["voc_total_return"].
352     dropna().values
353 )
354 mrp_vals = voc_vals * 12.0 - RF
355 betas = rng.normal(
356     loc=capm["beta_ols"]["beta_hat"],
357     scale=capm["beta_ols"]["se_beta"],
358     size=N_BOOT,
359 )
360 rf_vals = rng.uniform(0.04, 0.06, size=N_BOOT)
361
362 w_t_boot = []
363 var_t_draw = []
364 mrp_draw = []
365 beta_draw = []
366 for i in range(N_BOOT):
367     t = rng.choice(tulip_vals, size=len(tulip_vals), replace=
368         True)
369     v = rng.choice(voc_vals, size=len(voc_vals), replace=True)
370     var_t = np.var(t, ddof=1) * 12.0
371     var_v = np.var(v, ddof=1) * 12.0
372     cv = 0.0
373     b = max(0.2, betas[i])
374     m = rng.choice(mrp_vals)
375     mu_t = rf_vals[i] + b * m
376     mu_v = rf_vals[i] + m
377     c = np.array([[max(1e-6, var_t), cv], [cv, max(1e-6, var_v)
378         ]])
379     w = tangency_weights(np.array([mu_t, mu_v]), c, rf_vals[i])
380     w_t_boot.append(float(w[0]))
381     var_t_draw.append(var_t)
382     mrp_draw.append(m)
383     beta_draw.append(b)
384
385 w_t_boot = np.array(w_t_boot)
386 w_df = pd.DataFrame(
387     {
388         "w_tulip_mean": [float(np.mean(w_t_boot))],
389         "w_tulip_median": [float(np.median(w_t_boot))],
390         "w_tulip_p05": [float(np.quantile(w_t_boot, 0.05))],
391         "w_tulip_p95": [float(np.quantile(w_t_boot, 0.95))],
392         "point_estimate": [float(dp["w_tulip"])],
393     }
394 )
395 w_df.to_csv("data/weight_bootstrap_summary.csv", index=False)

```

```

393 fig, ax = plt.subplots(figsize=(8, 4.8))
394
395 # Use more bins and clip extreme outliers for readability
396 bins = np.linspace(0, 100, 80)
397 ax.hist(
398     w_t_boot * 100,
399     bins=bins,
400     color=VOC_BLUE,
401     alpha=0.7,
402     edgecolor="white",
403     linewidth=0.3,
404     zorder=2,
405 )
406 ax.axvline(
407     dp["w_tulip"] * 100,
408     color=TULIP_RED,
409     lw=2.5,
410     linestyle="--",
411     label=f"Point estimate ({dp['w_tulip']:.1%})",
412     zorder=3,
413 )
414
415 # Confidence interval
416 p5 = np.quantile(w_t_boot, 0.05) * 100
417 p95 = np.quantile(w_t_boot, 0.95) * 100
418 ax.axvspan(p5, p95, alpha=0.08, color=DINNER_GOLD, zorder=0)
419 ax.axvline(p5, color=DINNER_GOLD, lw=1, linestyle=":", alpha
420           =0.7)
421 ax.axvline(p95, color=DINNER_GOLD, lw=1, linestyle=":", alpha
422           =0.7)
423 ax.text(
424     (p5 + p95) / 2,
425     ax.get_ylim()[1] * 0.85 if ax.get_ylim()[1] > 10 else 10,
426     f"90% CI: [{p5:.0f}%, {p95:.0f}%]",
427     ha="center",
428     fontsize=8,
429     color=DINNER_GOLD,
430     fontweight="bold",
431 )
432
433 # Median line
434 med = np.median(w_t_boot) * 100
435 ax.axvline(
436     med,
437     color=COMMODITY_GREEN,
438     lw=1.5,
439     linestyle="-.",
440     label=f"Median ({med:.1f}%)",
441     zorder=3,
442 )
443
444 ax.set_xlabel("Tulip tangency weight (%)")
445 ax.set_ylabel("Bootstrap frequency")
446 ax.set_title(
447     "Bootstrap Distribution of Recommended Tulip Weight",
448     pad=10,
449 )
450 ax.text(

```

```

449         0.5,
450         1.01,
451         f"$n$={N_BOOT:}, draws; beta, variance, MRP all resampled",
452         transform=ax.transAxes,
453         ha="center",
454         fontsize=9,
455         color=GREY,
456         style="italic",
457     )
458     ax.legend(fontsize=8, loc="upper right")
459
460     plt.tight_layout()
461     plt.savefig("images/fig_weight_bootstrap.png")
462     plt.close()
463     print("    Saved.")
464
465     # ----- # 6) Parameter
466     # ----- print("Generating
467     position boxplots ' polished
468     fig_parameter_position_boxplots.png ...")
469
470     fig, axes = plt.subplots(1, 4, figsize=(13, 4.8), sharey=False)
471     panels = [
472         (beta_draw, dp["beta"], "Beta ($\\beta$)", ""),
473         (mrp_draw, capm["mrp"]["mrp_annual"], "Market Risk\\nPremium",
474          "%"),
475         (var_t_draw, dp["tulip_var_annual"], "Tulip Variance\\n(
476          annual)", ""),
477         (list(w_t_boot), dp["w_tulip"], "Tulip Weight\\n(tangency)",
478          "%"),
479     ]
480
481     for ax, (dist, point, title, fmt) in zip(axes, panels):
482         bp = ax.boxplot(
483             dist,
484             vert=True,
485             widths=0.45,
486             patch_artist=True,
487             boxprops={"facecolor": "#d1e5f0", "edgecolor": VOC_BLUE,
488                      "linewidth": 0.8},
489             medianprops={"color": DINNER_GOLD, "linewidth": 1.5},
490             whiskerprops={"color": VOC_BLUE, "linewidth": 0.8},
491             capprops={"color": VOC_BLUE, "linewidth": 0.8},
492             flierprops={
493                 "marker": "o",
494                 "markerfacecolor": LIGHT_GREY,
495                 "markersize": 3,
496                 "alpha": 0.4,
497             },
498         )
499         ax.scatter(
500             [1],
501             [point],
502             color=TULIP_RED,
503             s=60,
504             zorder=5,
505             edgecolors="white",
506             linewidths=0.8,

```

```

501         marker="D",
502     )
503     # Label the chosen point
504     if fmt == "%":
505         label = f"{point:.1%}"
506     else:
507         label = f"{point:.2f}"
508     ax.annotate(
509         f"Chosen: {label}",
510         xy=(1, point),
511         xytext=(1.25, point),
512         fontsize=7.5,
513         color=TULIP_RED,
514         fontweight="bold",
515         arrowprops=dict(arrowstyle="->", color=TULIP_RED, lw
                    =0.7),
516     )
517     ax.set_title(title, fontsize=10, fontweight="bold")
518     ax.set_xticks([])
519     if fmt == "%":
520         ax.yaxis.set_major_formatter(mticker.FuncFormatter(
                    pct_formatter))
521
522     fig.suptitle(
523         "Chosen Inputs vs Bootstrap Ranges",
524         fontsize=13,
525         fontweight="bold",
526         y=1.02,
527     )
528     fig.text(
529         0.5,
530         -0.02,
531         "Red diamond = adopted point estimate; box = interquartile
                    range of bootstrap draws",
532         ha="center",
533         fontsize=9,
534         color=GREY,
535         style="italic",
536     )
537     fig.tight_layout()
538     fig.savefig("images/fig_parameter_position_boxplots.png")
539     plt.close(fig)
540     print("    Saved.")
541
542     # ----- # 7) Out-of-sample
543     path `` polished print("Generating
                    fig_out_of_sample_path.png ...")
544
545     tulip = tulip.copy()
546     tulip["date"] = tulip["date_ord"].apply(datetime.date.
                    fromordinal)
547     t = tulip.sort_values("date")
548     t = t[t["date"] >= datetime.date(1636, 5, 1)]
549     base = t.iloc[0]["index_excl"]
550     t["rebased"] = t["index_excl"] / base * 100
551
552     fig, ax = plt.subplots(figsize=(8.5, 4.8))

```

```

553
554 # Shade pre-decision and post-decision regions
555 ax.axvspan(
556     t["date"].iloc[0],
557     CUTOFF,
558     alpha=0.04,
559     color=VOC_BLUE,
560     zorder=0,
561     label="In-sample (pre-decision)",
562 )
563 ax.axvspan(
564     CUTOFF,
565     t["date"].iloc[-1],
566     alpha=0.04,
567     color=TULIP_RED,
568     zorder=0,
569     label="Out-of-sample (post-decision)",
570 )
571
572 # Line with markers at actual observation dates
573 ax.plot(
574     t["date"],
575     t["rebased"],
576     color=TULIP_RED,
577     lw=1.6,
578     zorder=2,
579 )
580 # Pre-cutoff markers
581 pre = t[t["date"] <= CUTOFF]
582 post = t[t["date"] > CUTOFF]
583 ax.scatter(
584     pre["date"],
585     pre["rebased"],
586     color=DINNER_GOLD,
587     s=30,
588     zorder=3,
589     edgecolors="white",
590     linewidths=0.5,
591     label="Pre-decision observations",
592 )
593 ax.scatter(
594     post["date"],
595     post["rebased"],
596     color=TULIP_RED,
597     s=30,
598     zorder=3,
599     edgecolors="white",
600     linewidths=0.5,
601     marker="D",
602     label="Post-decision observations",
603 )
604
605 # Decision date
606 ax.axvline(
607     CUTOFF,
608     color=DINNER_GOLD,
609     linestyle="-. ",
610     lw=1.8,

```

```

611         zorder=4,
612     )
613     ylim = ax.get_ylim()
614     ax.text(
615         CUTOFF - datetime.timedelta(days=2),
616         ylim[1] * 0.93 if ylim[1] > 100 else 380,
617         "Decision date\n25 Nov 1636",
618         ha="right",
619         va="top",
620         fontsize=8.5,
621         color=DINNER_GOLD,
622         fontweight="bold",
623     )
624
625     # Crash date
626     ax.axvline(
627         CRASH,
628         color=GREY,
629         linestyle="--",
630         lw=1.4,
631         zorder=4,
632     )
633     ax.text(
634         CRASH + datetime.timedelta(days=1),
635         ylim[1] * 0.93 if ylim[1] > 100 else 380,
636         "Crash auction\n5 Feb 1637",
637         ha="left",
638         va="top",
639         fontsize=8.5,
640         color=GREY,
641         style="italic",
642     )
643
644     # Baseline
645     ax.axhline(100, color=LIGHT_GREY, linewidth=0.6, alpha=0.5,
646               zorder=0)
647
648     ax.set_title(
649         "Pre-Decision Recommendation vs Out-of-Sample Outcome",
650         pad=10,
651     )
652     ax.text(
653         0.5,
654         1.01,
655         "CAPM said buy. The crash was 10 weeks later.",
656         transform=ax.transAxes,
657         ha="center",
658         fontsize=9,
659         color=GREY,
660         style="italic",
661     )
662     ax.set_ylabel("Index (rebased to 100)")
663     ax.set_xlabel("Date")
664     ax.legend(fontsize=8, loc="upper left")
665
666     plt.tight_layout()
667     plt.savefig("images/fig_out_of_sample_path.png")
668     plt.close()

```



```
668     print("  Saved.")
669
670     print("\nWrote diagnostics tables to data/")
671     print("Wrote diagnostics figures to images/")
672
673
674 if __name__ == "__main__":
675     main()
```